NXPの最新マイコン MCX搭載の開発ボードを使って、次世代シリアルバス"I3C"を動かしてみよう！

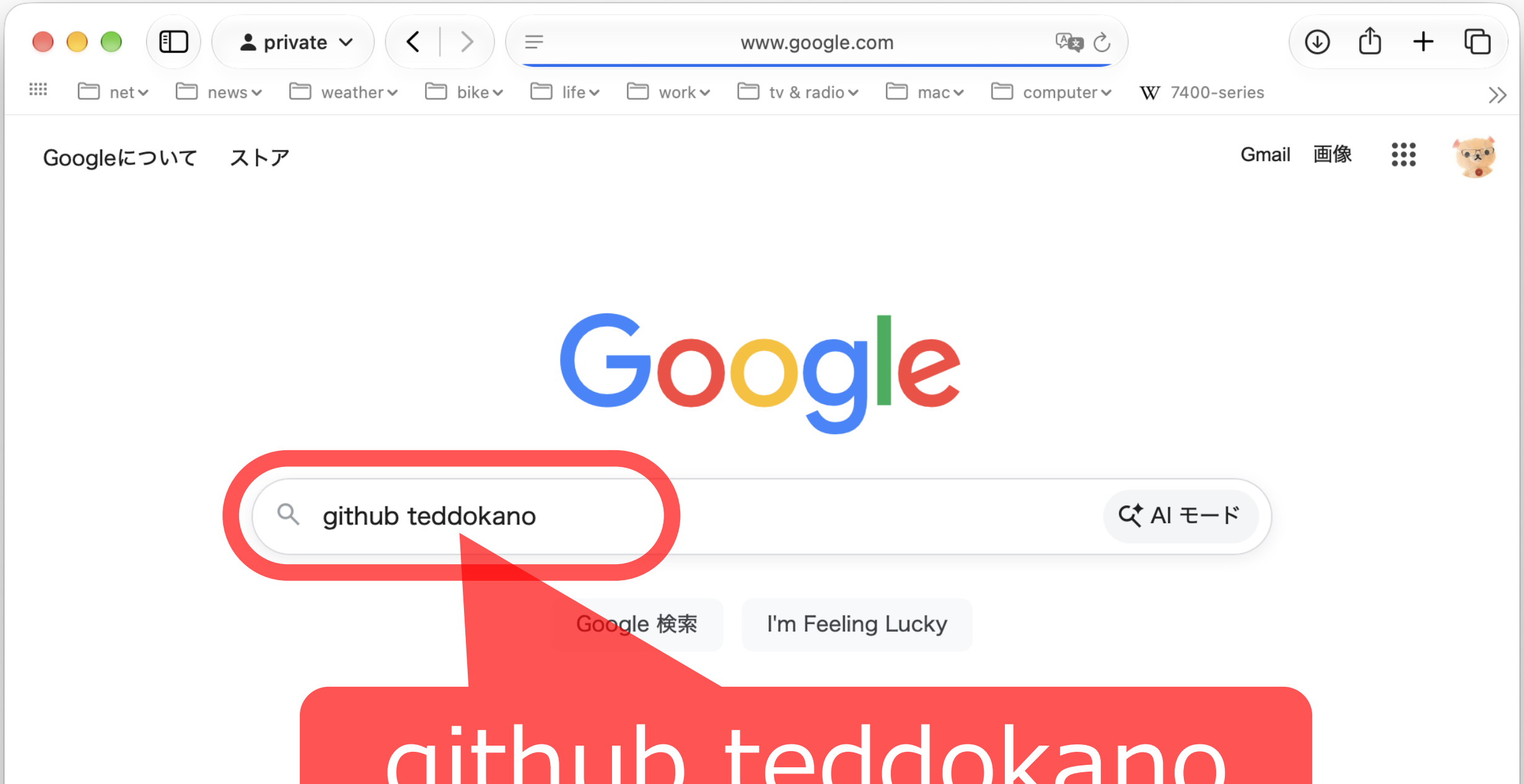
岡野 彰文

2026年10月16日 @共立電子セミナールーム

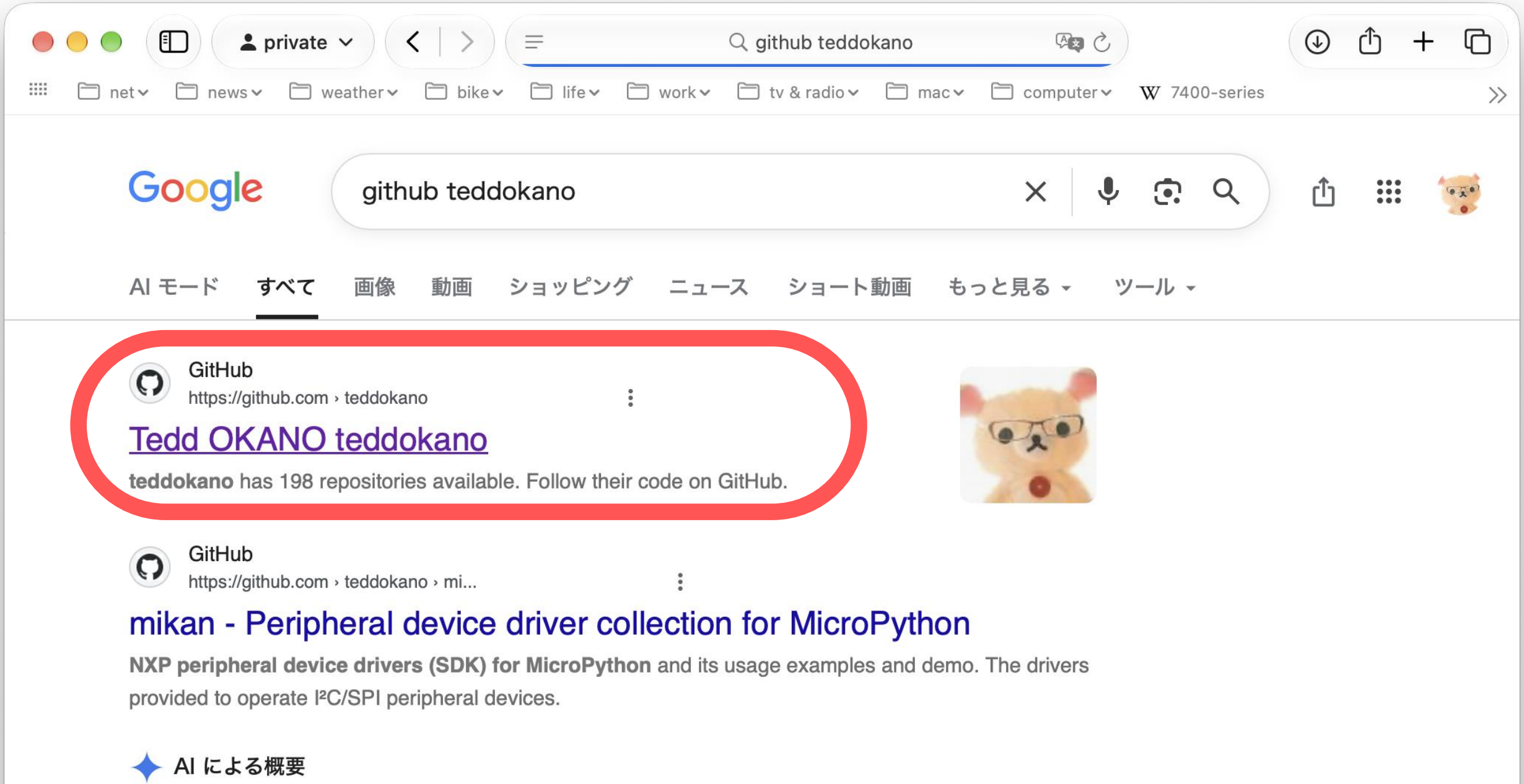
| **Public** | NXP and the NXP logo are trademarks of NXP B.V. All other product or service names are the property of their respective owners. © 2026 NXP B.V. Version 3.0.

NXP Public

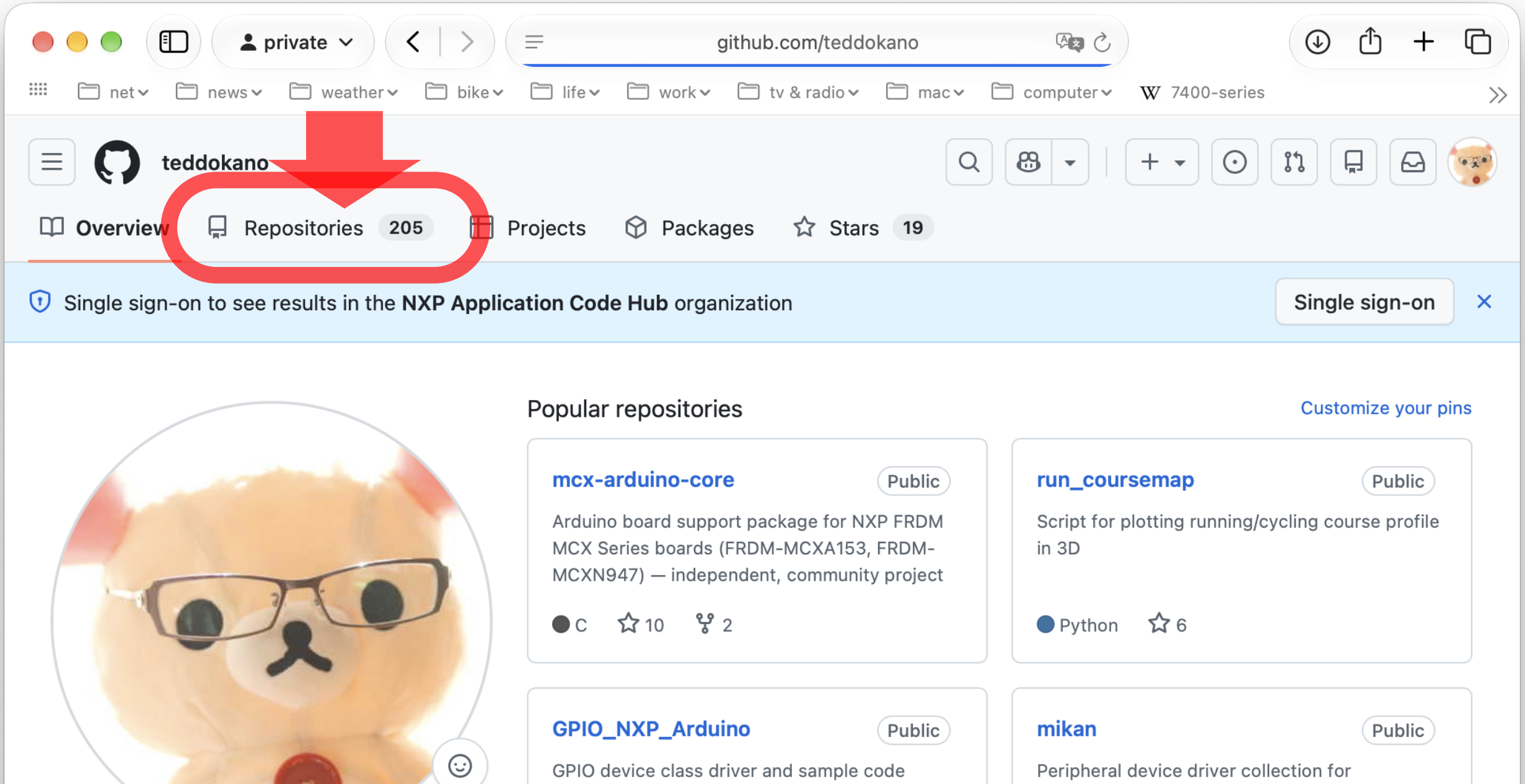
この資料の置き場所 (1/5)



この資料の置き場所 (2/5)



この資料の置き場所 (3/5)



この資料の置き場所 (4/5)

private

github.com/teddokano?tab=repositories

net news weather bike life work tv & radio mac computer 7400-series

teddokano

Overview Repositories 205 Projects Packages Stars 19

Single sign-on to see results in the **NXP Application Code Hub** organization

Single sign-on



Find a repository... Type Language Sort New

NXP-MCX-I3C- Public

共立電子セミナー10月16日資料共有

Updated 2 minutes ago

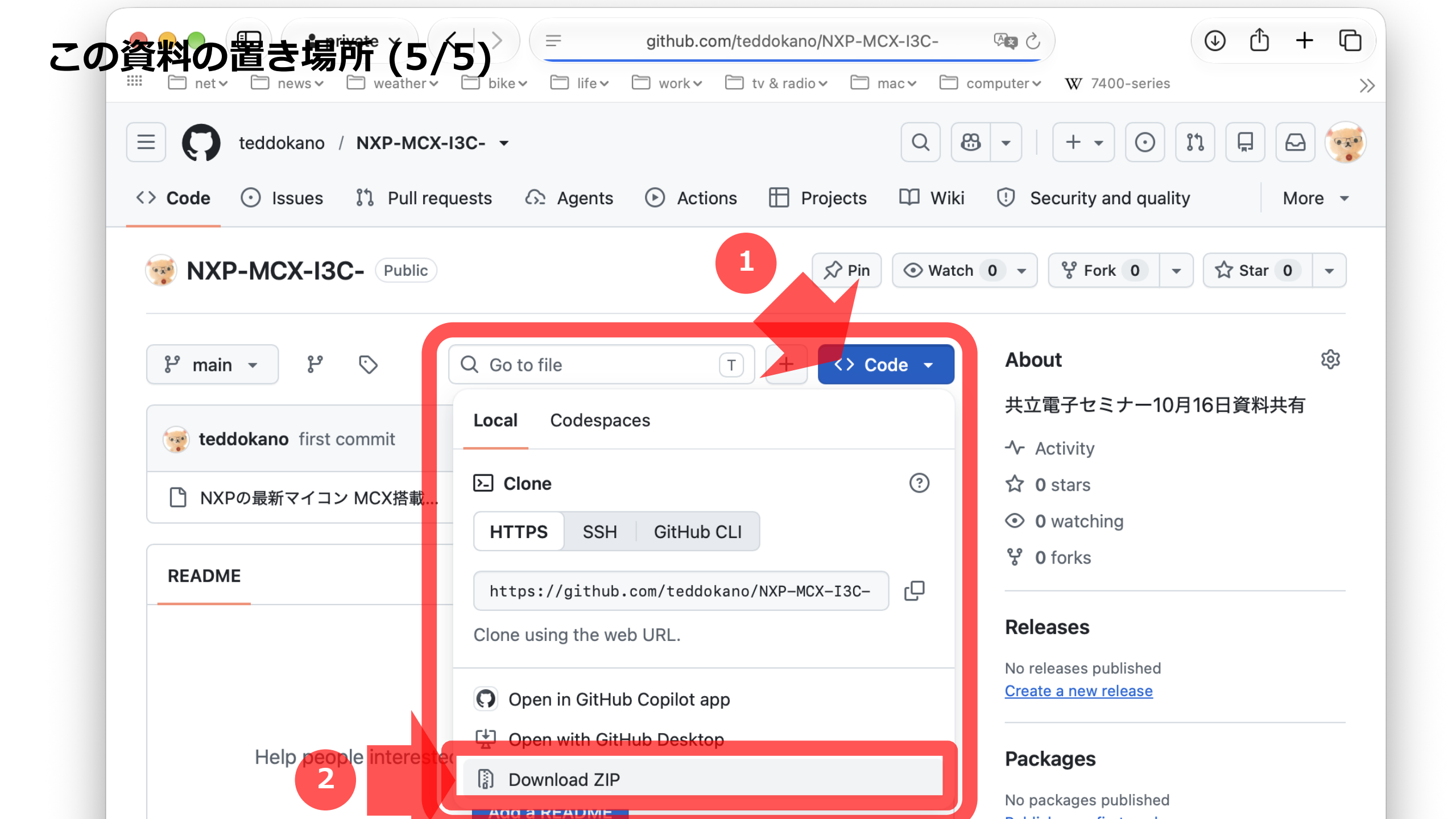
Star

mcx-arduino-core Public

Arduino board support package for NXP FRDM

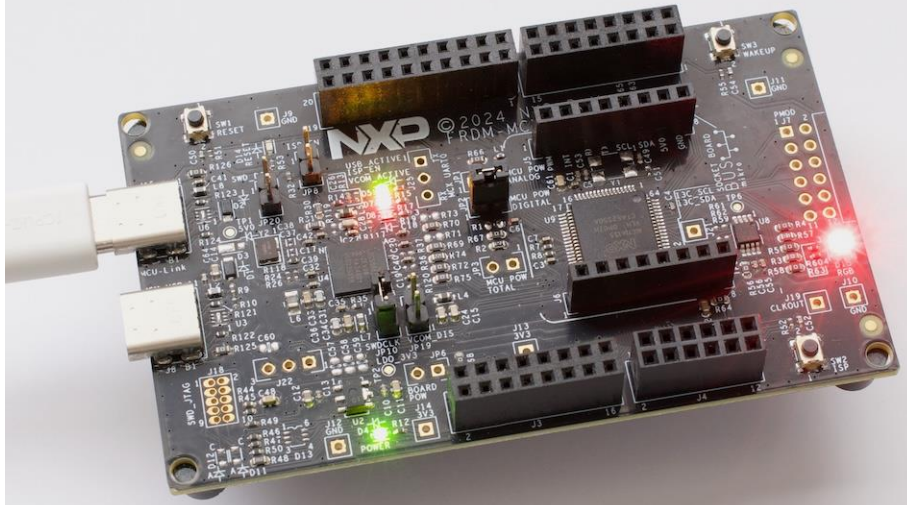
Starred

この資料の置き場所 (5/5)



きょう 話すこと/やってみること

- MCXマイコンってなに？
- I3Cってなに？
- Arduino IDE/SDKを使ってFRDM-MCXA153を動かしてみる
  - 環境のインストール
    - Arduino IDEのインストール
    - LinkServerのインストール
    - mcx-Arduino-coreのインストール
  - 動作確認
    - Lチカ
  - I3Cサンプルコードの動作
    - P3T1755温度センサからの値を読む
    - 波形デモ
    - コード例の解説

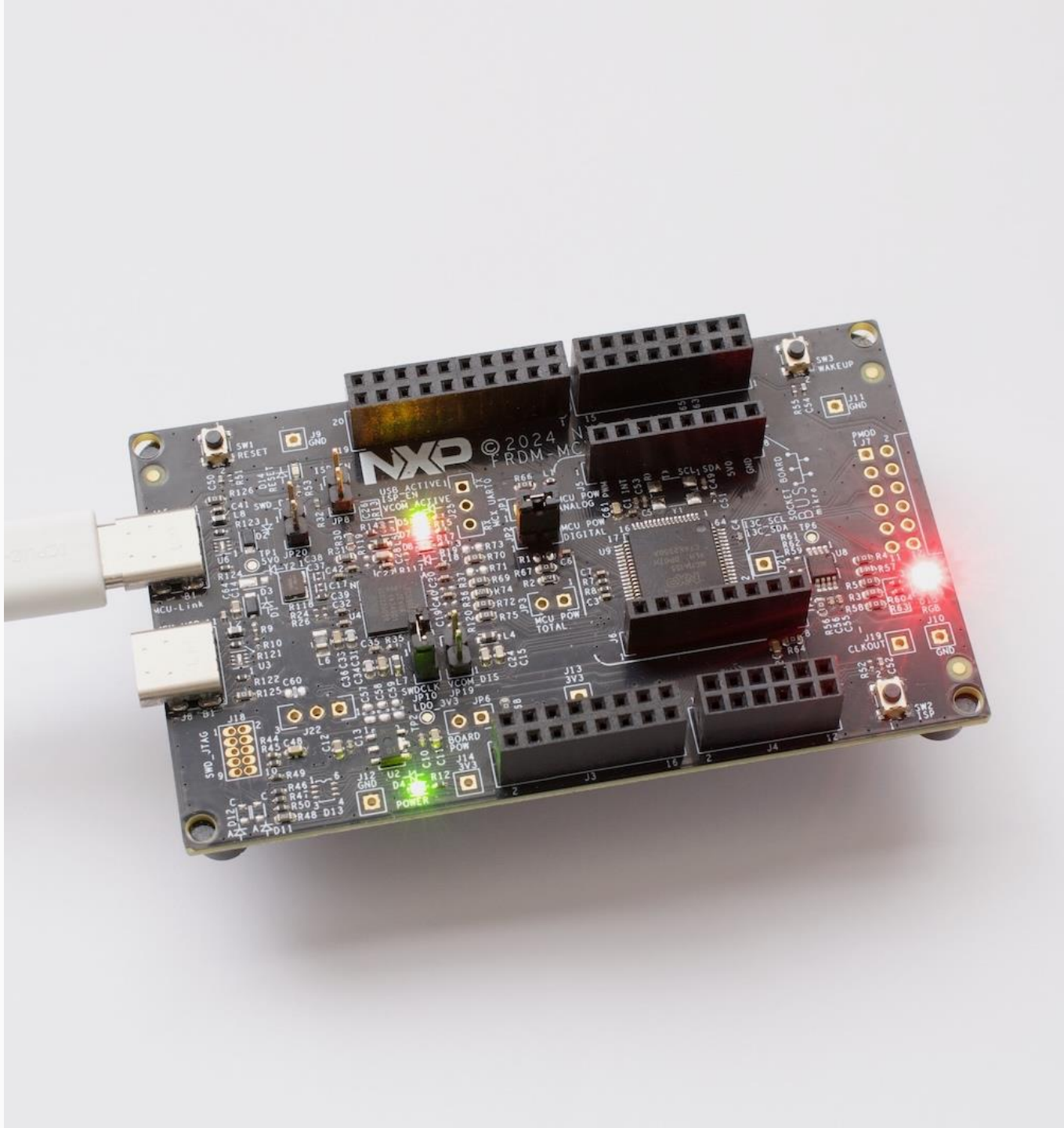


NXP Public

01

I3Cってなに？

MIPI I3Cバスの紹介



I3C ↔ I²C

アイ・スリー・シー

Improved Inter Integrated Circuit

アイ・スクエアド・シー

Inter Integrated Circuit = IIC = I²C

https://www.**mipi.org**/specifications/i3c-sensor-specification

英語 rev7.0 : https://www.**nxp.com**/search?keyword=um10204

日本語 rev5.0J : https://www.**nxp.com**/docs/ja/user-guide/UM10204.pdf

NXP Public

01-1

MIPI I3Cの紹介

なぜI3C？

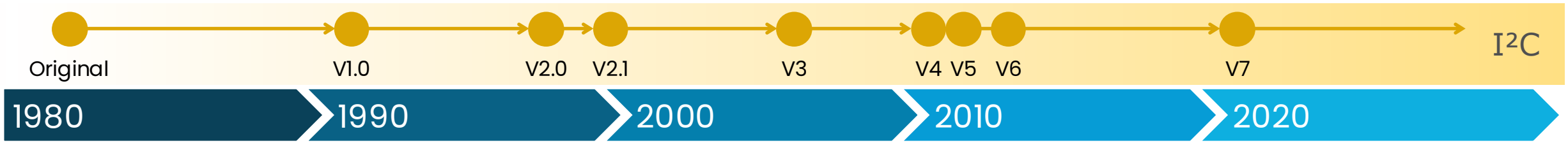
背景

- NXP (旧フィリップス) が I²C仕様を管理
- 1982年に発明．1992年に公開
- 制御と通信．特にセンサに多く使われている

なぜI3C？

背景

- NXP (旧フィリップス) が I²C仕様を管理
- 1982年に発明．1992年に公開
- 制御と通信．特にセンサに多く使われている



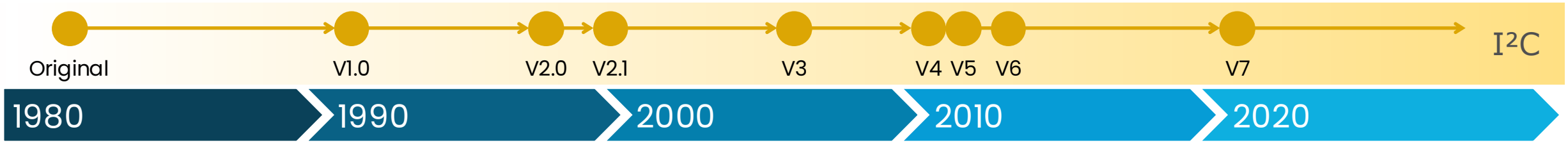
なぜI3C？

背景

- NXP (旧フィリップス) が I²C仕様を管理
- 1982年に発明．1992年に公開
- 制御と通信．特にセンサに多く使われている

課題とゴール

- センサの数が増える → 通信帯域幅の限界
- GPIOが多く必要 → ピン数が足らない
- MIPI I3C アップグレードを2013年に開始
- ゴール: インバンド割込, 動的アドレッシング, 高帯域幅, 省電力化



なぜI3C？

背景

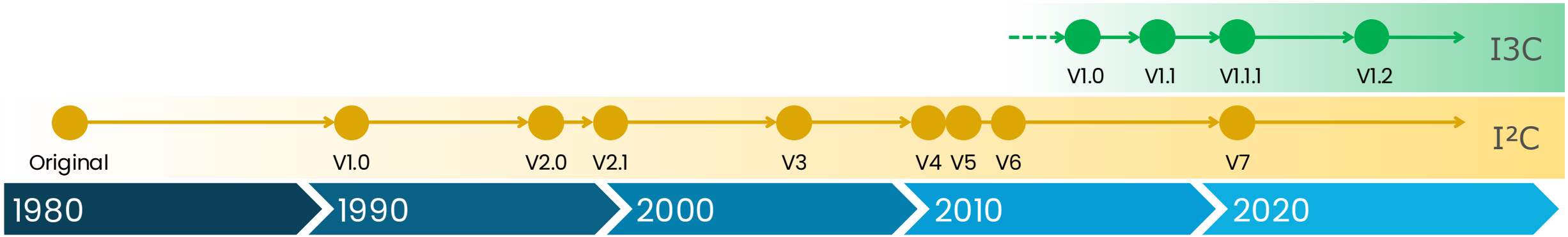
- NXP (旧フィリップス) が I²C仕様を管理
- 1982年に発明．1992年に公開
- 制御と通信．特にセンサに多く使われている

MIPI I3C Spec

- 2016年リリース
- 最新版 v1.2: 2025
- 主要策定社: NXP, Qualcomm, Intel
- 策定参加社: Invensense, TI, STM, Synopsys, Cadence, Mentor, Sony, Knowles, Lattice

課題とゴール

- センサの数が増える → 通信帯域幅の限界
- GPIOが多く必要 → ピン数が足らない
- MIPI I3C アップグレードを2013年に開始
- ゴール: インバンド割込, 動的アドレッシング, 高帯域幅, 省電力化



MIPI I3C = I²Cからできた次世代シリアルバス

- I²Cに続くMIPI I3C
  - 使用法，電力，性能の面で大きな改善
  - 中速度(~30MHz)のSPIの置き換えも狙う
- 背景
  - I²CはNXP(旧Philips)が開発．仕様を管理，リーダー的立場
  - I²Cは特にセンサの制御・通信インターフェースとして大きなシェア（MIPIの2013の調査では90%超）
  - MIPIアライアンス：センサ・インターフェース・ワークグループが2013年に策定開始
- 策定の理由
  - モバイル機器に使われるセンサの増加．In-band interrupt (インバンド割込)によりSoCに接続されるGPIO線の削減
  - データ量増加に対するI²C帯域幅限界の克服
  - アップグレードの容易性/使いやすさの維持 – 簡潔性の維持：(SPIにはない)マルチドロップ接続，後方互換性
    - 普及までの時期， I²CからMIPI I3Cへ，スムーズな移行を促す
- MIPI I3C仕様策定協力会社
  - Primary Spec authoring: NXP (Paul Kimelman), Qualcomm, Intel
  - Other contributors: Invensense, TI, STM, Synopsys, Cadence, Mentor, Sony, Knowles, Lattice

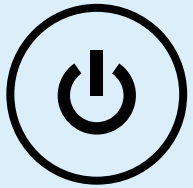
I3C

compatible with I²C



高スループット

プッシュプル信号を用いてより多くのデータ転送を可能に
12.5Mbps Standard Data Rate (SDR) と ～30Mbps High Data Rate (HDR)



高電力効率

オープンドレインからプッシュプルへの信号駆動方法変更で，
データ転送単位あたり最大約66%の電力効率を改善



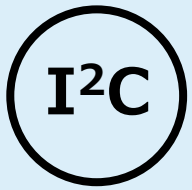
インバンド割込 (In-band Interrupt: IBI)

追加ピンなしでの通知機能.
周辺からコアへの割込通知は，2本のバス線で行える



動的アドレス割当

あらかじめハードウェアで設定しておかなくても，
各周辺デバイスには初期化時にアドレスが割り当てられる



I²C互換

I3CはI²Cとの互換性を維持.
同一バス上で両方のプロトコルの共存が可能

I3Cが，すでに多く使われているアプリケーション



多くのセンサを使うスマートフォン/ウェアラブル・デバイス



サーバ，PCなど (DDR5ではI3Cは必須)

NXP Public

MIPI I3C仕様書

https://www.mipi.org/specifications/i3c-sensor-specification

特徴	I3C v1.0	I3C Basic	I3C v1.1	I3C Basic v1.1
12.5 MHz SDR (コントローラ、ターゲット、およびレガシー I²C ターゲットの互換性)	√	√	√	√
ターゲットは、I²C バスおよび HDR モードを使用する I3C バス上で I²C デバイスとして動作可能	√	√	√	√
ターゲットリセット			√	√
1.2V-3.3V 50pf C 負荷での動作仕様	√	√	√	√
インバンド割り込み (w/MDB)	√	√	√	√
動的アドレス割当て	√	√	√	√
エラー検出と復帰	√	√	√	√
セカンダリ・コントローラ	√	√	√	√
ホットジョイン機構	√	√	√	√
共通コマンドコード (必須／オプション)	√	√	√	√
1.0V 動作 (100pf C 負荷での動作仕様)		√		√
スタティック・アドレスをダイナミック・アドレスとして設定する CCC (SETAASA)		√	√	√
同期タイミング制御	√		√	
非同期タイミング制御 (モード 0)	√		√	√
非同期タイミング制御 (モード 1-3)	√		√	
HDR-DDR	√		√	√
HDR-TSL/TSP	√		√	
HDR-BT (マルチレーンバルク転送)			√	√
グループ化アドレッシング			√	√
デバイス間トンネリング			√	
高速化のためのマルチレーン (SDR および HDR-DDR ではデュアル / クアッド)			√	
モニタリング・デバイス EaryTermination			√	

- I3C 仕様の**フルバージョン**
  - MIPI アライアンスのメンバーのみに提供
- I3C **Basic**
  - 開発者や他の標準化団体も利用できるよう一般的に必要とされるI3C の機能をまとめた公開バージョン
  - MIPI の会員でなくてもロイヤリティフリーのライセンス下で実装することを可能に

参考資料：「システム・マネジメント I2C, I3C, SPI セレクタ・ガイド」

https://www.nxp.com/docs/ja/product-selector-guide/I2CSELECTORBROC.pdf



最新仕様書はv1.2

v1.2を含む仕様比較表は→ https://www.mipi.org/hubfs/Specification-Feature-Tables/MIPI-I3C-and-I3C-Basic-Version-History-Table-030625.pdf

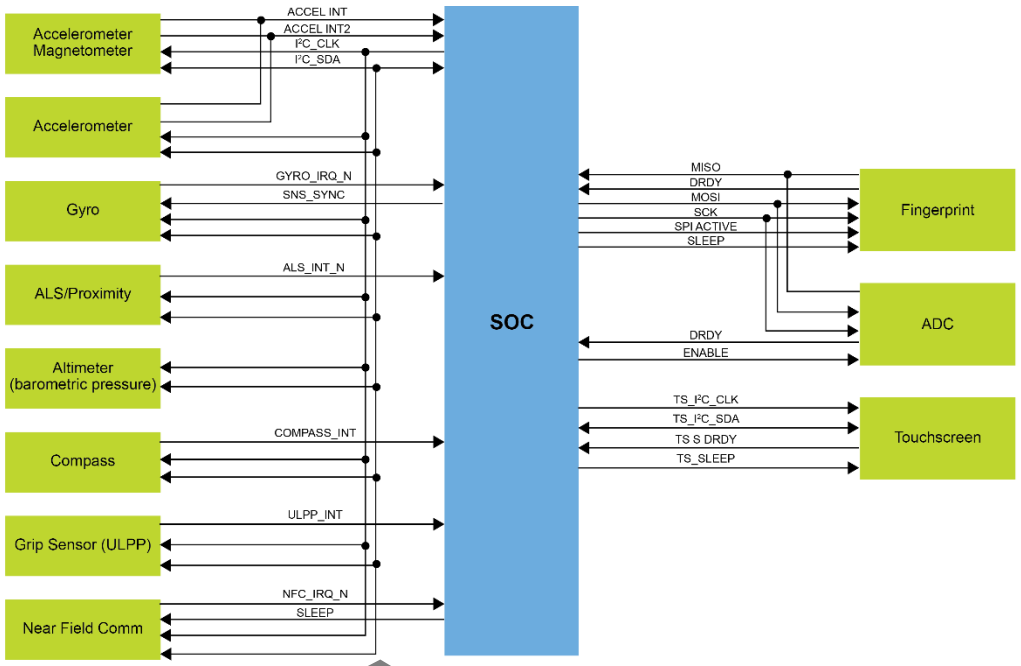
01-2

I3C vs I²C (&SPI) 利点とユースケース

センサ・インターフェース ブロック図

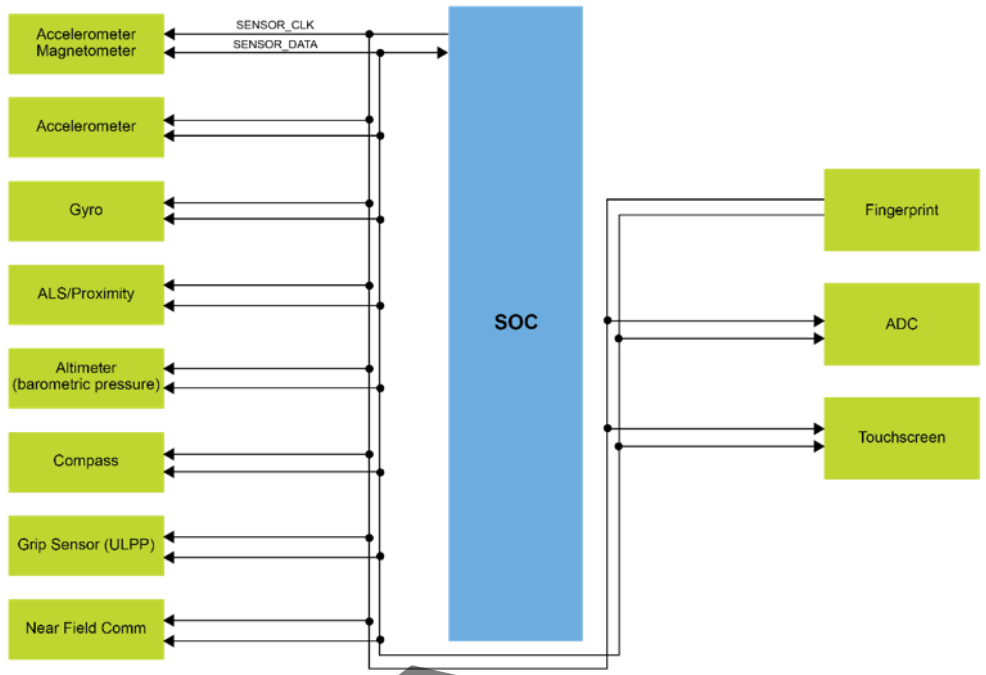
- ターゲットの増加でコスト高に．データレートの制限に加え，多くの場合，割込やイネーブル/リセット，スリープなどの追加ピン/配線が必要になる
- GPIOの追加がシステム・コストを上げる．SoCのピン数と基板レイヤ数の増加による

現在のかたち



I²CとSPI デバイスにはENやRSTn, INTの追加信号が必要

理想のかたち



MIPI I3Cではインバンド割込， Common Command Codesでデバイスを制御

NXP Public

01-3

MIPI I3C 信号とプロトコル



01-3a

MIPI I3C 信号とプロトコル

通常の転送

I3Cメッセージの概要

MIPI I3C SDR は I²C とほぼ同じ

例：Write data

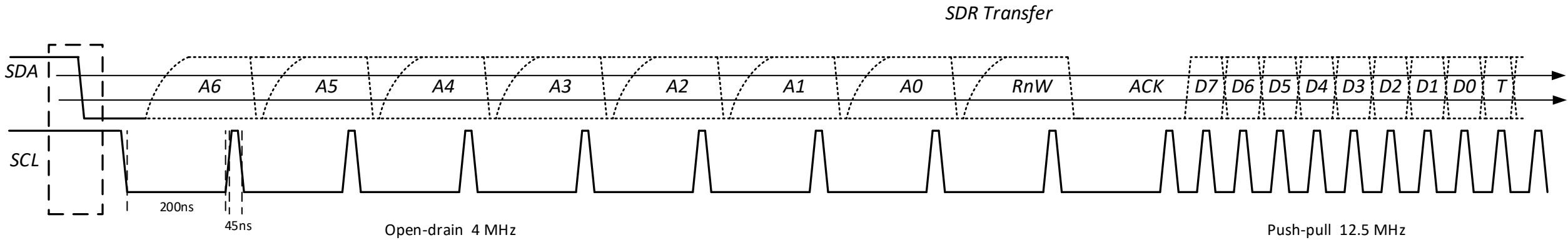
	1 bit	8 bits	1 bit	8 bits	1 bit	…	1 bit
MIPI I3C	S or Sr	Addr+W	ACK/ NACK	1 Byte data	T bit = Parity	More data	Sr or P
I²C	same	same	same	same	ACK/ NACK	same	same

例：Read data (typical approach)

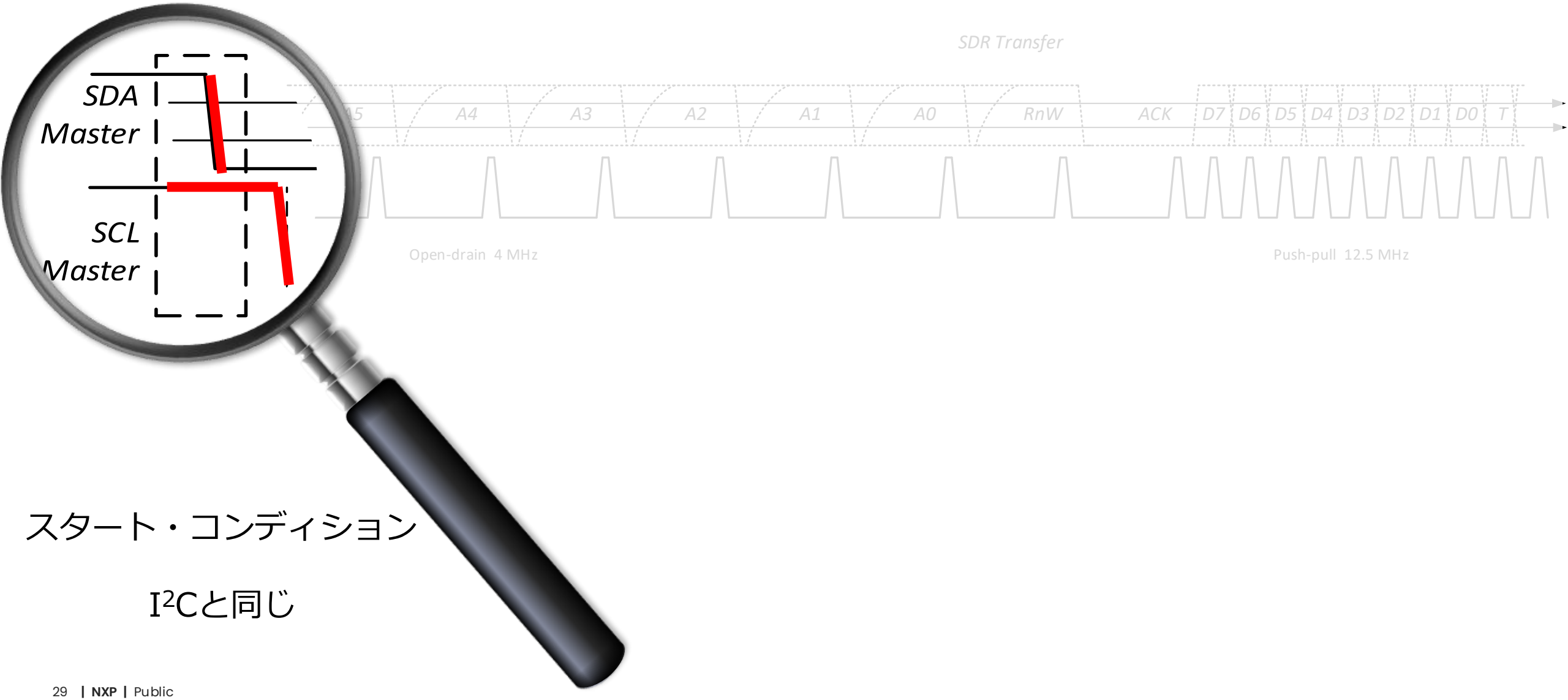
	1 bit	8 bits	1 bit	8 bits	1 bit	1 bit	8 bit	1 bit	8 bit	1 bit	…	1 bit
MIPI I3C	S or Sr	Addr +W	ACK / NACK	1 Byte data	T bit = Parity	Sr	Addr + R	ACK / NACK	1 Byte from Slave	T bit = ‘1 then Z’ to continue T bit = ‘0’ Slave ends transmission	More data	Sr or P
I²C	same	same	same	same	ACK/ NACK	same	same	same	same	ACK/ NACK **Slave can’t abort read**	same	Master ends read

NXP Public

I3C信号 (SDR モード, 動的アドレス割当後)



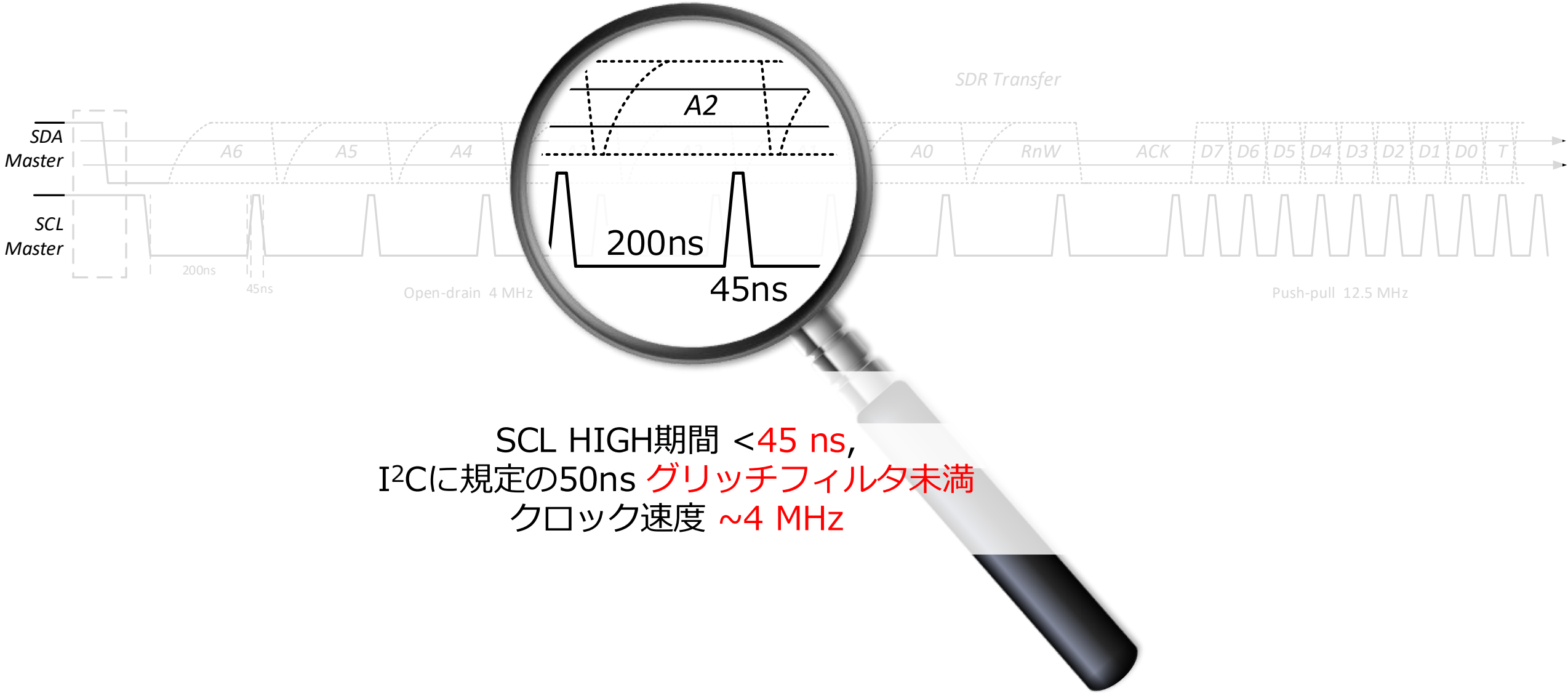
I3C信号 (SDR モード，動的アドレス割当後)



スタート・コンディション

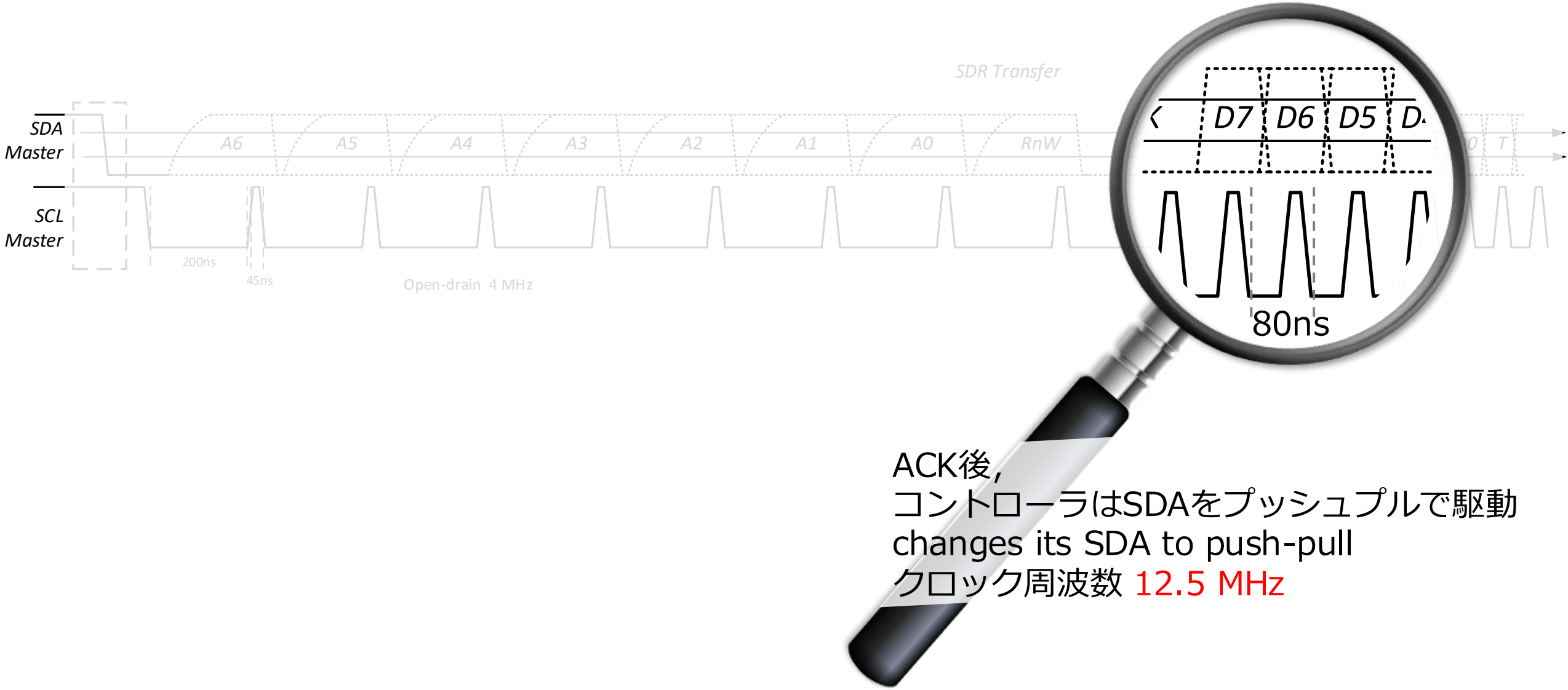
I²Cと同じ

I3C信号 (SDR モード, 動的アドレス割当後)



SCL HIGH期間 <45 ns,
I²Cに規定の50ns グリッチフィルタ未満
クロック速度 ~4 MHz

I3C信号 (SDR モード， 動的アドレス割当後)



ACK後，
コントローラはSDAをプッシュプルで駆動
changes its SDA to push-pull
クロック周波数 12.5 MHz

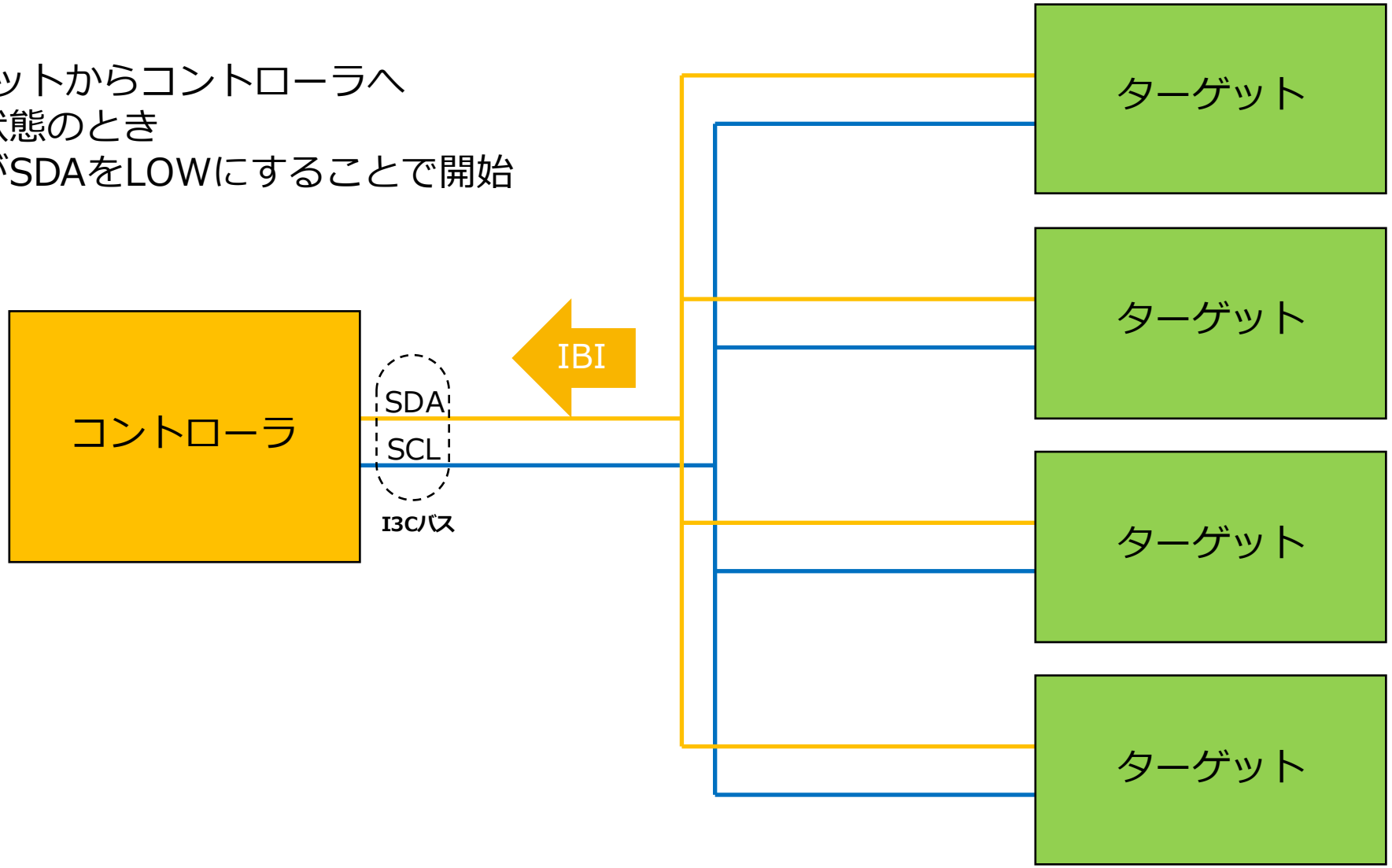
01-3b

MIPI I3C 信号とプロトコル

インバンド割込

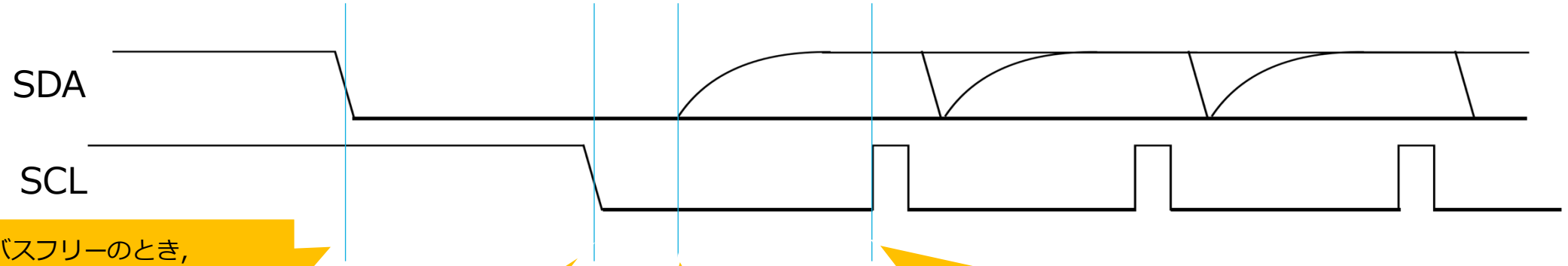
IBI (インバンド割込：In-Band Interrupt)

IBIはターゲットからコントローラへ
バスフリー状態のとき
ターゲットがSDAをLOWにすることで開始



NXP Public

IBI (インバンド割込：In-Band Interrupt)



バスフリーのとき，
ターゲットがSDAを
LOWに下げる

SDAがLOWになったことを
コントローラが認識，
コントローラがSCLをLOWに

ターゲットが自身のアドレスを出力

コントローラは自身が出力するSCLパルスで
アドレスビットをラッチ

全てのアドレス・ビットで調停が終わると
コントローラはACKを出してSDAもプッシュプルへ移行

このとき，もし複数のターゲットがアドレスを出力.
LOWを出しているデバイスと，HIGHを出しているデバイスがあるとSDAはLOWに.
SDAをHIGHにできなかったターゲットは調停に負ける

同時にインバンド割り込みを出した複数のターゲットは，
より小さいアドレスを持つ方がバス調停に勝つ

NXP Public

IBI (インバンド割込：In-Band Interrupt)

インバンド割込：ターゲットからコントローラへの通知を可能に

- GPIOを用いた割込と同様の機能を実現
  - これに加えてIBIデータフレームでデータ転送も可能
  - さらにIBIの優先制御も行われる．小さいアドレス値のターゲットが高い優先順位を持つ
- 割込は，コントローラが「非アクティブ時」に開始できる：ターゲットがSDAをLOWに下げる
  - デバイス内部でのクロックは必要でないため，低消費電力化が可能
- オプションでタイムスタンプ機能．イベント・データを同期
  - 複数の方法を提供：コントローラへのIBIに関連付け

Open Drain			Hand Off	Push-Pull	Drive High or Low, and then High-Z	Optional (push-pull)	Push-Pull
S	Slave_addr_as_IBI/R	**Master_ACK**	SDA High	Slave_byte (‘mandatory byte’)	T	More bytes	**Sr**

NXP Public

HDR：高スループット・モード

High data rate (HDR) モードはオプションとして規定

- クロック高速化ではない 転送ビット数を増加させる(効率を上げる)
- コントローラとターゲットそれぞれにオプションとして
  - サポート外デバイスは無視．サポートするデバイス間で安全に通信
- HDR-DDR フォーマット
  - SDRの約2倍のデータレート (SCL=12.5 MHzで，実効約20Mbps)
  - リード・ライトでCRC and parity をサポート
  - 同じSCLクロックを使うため，ターゲット側追加回路は少
- HDR-BT (Bulk transport)
  - 実データレート，4線使用:97Mbps 2線使用48.5Mbps，1線使用24.3Mbps
  - ファームウェア・データや映像など大量のデータ転送に最適化

NXP Public

01-4

デバイス識別
Provisional-ID



デバイス識別子- MIPI I3C ターゲット・アドレス

デバイス識別子 (Device Identifier)

MIPI I3Cデバイスは，ダイナミック・アドレス割当手順の開始前に，次の2つのうちいずれかの方法で個々が識別できるようになっていなければなりません

1. I^2C同様の静的アドレス．この場合，コントローラは静的アドレスを用いて高速にダイナミックアドレスの割り当てができます．
2. 全てのデバイスには48ビットのProvisional ID (unique ID) が用意されています．
   静的アドレスが用意されていない場合にはこのIDを使ってアドレス割当が行われます

Bits [47:33]	Bit [32]	Bits [31:00]		
		[31:16] 16 bits	[15:12] 4 bits	[11:0] 12 bit
MIPI Manufacturer ID (Note: MSB is discarded) Whether MIPI member or not	Provisional ID Type Selector 1’b1: Random 1’b0: Fixed (normal)	Part ID: The meaning of this 16-bit field is left to the Device vendor to define	Instance ID: Value to identify the individual chip on bus (when multiple of same type): using straps, fuses, non-volatile memory, or another appropriate method	This is left for definition with additional meaning. For example: deeper Device Characteristics, which could optionally extend Device Characteristic Register values
	If Bit [32] = 1’b1: Random Value: Bits [31:0]: 32-bit value randomly generated by the Device.			

NXP Public

01-5

Common Command Codes (CCC)



コマンド空間 (CCC – Common Command Codes)

- 通常のコントローラからターゲットへのメッセージ衝突を回避するため，ビルトイン・コマンド(>40)を定義
  - –バスの振る舞いの制御，モードと状態，低電力状態，問い合わせなど
    - SET, GET, SET-GET コマンド
    - 必須とオプション
  - –別団体で定義できる予備空間
    - 例：JEDEC, VESA, ETSI, Sim Alliance, PCI-Sig, etc
    - 個別用途のためにベンダー空間も用意
  - –個別ターゲットへの通信と，全てのターゲットに向けたブロードキャスト

NXP Public

具体例：P3T1755 https://www.nxp.jp/docs/en/data-sheet/P3T1755.pdf

~~Table 9. Bus Characterization Register (BCR)~~ **Common Command Code**

Common Command Code	CCC type	Command Name	Default setting	Description
0x00	Broadcast	ENEC[1] Enable Events Command	Enabled	Enable Target event driven interrupts
0x06	Broadcast	RSTDAA[1] Reset Dynamic Address Assignment	-	Forget current Dynamic Address and wait for new assignment
0x07	Broadcast	ENTDAA[1] Enter Dynamic Address Assignment	-	Entering Controller initiation of Target Dynamic Address Assignment. Don't participate if the Target already has an Address assigned.
0x80	Direct	ENEC[1] Enable Events Command	enabled	Enable Target event driven interrupts
0x81	Direct	DISEC[1] Disable Events Command	disabled	Disable Target event driven interrupts
0x87	Direct Set	SETDASA[1] Set Dynamic Address from Static Address	-	Controller assigns a Dynamic Address to a target with a known Static Address
0x88	Direct Set	SETNEWDA[1] Set New Dynamic Address	-	Controller assigns a new Dynamic Address to any I3C Target
0x8D	Direct Get	GETPID[1] Get Provisional ID	-	get the Target's Provisional-ID
0x8E	Direct Get	GETBCR Get Bus Characteristics Register	-	Get a Device's Bus Characteristic Register (BCR)
0x8F	Direct Get	GETDCR Get Device Characteristics Register	-	Get a Device's Device Characteristic Register (DCR)
0x90	Direct Get	GETSTATUS Get Device Status	-	Get Device's operating status
0x9A	Direct	RSTACT Target Reset Action	-	Configure and query Target Reset action and timing

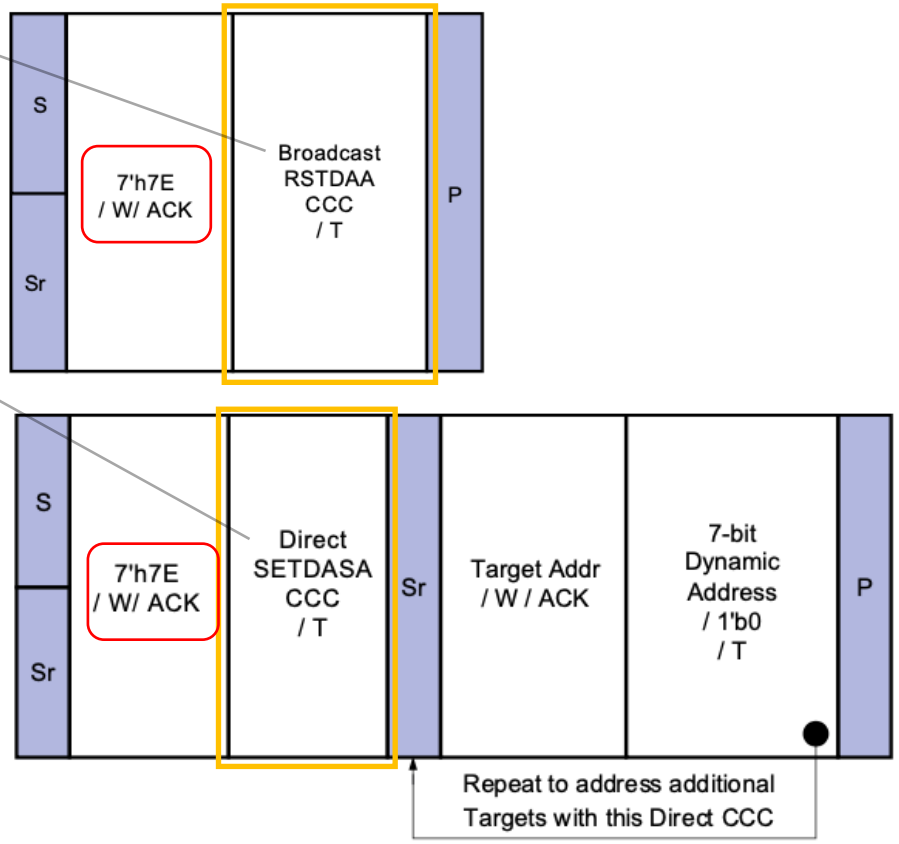
NXP Public

具体例：P3T1755 https://www.nxp.jp/docs/en/data-sheet/P3T1755.pdf

~~Table 9. Bus Characterization Register (BCR)~~ Common Command Code

Common Command Code	CCC type	Command Name	Default setting	Description
0x00	Broadcast	ENEC[1] Enable Events Command	Enabled	Enable Target event driven interrupts
0x06	Broadcast	RSTDAA[1] Reset Dynamic Address Assignment	-	Forget current Dynamic Address and wait for new assignment
0x07	Broadcast	ENTDAA[1] Enter Dynamic Address Assignment	-	Entering Controller initiation of Target Dynamic Address Assignment. Don't participate if the Target already has an Address assigned.
0x80	Direct	ENEC[1] Enable Events Command	enabled	Enable Target event driven interrupts
0x81	Direct	DISEC[1] Disable Events Command	disabled	Disable Target event driven interrupts
0x87	Direct Set	SETDASA[1] Set Dynamic Address from Static Address	-	Controller assigns a Dynamic Address to a target with a known Static Address
0x88	Direct Set	SETNEWDA[1] Set New Dynamic Address	-	Controller assigns a new Dynamic Address to any I3C Target
0x8D	Direct Get	GETPID[1] Get Provisional ID	-	get the Target's Provisional-ID
0x8E	Direct Get	GETBCR Get Bus Characteristics Register	-	Get a Device's Bus Characteristic Register (BCR)
0x8F	Direct Get	GETDCR Get Device Characteristics Register	-	Get a Device's Device Characteristic Register (DCR)
0x90	Direct Get	GETSTATUS Get Device Status	-	Get Device's operating status
0x9A	Direct	RSTACT Target Reset Action	-	Configure and query Target Reset action and timing

アドレス 7'h7Eの指定のあと，CCCを送信



NXP Public

01-6

DAA

(Dynamic Address Assignment)



ターゲットのI3C動作はCCCによってアドレスが割り当てられてから



CCC::BROADCAST_**RSTDAA**
+
CCC::**ENTDAA** or DIRECT_**SETDASA**

POR

I²C

I3C

time

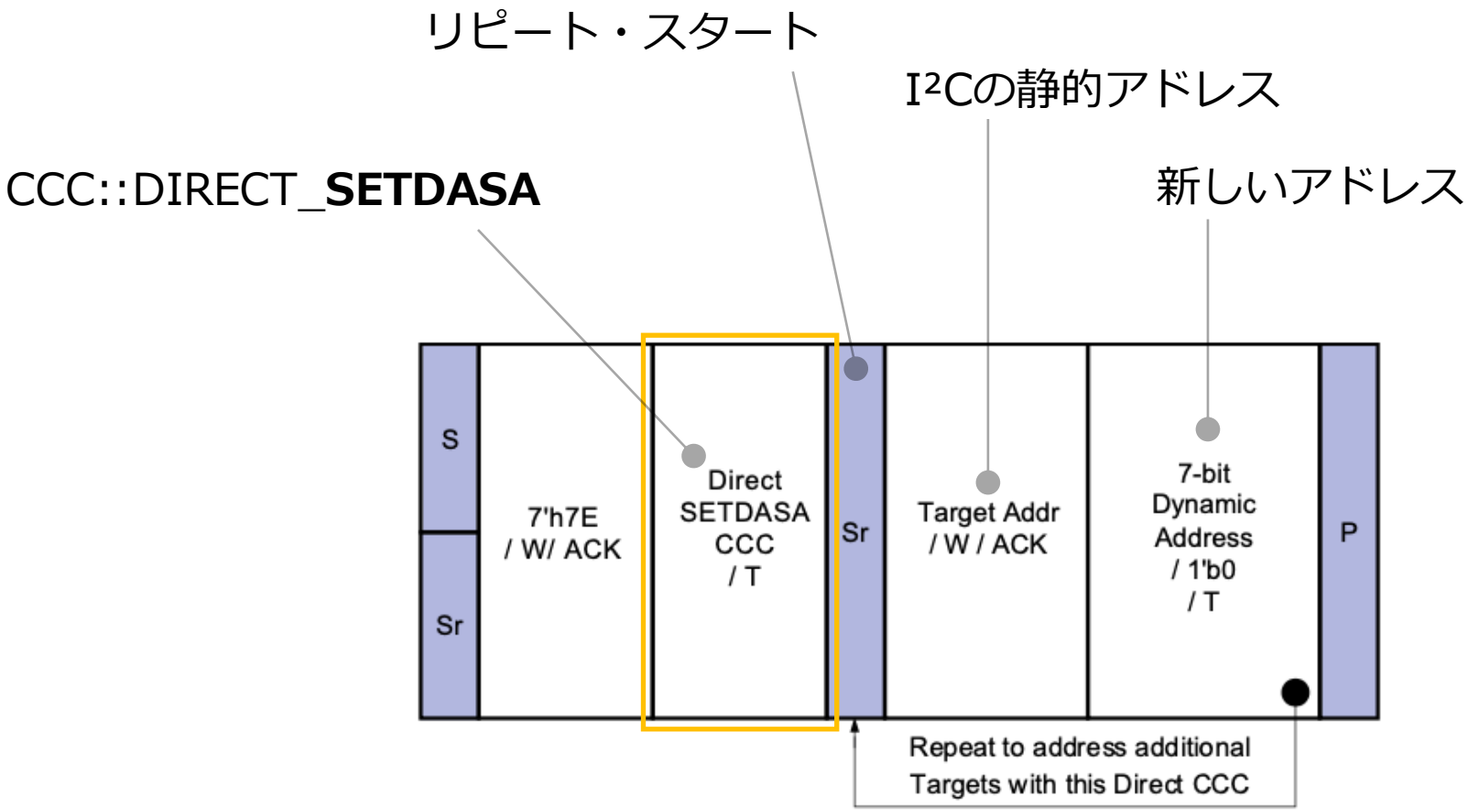
I3Cデバイスは
I²Cで動作を開始

CCCコマンドで新しいアドレスが
割り振られたのち，I3C動作に切り替わる

NXP Public

SETDASA (Set Dynamic Address from Static Address)

I3Cアドレスの割り当て方法，**その1**：I²Cの静的アドレスを使う



NXP Public

ENTDAA (Enter Dynamic Address Assignment)

I3Cアドレスの割り当て方法，**その2**：PIDを使う

次ページからその手順を説明

NXP Public

01-7

参考：
高データレート
実現のための制限事項



信号高速化のための制限

- C_b(バス容量) = 50pF @I3C
  - Specification ver 1.1.1
  - Ver 1.1まではターゲットは11個に制限

- C_b = 400pF @I²C Fast-mode (400kHz)
- C_b = 550pF @I²C Fast-mode Plus (1MHz)

- **NOT INCLUDED IN I3C BASIC:** Device to Device(s) Tunneling
- Multi-Lane Data Transfer, for some HDR Modes included in I3C Basic

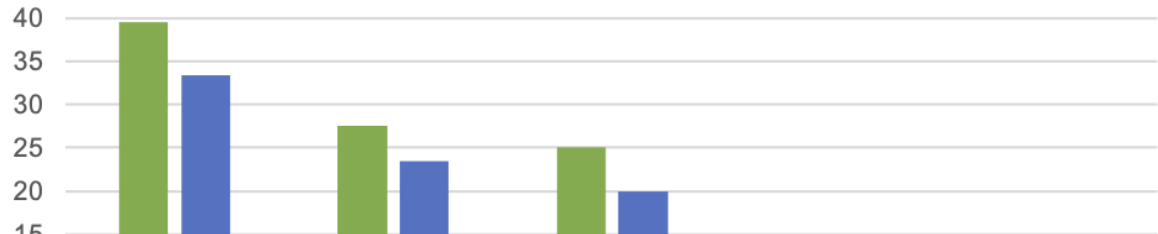
Note:

Multi-Lane Data Transfers for SDR Mode and HDR-DDR Mode is not included in I3C Basic

The I3C interface provides major efficiencies in Bus power while providing greater significant spe improvements over I²C. ***Figure 2*** through ***Figure 5*** show different Bus Mode options that provide tradeo for performance/power vs. target Device complexity. The following conditions were assumed for calculations:

- Devices are in close vicinity:
  - Targets clustered close to Controller
  - No delay on the wires
- Total leakage on each wire: 4 µA
- Driver's internal resistance: 90 Ω
- Total Bus capacitance: 50 pF
- Bus main clock: 12.5 MHz
- V_{DD} = 1.8V
- Pull-Up resistor on Open-Drain: 2833 Ω
- Equal probability for 1 and 0 on data transmission

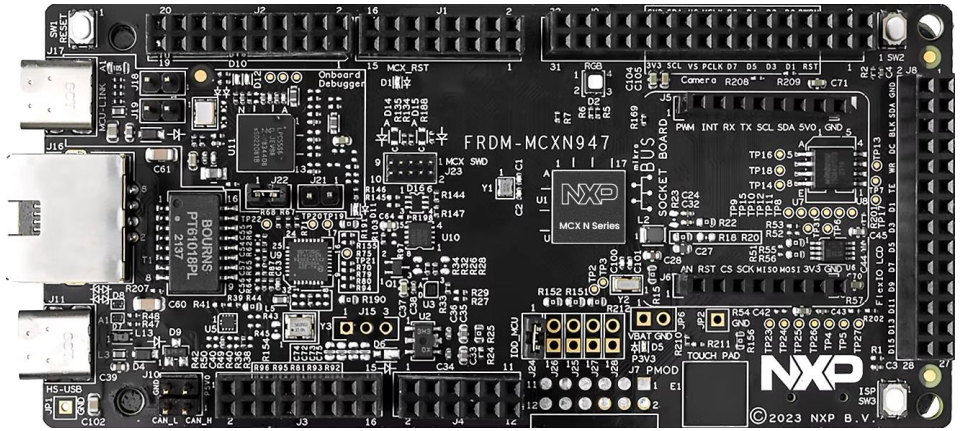
Figure 2 shows the Raw and Effective single-Lane bit rates of the I3C Modes compared to I²C FM+ (1 MH



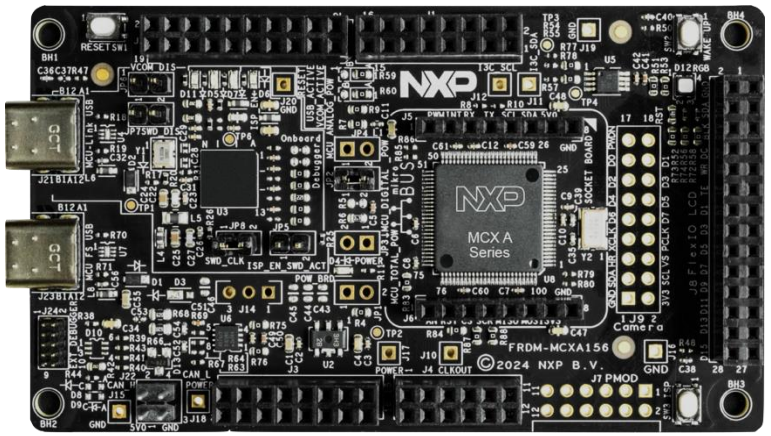
01-8

I3Cを
試してみよう！

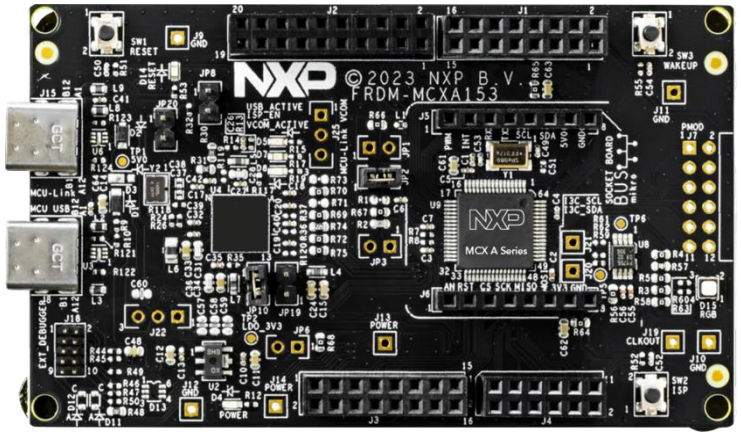
I3Cを簡単に試せる評価ボード，あります！



FRDM-MCXN947



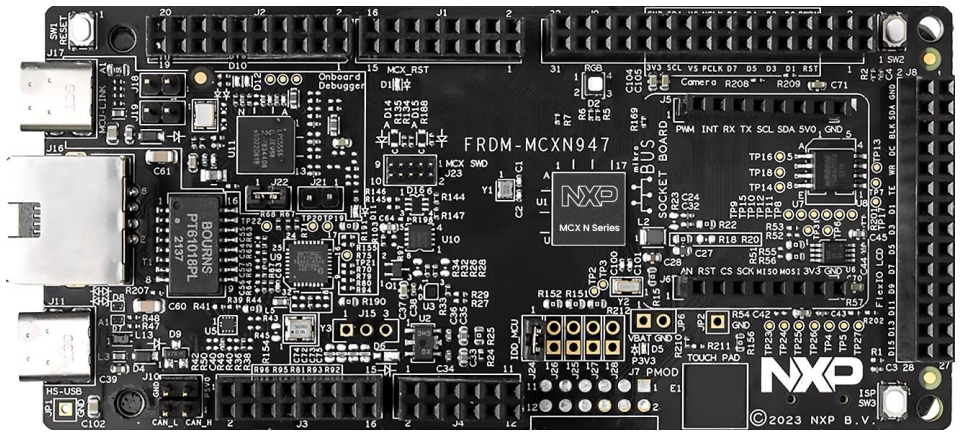
FRDM-MCXA156



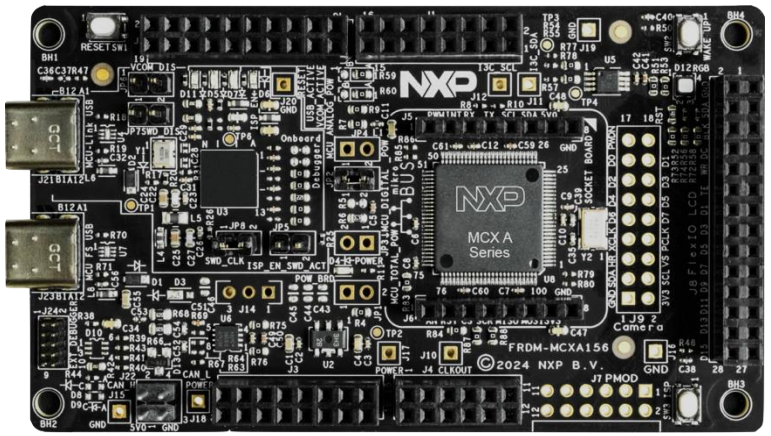
FRDM-MCXA153

NXP Public

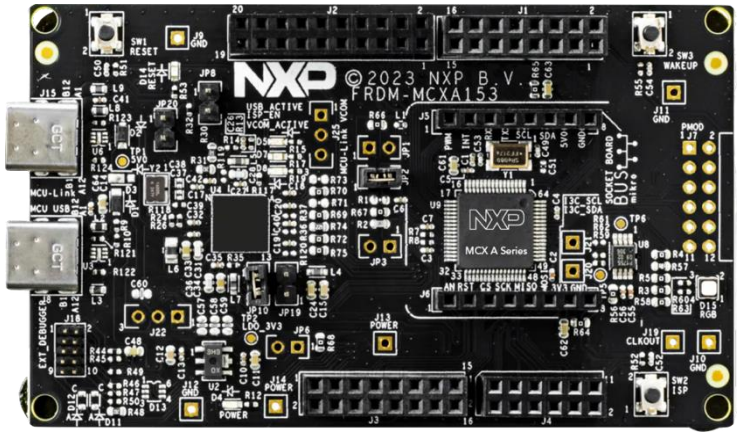
I3Cを簡単に試せる評価ボード，あります！



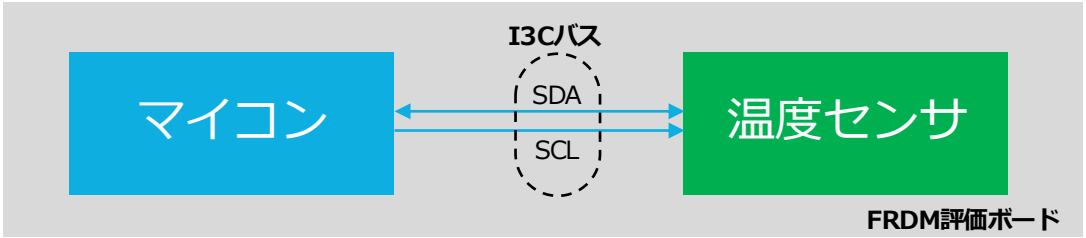
FRDM-MCXN947



FRDM-MCXA156

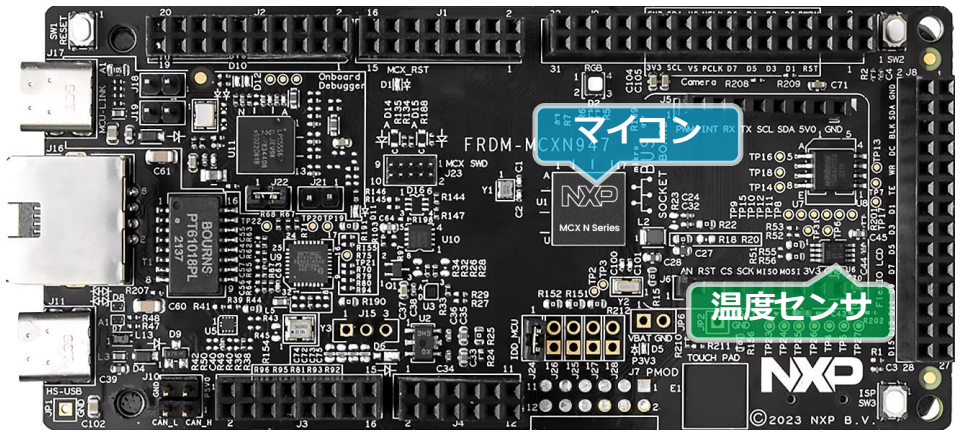


FRDM-MCXA153

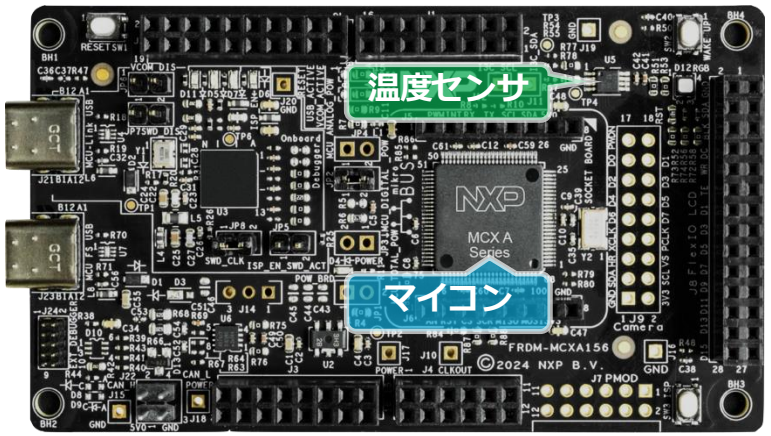


これらの評価ボードにはI3C対応のマイコンと温度センサを搭載
サンプルコードを使ってすぐに動かしてみることができます！

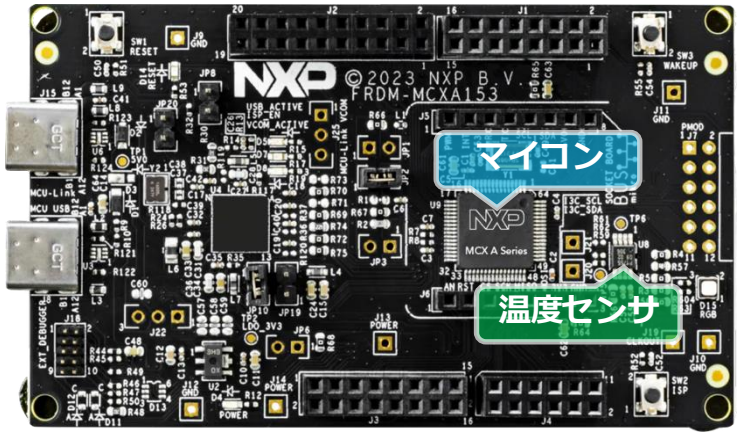
I3Cを簡単に試せる評価ボード，あります！



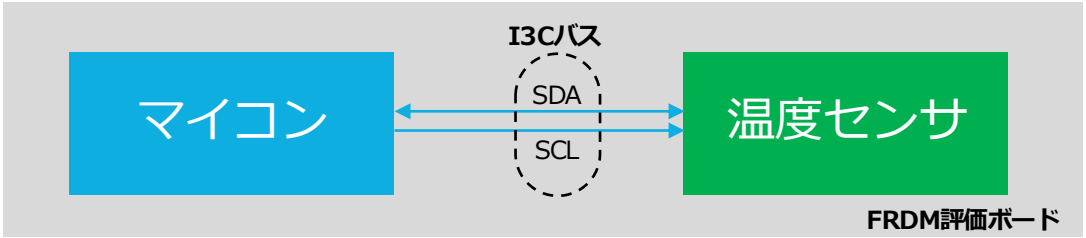
FRDM-MCXN947



FRDM-MCXA156

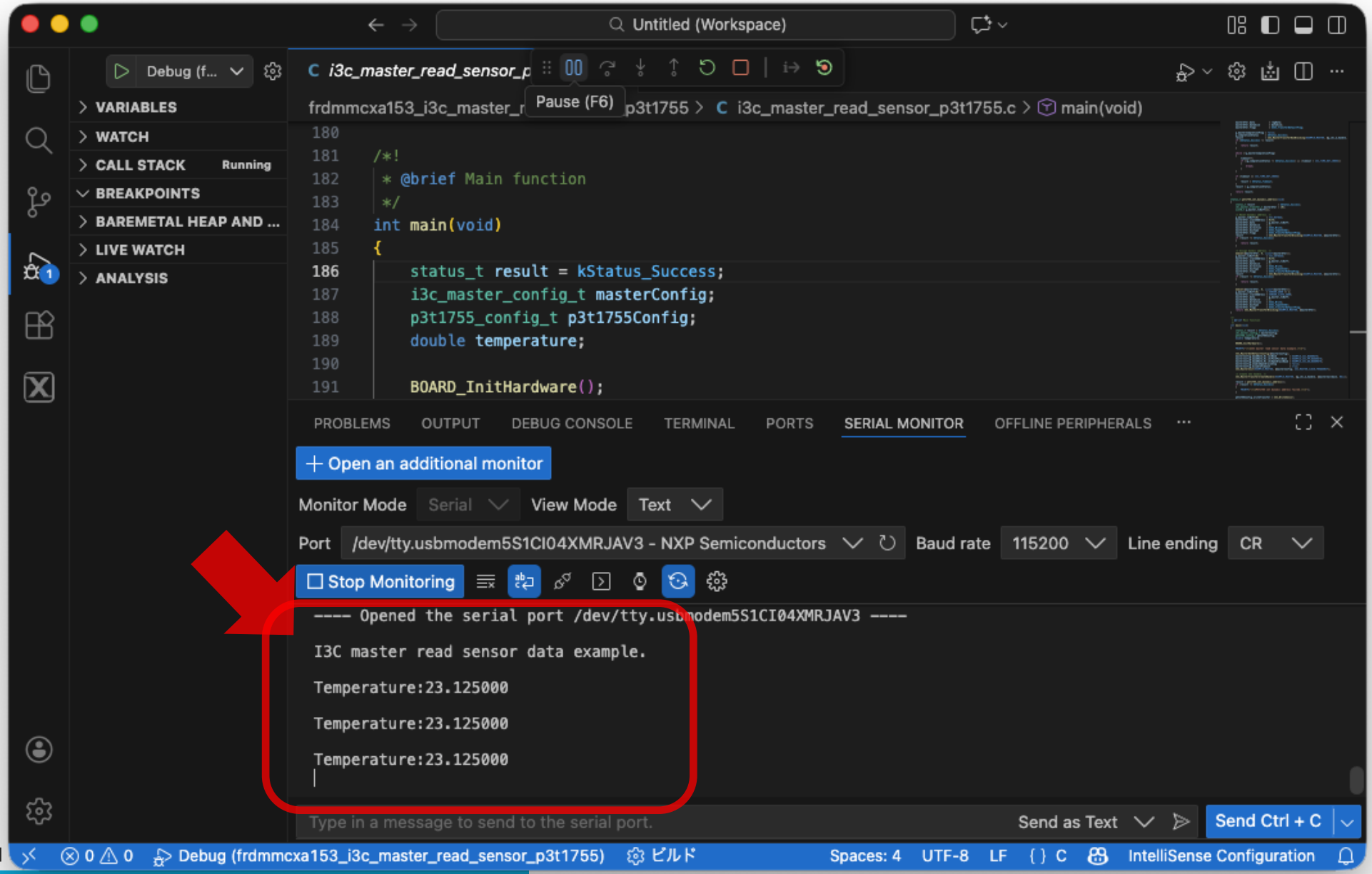


FRDM-MCXA153



これらの評価ボードにはI3C対応のマイコンと温度センサを搭載
サンプルコードを使ってすぐに動かしてみることができます！

I3Cを簡単に試せる評価ボード，あります！



MCUXpresso for VSCで
サンプルコード動作

I3Cを簡単に試せる評価ボード，あります！

マイコン・ツールのインストールは簡単.
ガイドに沿って操作するだけ□





マイコン・ソフト開発ツールの
MCUXpressoは，
統合開発環境(IDE)版と
Visual Studio Code対応版があります．

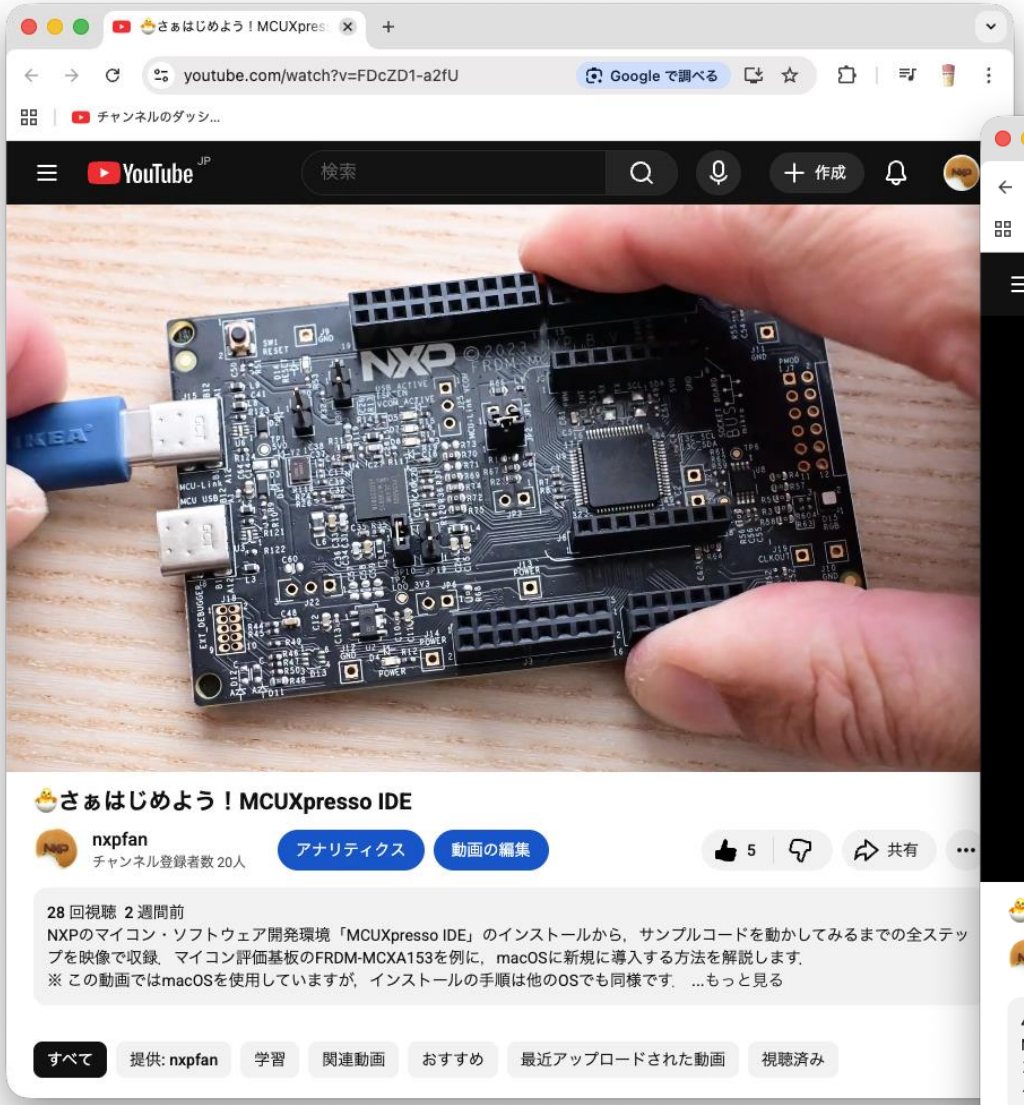
どちらでも，好みの環境が使えます

[MCUXpresso IDEとSDKのインストール (日本語ブログ)]
[MCUXpresso for VSCとSDKのインストール (日本語ブログ)]

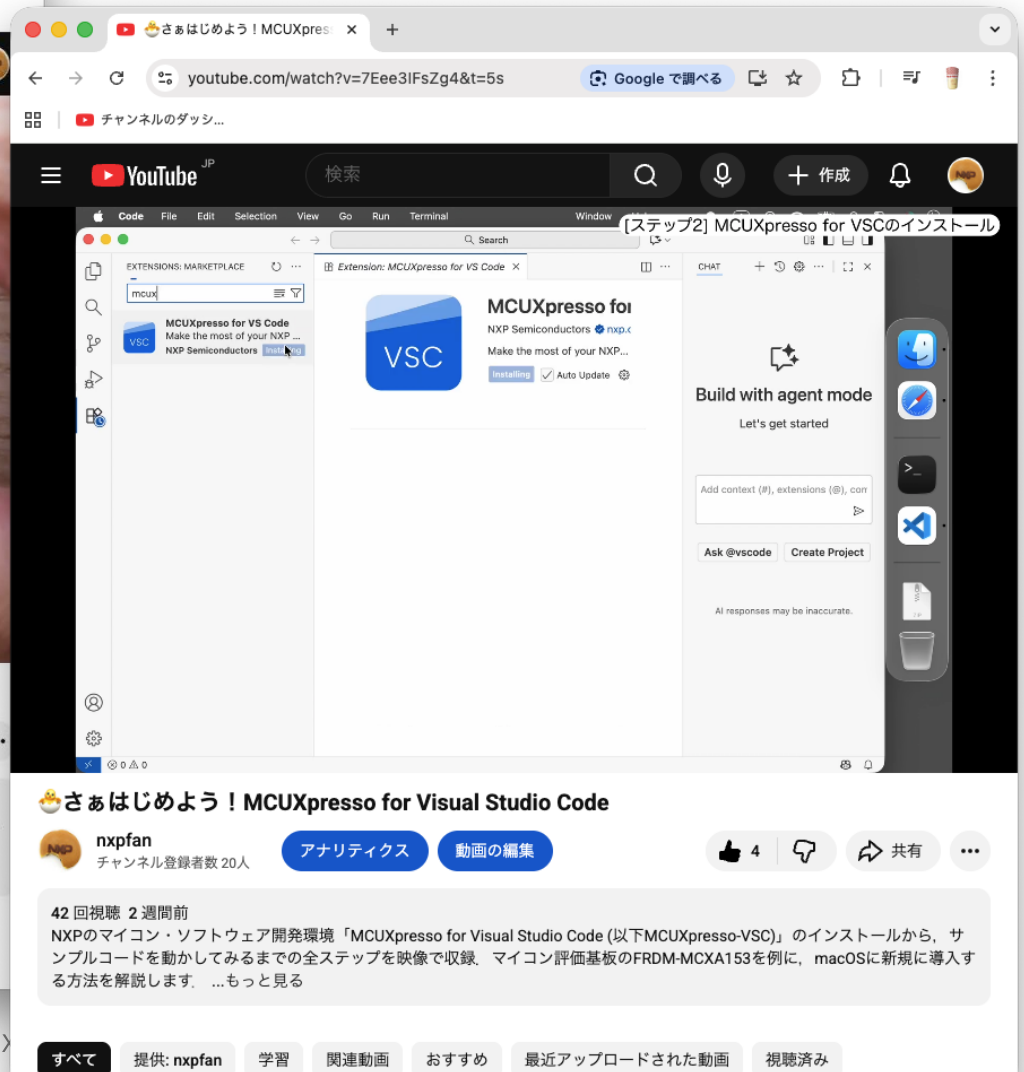
I3Cを簡単に試せる評価ボード，あります！

ガイド動画もあります

[さぁはじめよう！MCUXpresso IDE]

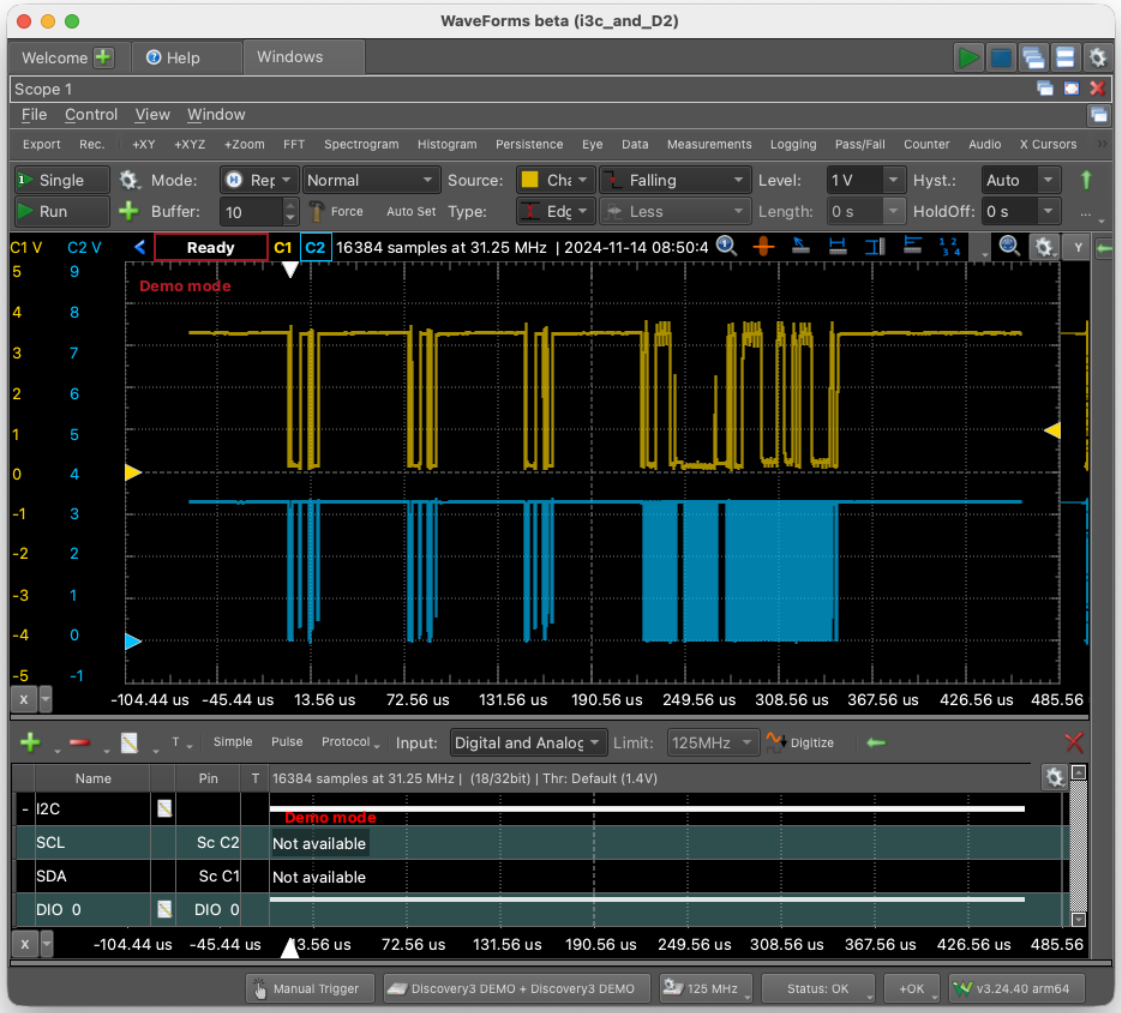


[さぁはじめよう！MCUXpresso for Visual Studio Code]



I3Cを簡単に試せる評価ボード，あります！





オシロスコープ画面

サンプルコードでI3C温度センサを3個，I²C温度センサを1個
同一バス上で動作させた例

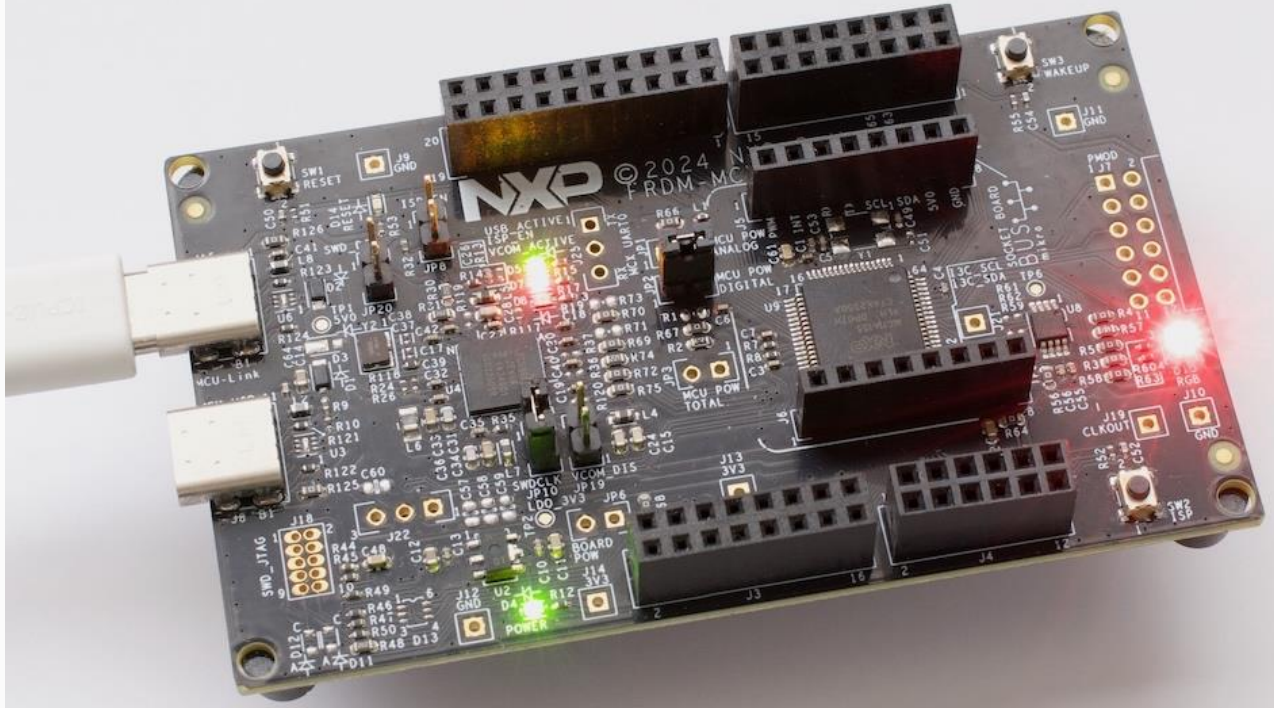
[I3C動作サンプルコード：「i3c-temperature-sensor」の動かし方]

I3Cを試してみよう！

02

Arduino IDE/SDKを使って FRDM-MCXA153を動かしてみる

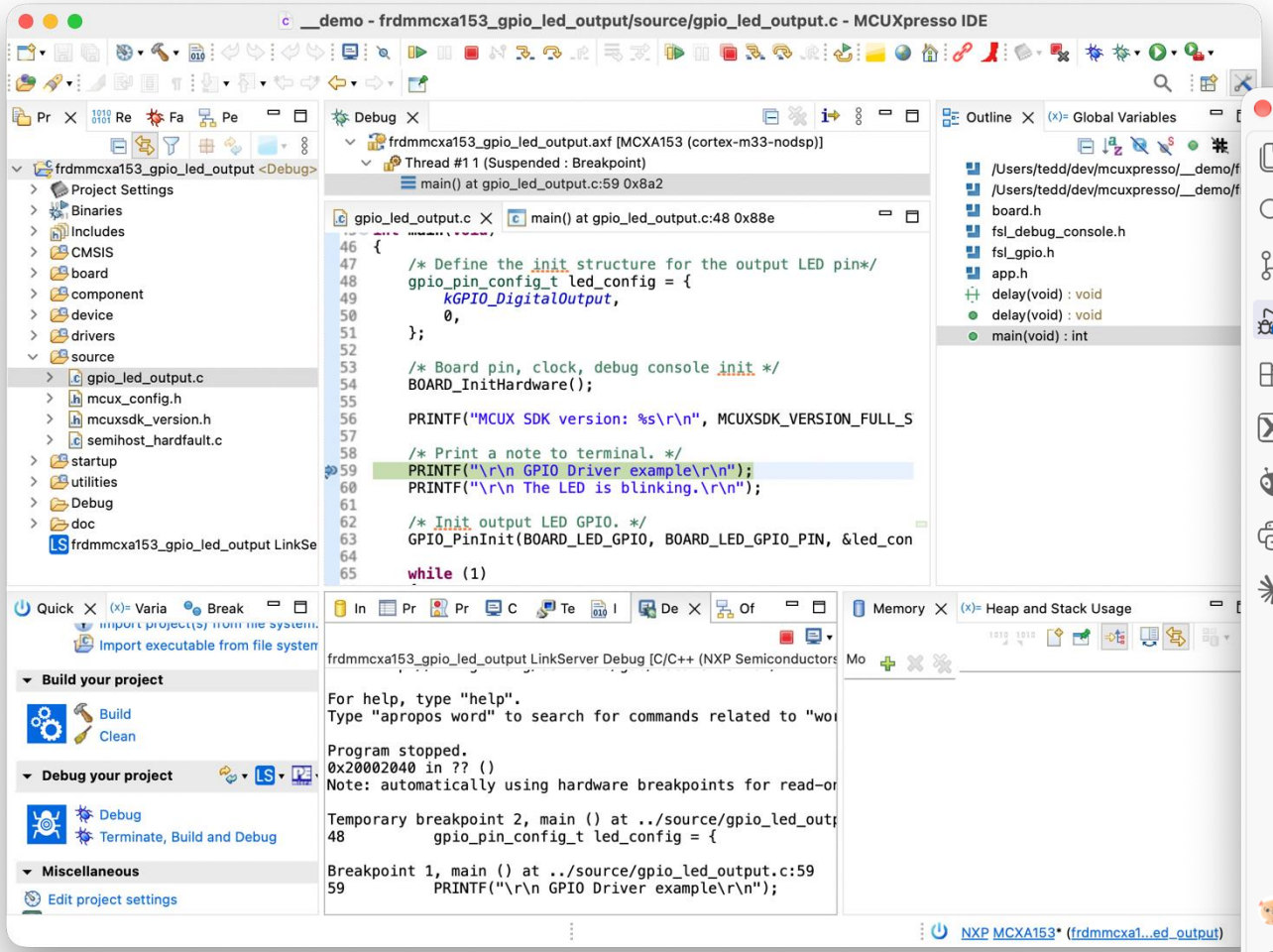
mcx-arduino-coreと
サンプルコードの動作



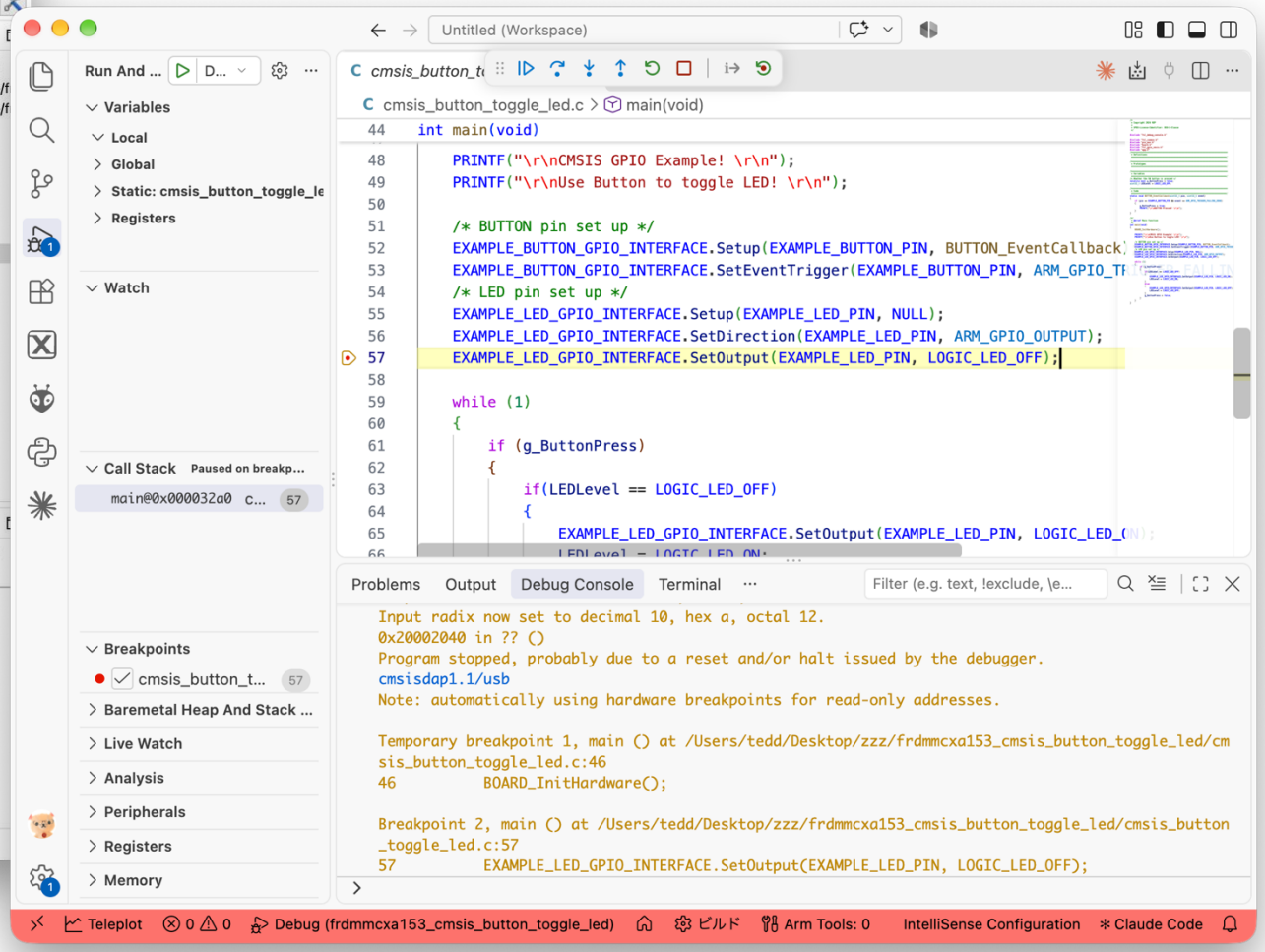
02-1

MCXの開発環境

NXPの公式(フリー)開発ツール



MCUXpresso IDE

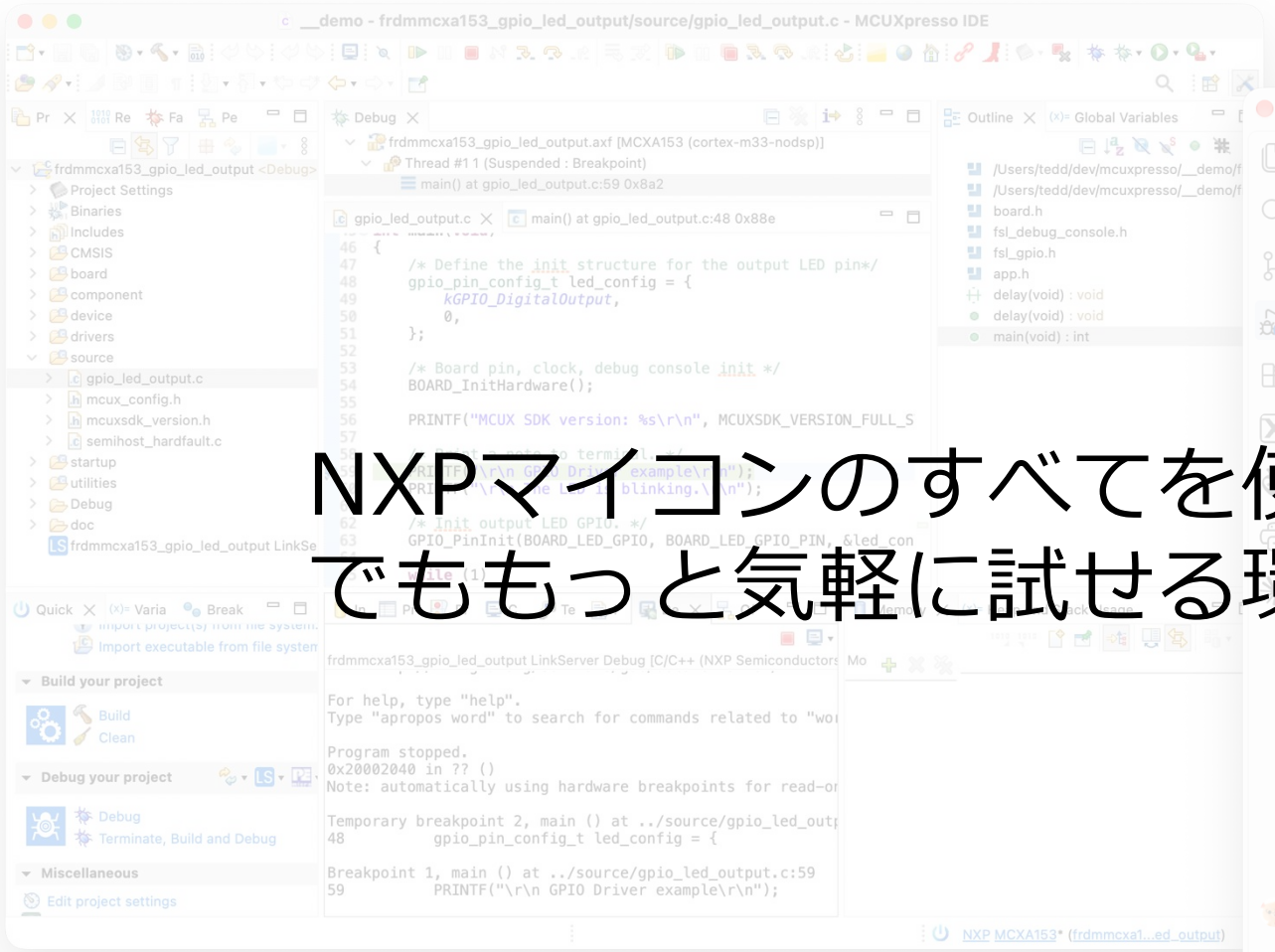


MCUXpresso for VisualStudio Code

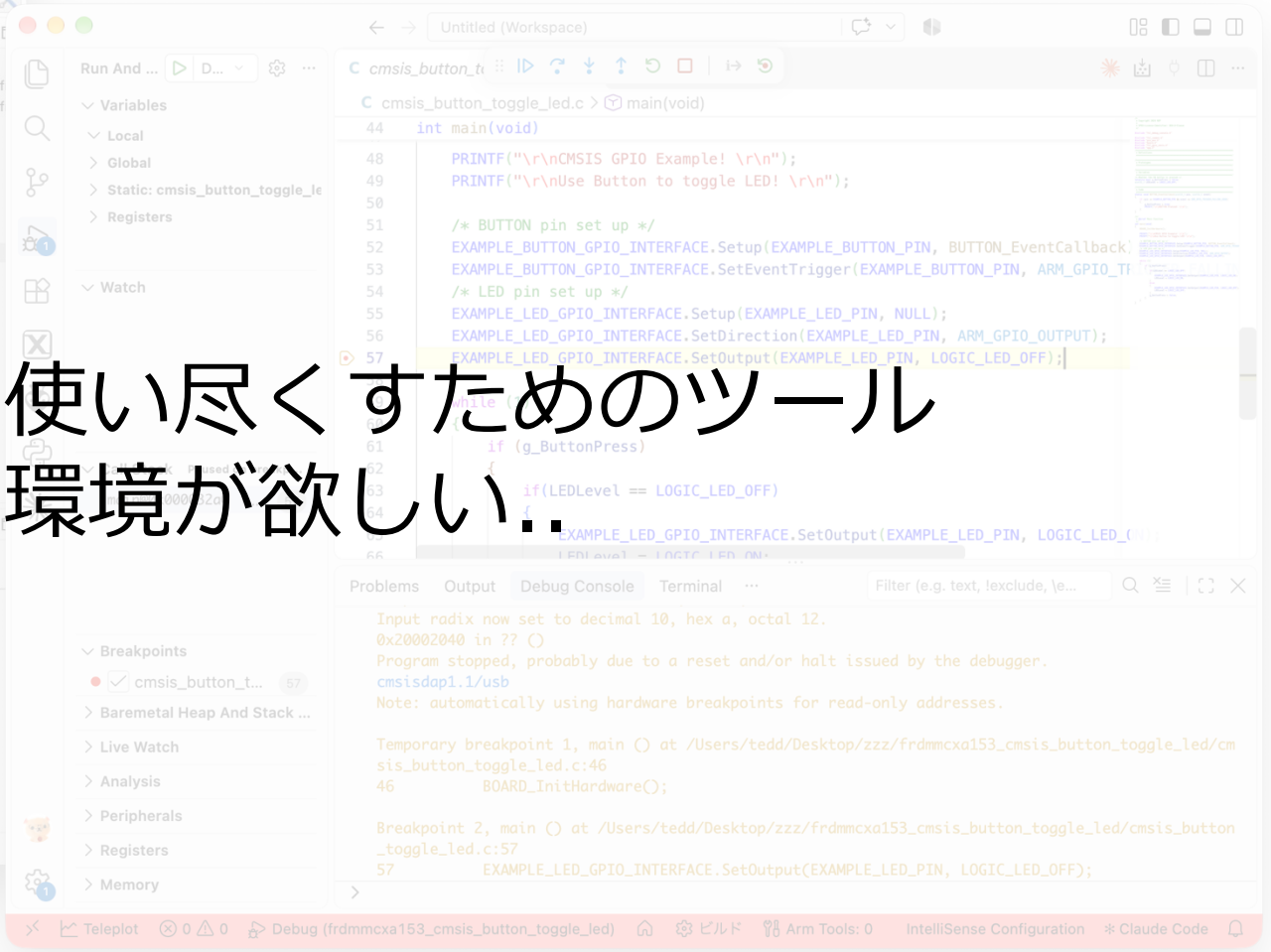
NXP Public

NXPの公式(フリー)開発ツール

NXPマイコンのすべてを使い尽くすためのツール
でももっと気軽に試せる環境が欲しい..



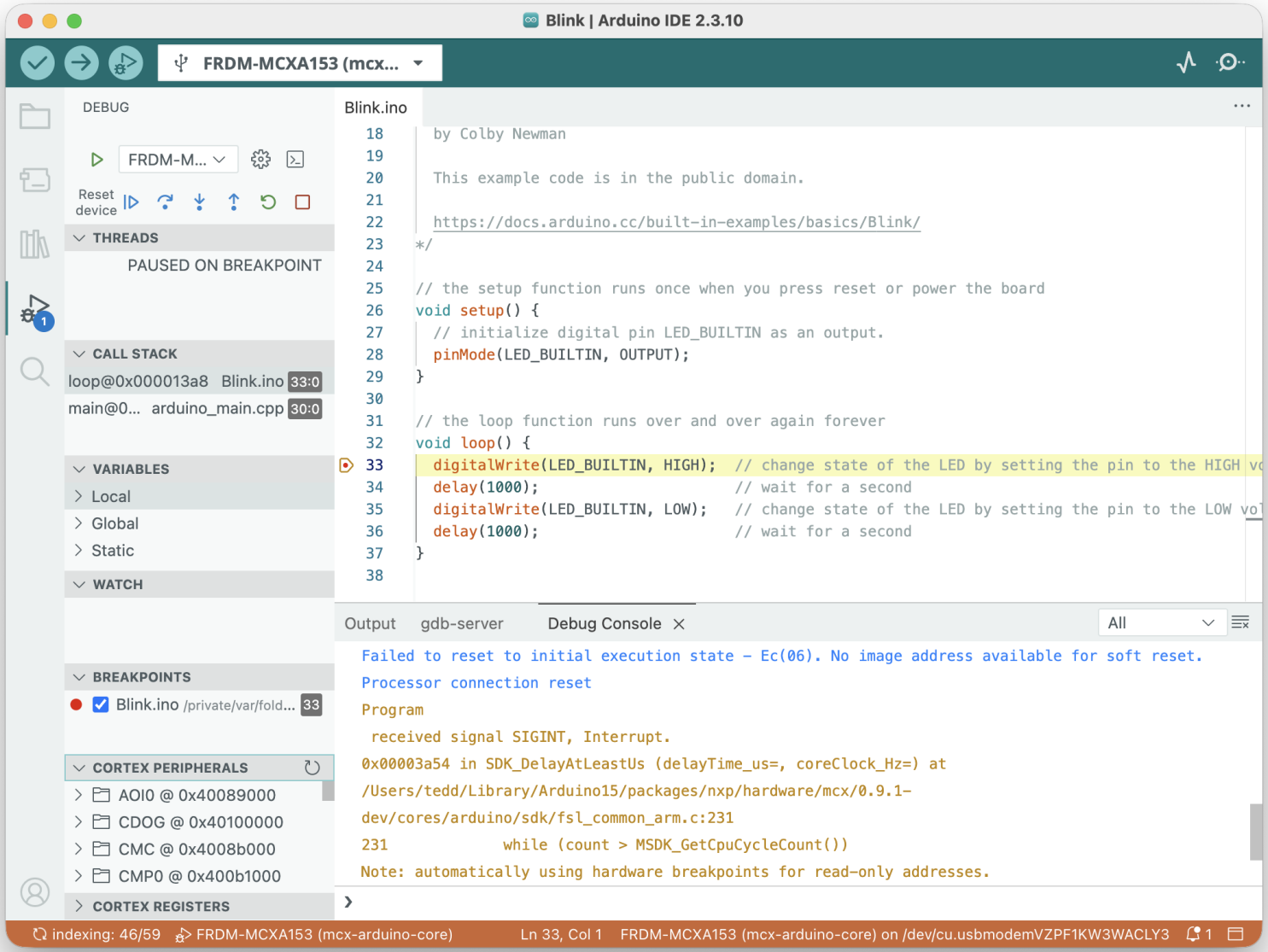
MCUXpresso IDE



MCUXpresso for VisualStudio Code

NXP Public

Arduino IDEで使えるようにしてみた



https://github.com/teddokano/mcx-arduino-core/blob/main/TUTORIAL.ja.md

NXP Public

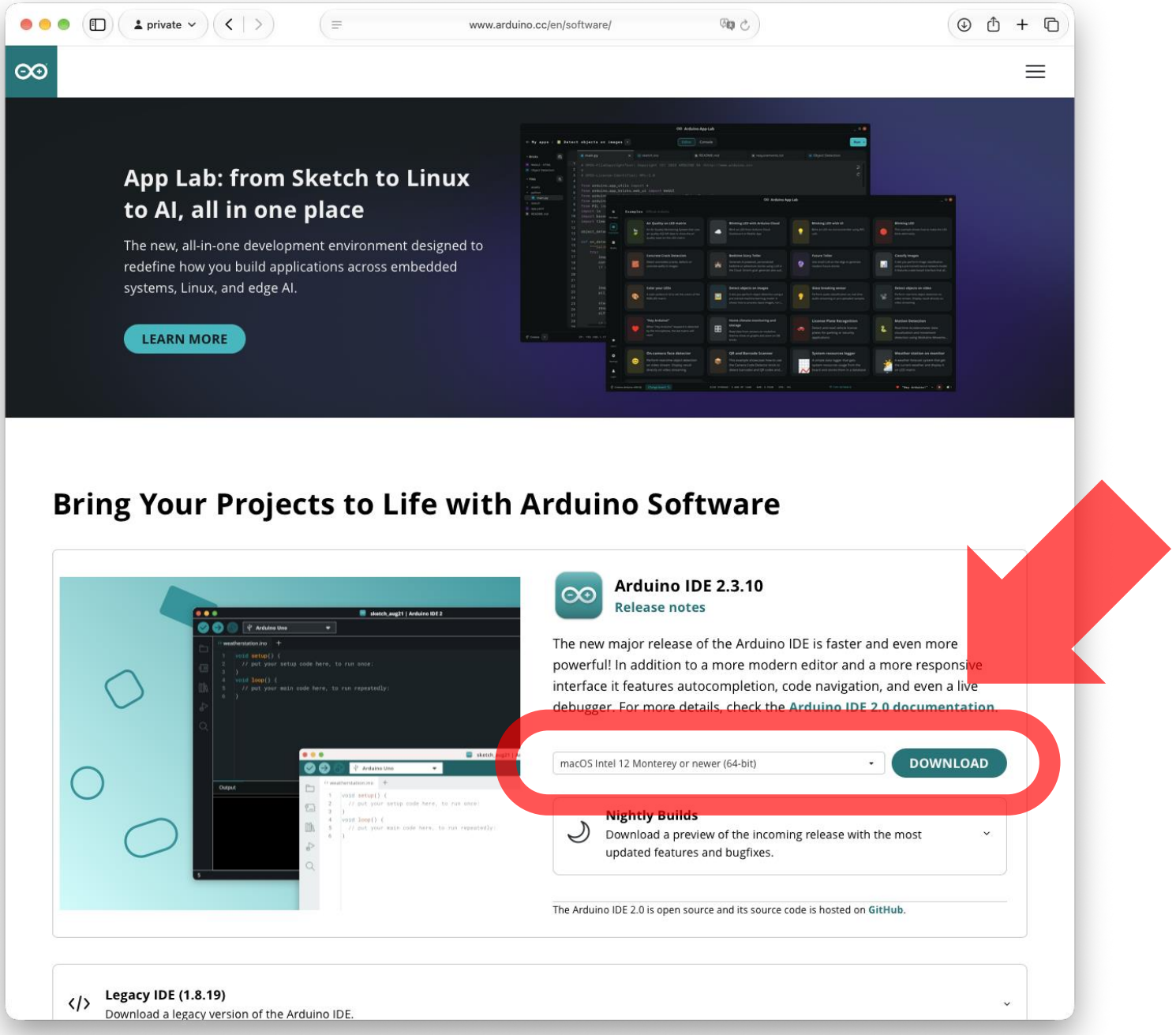
02-2

環境のインストール



Arduino IDEのインストール

https://www.arduino.cc/en/software/



NXP Public

LinkServerのインストール

LinkServerは，NXPマイコンへのフラッシュ書き込みや，ボード上のデバッグチップとの通信を行うソフトウェア.
Arduino IDEとボードの間の通信を行なってくれる

OS	LinkServer 26.6.137
Windows	LinkServer_26.6.137.exe
macOS（Apple silicon）	LinkServer_26.6.137.aarch64.pkg
macOS（Intel）	LinkServer_26.6.137.x86-64.pkg
Linux（x86_64）	LinkServer_26.6.137.x86_64.deb.bin
Linux（arm64）	LinkServer_26.6.137.aarch64.deb.bin

注意1: NXPのダウンロードページには前のバージョンへのリンクがないため，上記にリストしておいた
注意2:ダウンロードページから入手する場合と同じく、ダウンロードするとNXPのソフトウェア使用許諾に同意したことになる
注意3: 現行最新版の v26.9.130 はMCX-A系のメモリサイズの扱いに問題があるので，それ以前のバージョンを使う
注意4: v26.9.130(最新版)と過去のバージョンを両方インストールしておいても問題ない

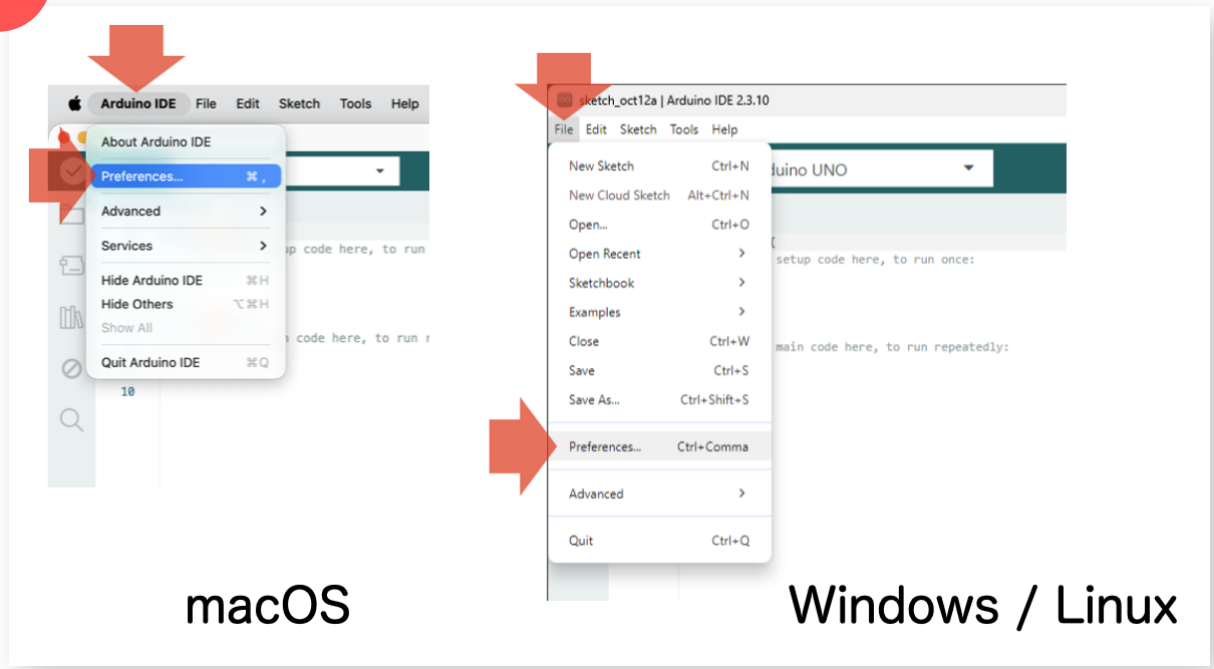
NXP Public

ボードパッケージのインストール (1/2)

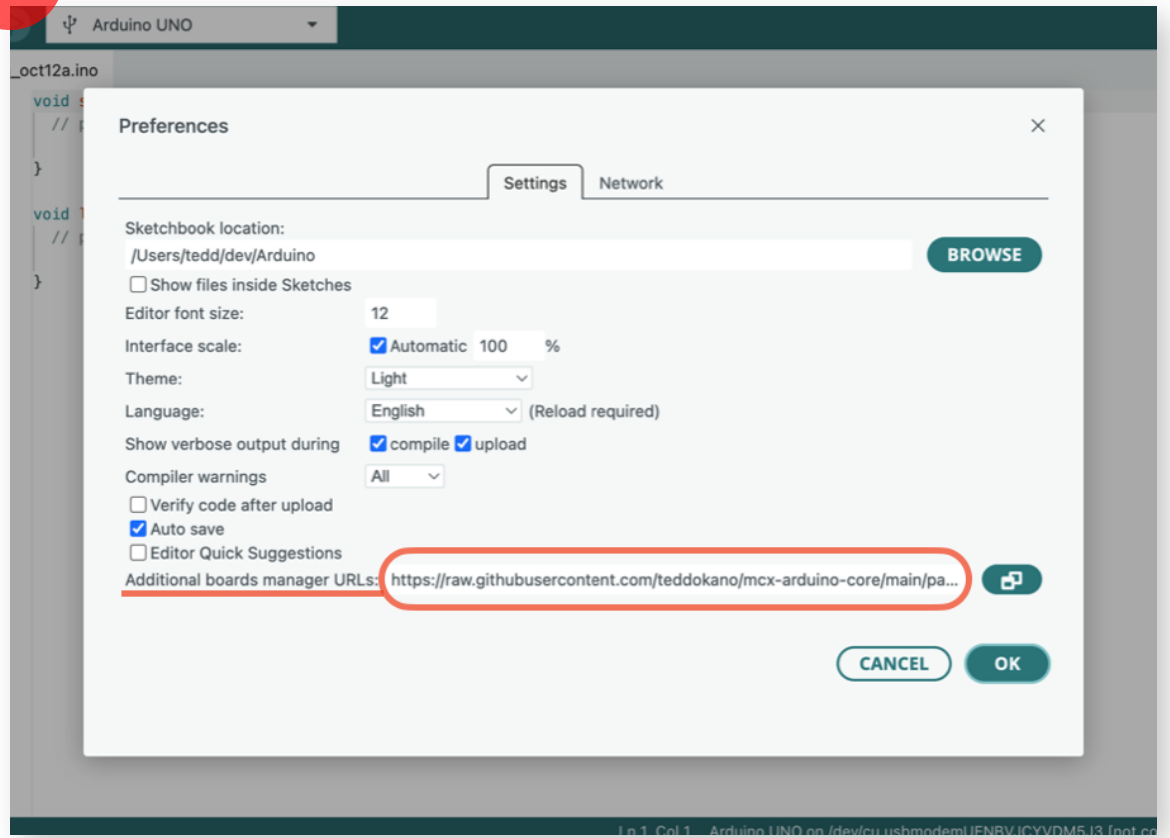
1. Arduino IDEを開き、**File** → **Preferences** (macOSの場合は **Arduino IDE** → **Preferences**)
2. Additional boards manager URLsに以下を追加:

```
https://raw.githubusercontent.com/teddokano/mcx-arduino-core/main/package_nxp_mcx_index.json
```

1

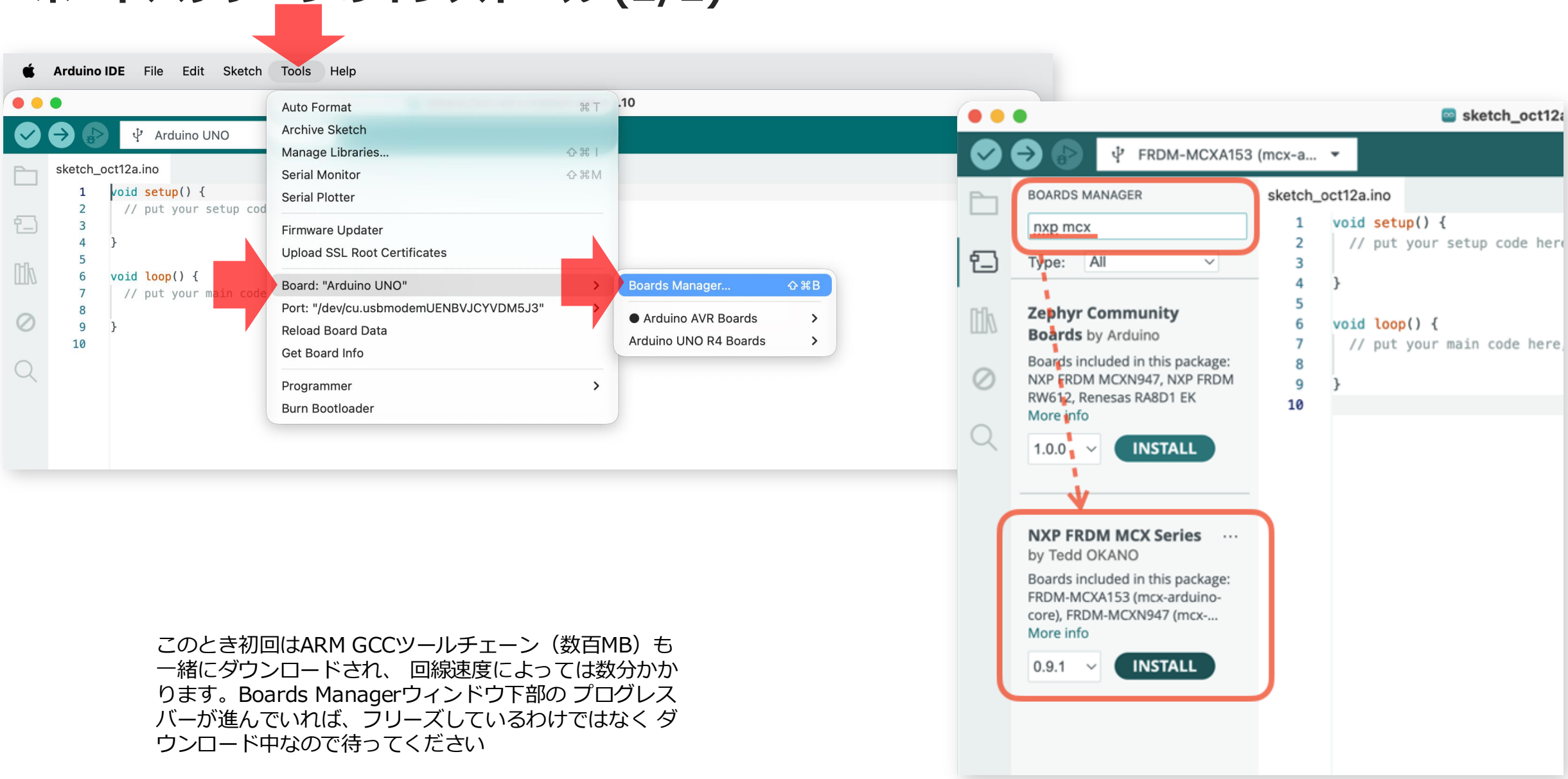


2



NXP Public

ボードパッケージのインストール (2/2)



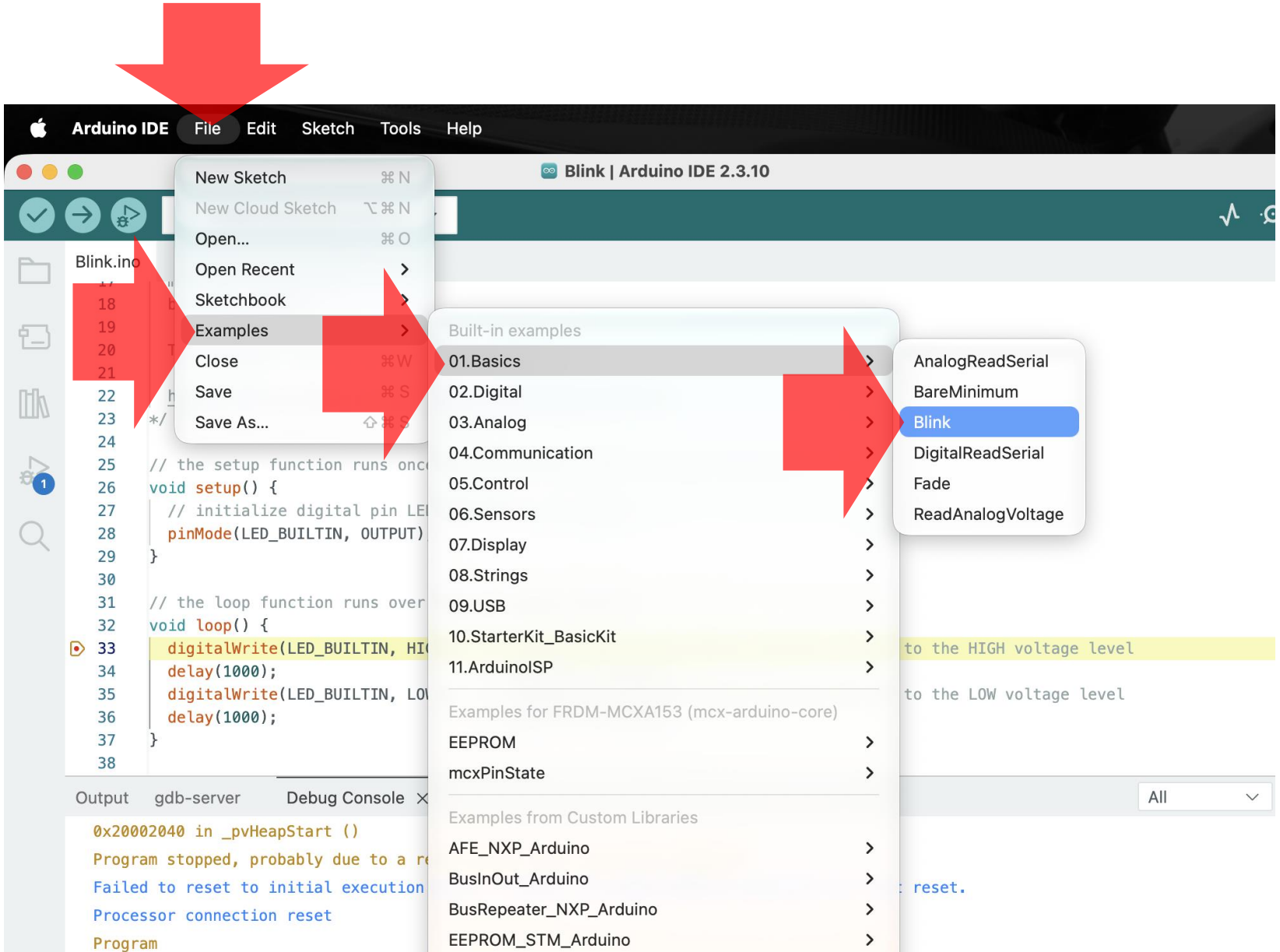
このとき初回はARM GCCツールチェーン（数百MB）も一緒にダウンロードされ、 回線速度によっては数分かかります。Boards Managerウィンドウ下部の プログレスバーが進んでいれば、フリーズしているわけではなく ダウンロード中なので待ってください

NXP Public

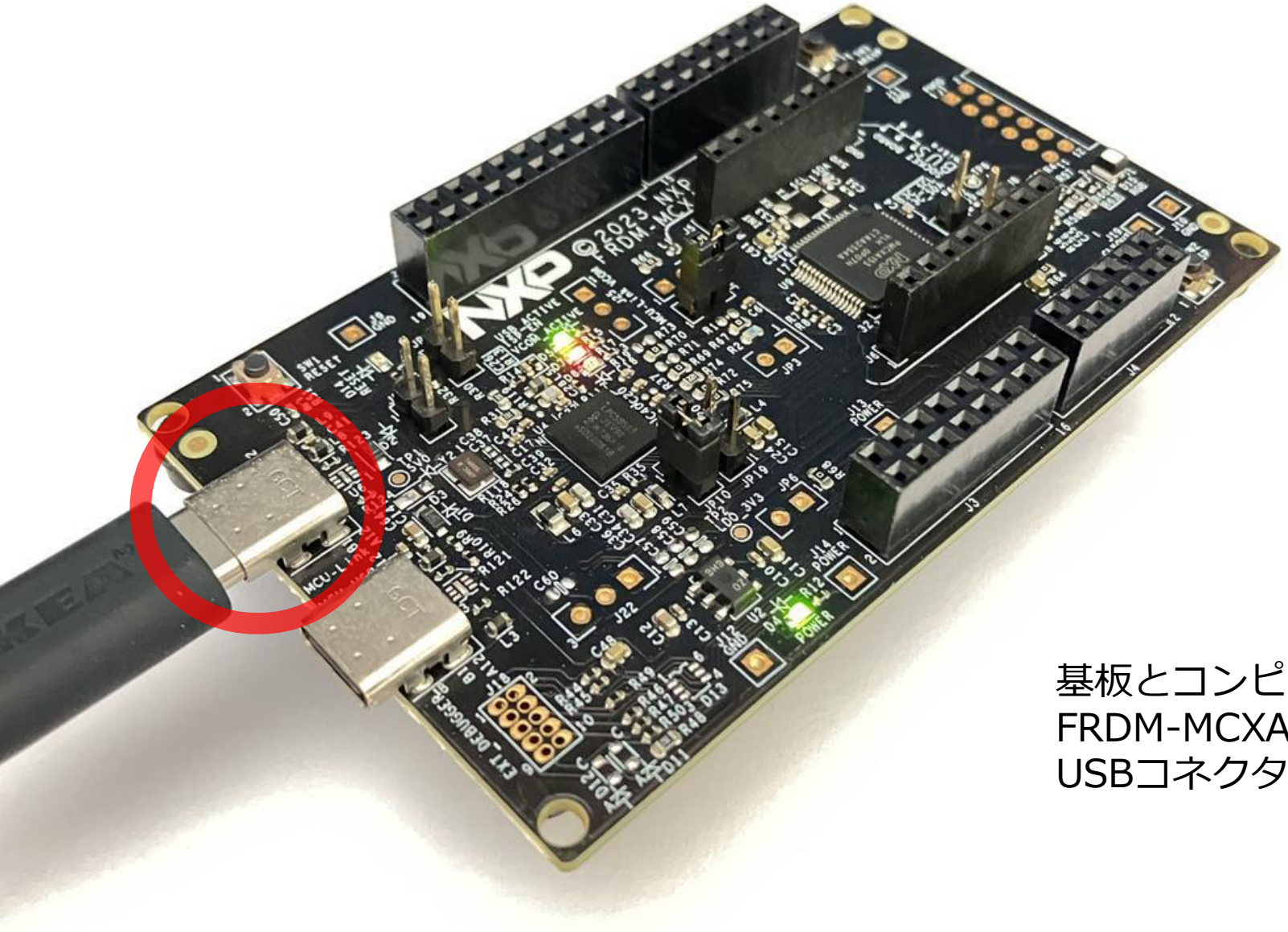
02-2

動作確認

MCXでLチカを試す: Lチカサンプルコードの選択



MCXでLチカを試す: ボードの接続



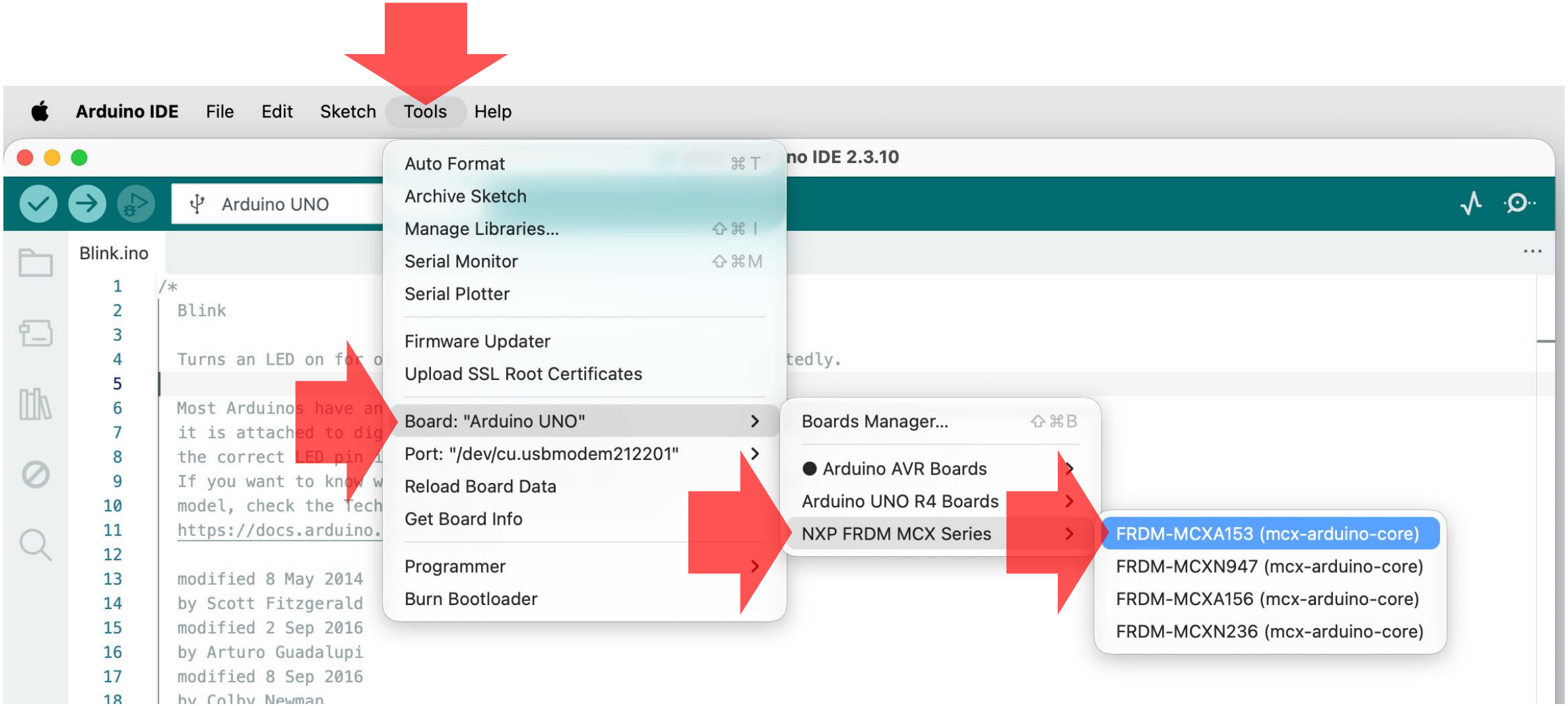
基板とコンピュータをUSBケーブルで接続します
FRDM-MCXA153では「MCU-Link」の表示のある
USBコネクタを使います

NXP Public

MCXでLチカを試す: ポートとボードの選択

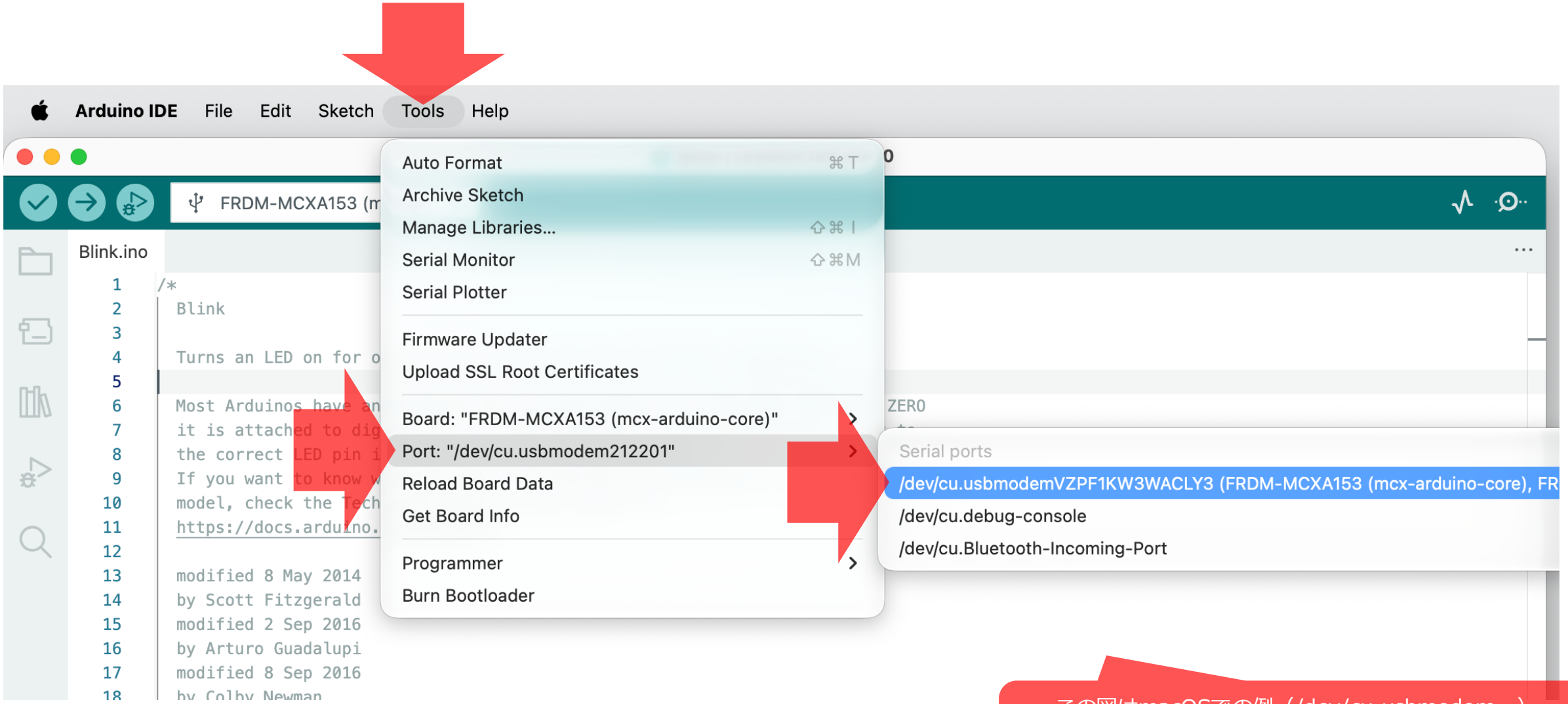


MCXでLチカを試す: ボードの選択



NXP Public

MCXでLチカを試す: ポートの選択



この図はmacOSでの例（/dev/cu.usbmodem...）
Windowsでは「COM3」のようなCOMポート，
Linuxでは「/dev/ttyACM0」のような名前で表示される

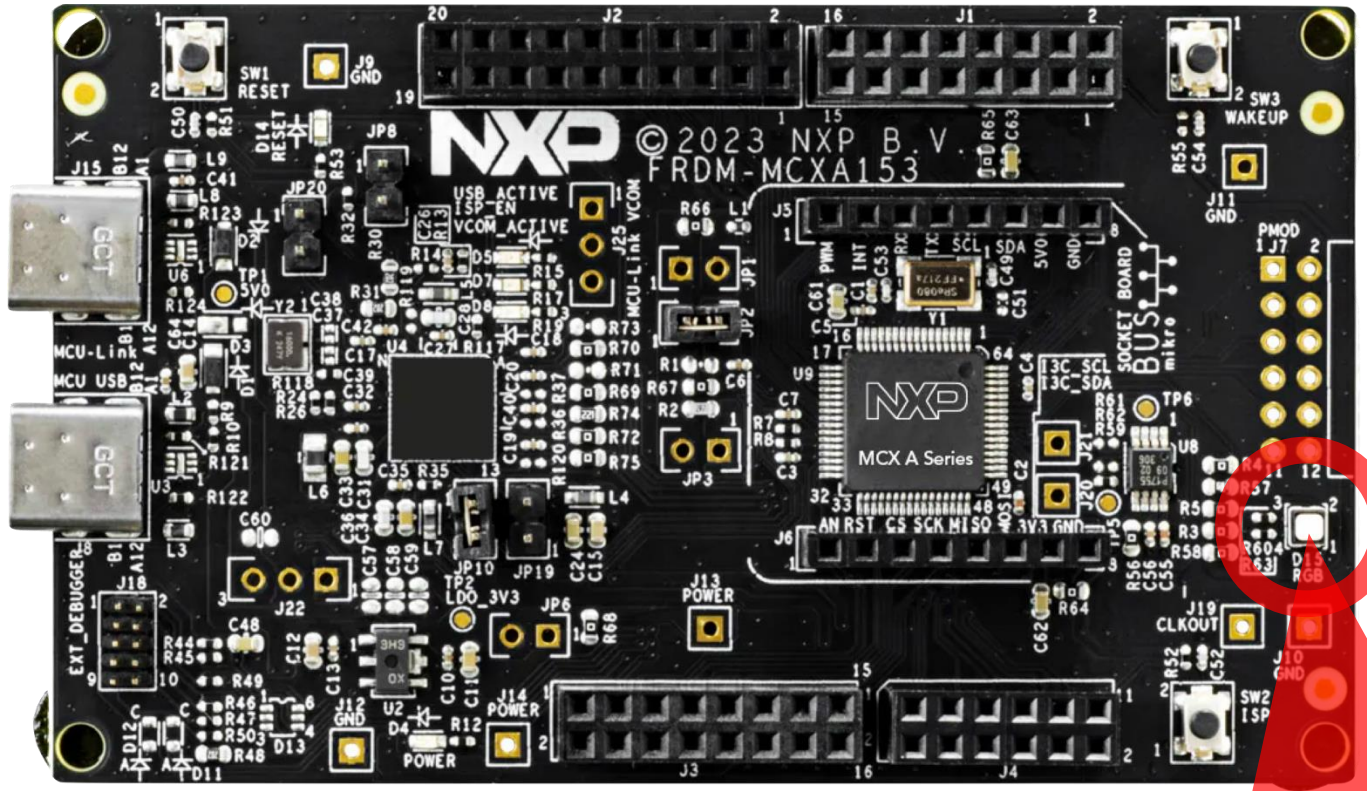
NXP Public

MCXでLチカを試す: 動作確認(1/2)



アップロード・ボタンを押して
ビルド，書き込み

MCXでLチカを試す: 動作確認(2/2)



緑のLEDが点滅すればOK

NXP Public

ボードパッケージについて：「mcx-arduino-core」（1/2）



mcx-arduino-core

FRDM-MCXA153

NXP FRDM MCXシリーズボード向けのArduinoボードサポートパッケージです。

English version → README.md

初めての方はチュートリアル日本語版（英語版もあります）から始めてください。 Arduino API対応状況の一覧は API_COMPATIBILITY.md、各ボードのピン配置はPIN_MAPPING_A153.md / PIN_MAPPING_N947.md / PIN_MAPPING_A156.md / PIN_MAPPING_N236.md、バージョン間の変更点はCHANGELOG.md、生成済みDoxygenクラスリファレンス（r01libドライバコアとArduino互換APIレイヤー）は docs/api/を参照してください（これらは英語のみです）。

標準Arduino APIの先にある上級者向けガイドもあります（いずれも英語のみ）：MCUXpresso SDKを直接呼び出す（GPIO速度チューニング）、r01libによるネイティブI3C（Wire 形式のAPIでは扱えない機能向け）、mcxPinStateによるピン所有状況のデバッグ（そのためのライブラリ、本パッケージに同梱済み）。同梱ライブラリはもう1つあります——mcxRCServoはPWMのピン（PWM0 〜 PWM5 と、FRDM-MCXA156・FRDM-MCXN236では D3 ・ D5 ・ D6 ・ D9 ）からRCサーボを駆動します。

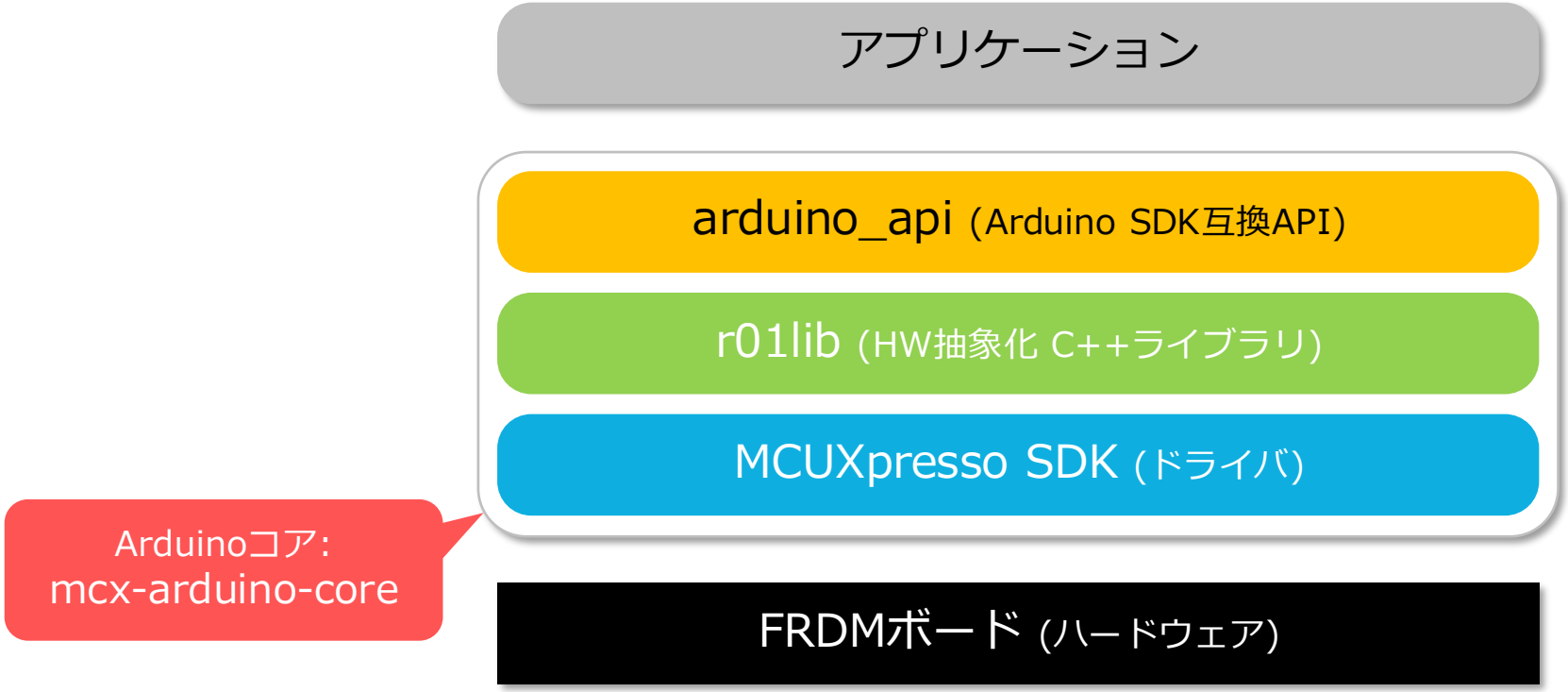
このコアにFRDM-MCXボードを追加する作業は、コアを使う作業とは別物なので専用の手順書があります: 新しいボードの移植（英語のみ）。

セットアップガイド動画もあります。

対応ボード

ボード	MCU	コア
FRDM-MCXA153	MCXA153 (Cortex-M33)	✅
FRDM-MCXA156	MCXA156 (Cortex-M33)	✅ (0.8.0から。下の注を参照)
FRDM-MCXN947	MCXN947 (Cortex-M33)	✅
FRDM-MCXN236	MCXN236 (Cortex-M33)	✅ (0.9.0から。下の注を参照)

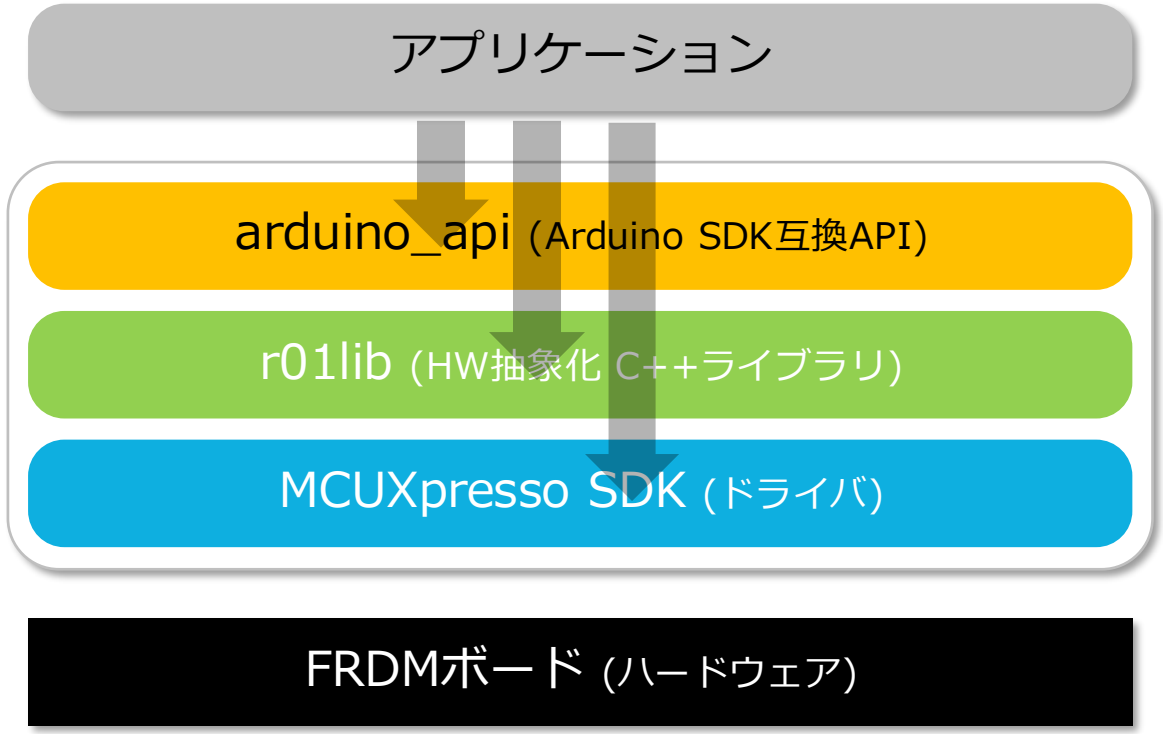
[mcx-arduino-core]:
- NXP非純正.
- FRDM-MCXマイコン基板をArduino IDE/SDKで使えるようにするもの
- **A153**，**A156**，**N236**，**N947**基板に対応
- NXPマイコンをArduinoに対応させるボードパッケージには，Arduino本家が提供している「ArduinoCore-zephyr」があるが，A153には非対応



ボードパッケージについて: 「mcx-arduino-core」 (2/2)

mcx-arduino-core:

- Arduino APIは UNO R3レベルの基本APIに対応
- 基本APIで対応できない機能は，下層の **r01lib**，**MCUXpresso SDK** のAPIで対応可能
  - マイコン内蔵周辺ブロックへのアクセスの例: SDK APIを使ったGPIO速度の最適化
  - Arduino APIにはまだ用意されてないI3Cバス・アクセス例



NXP Public

02-3

I3Cサンプルコードの動作

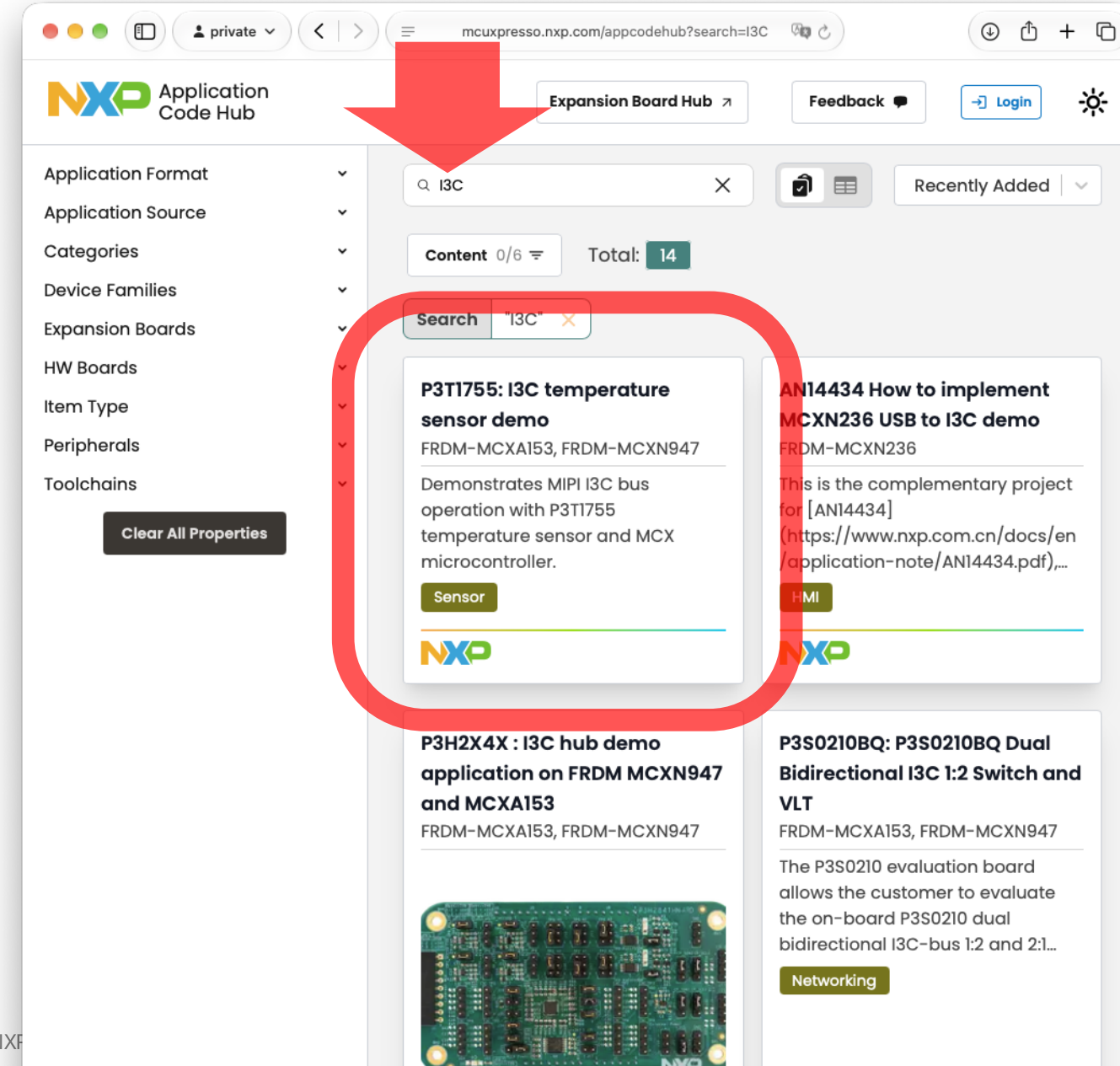


サンプルコードの入手: NXPの「Application Code Hub」から

[Application Code Hub]():
- NXP公式のサンプル/デモ・コード配布場所
- I3C関連のコードもここに
- 今回は**P3T1755**のコードを使ってみる

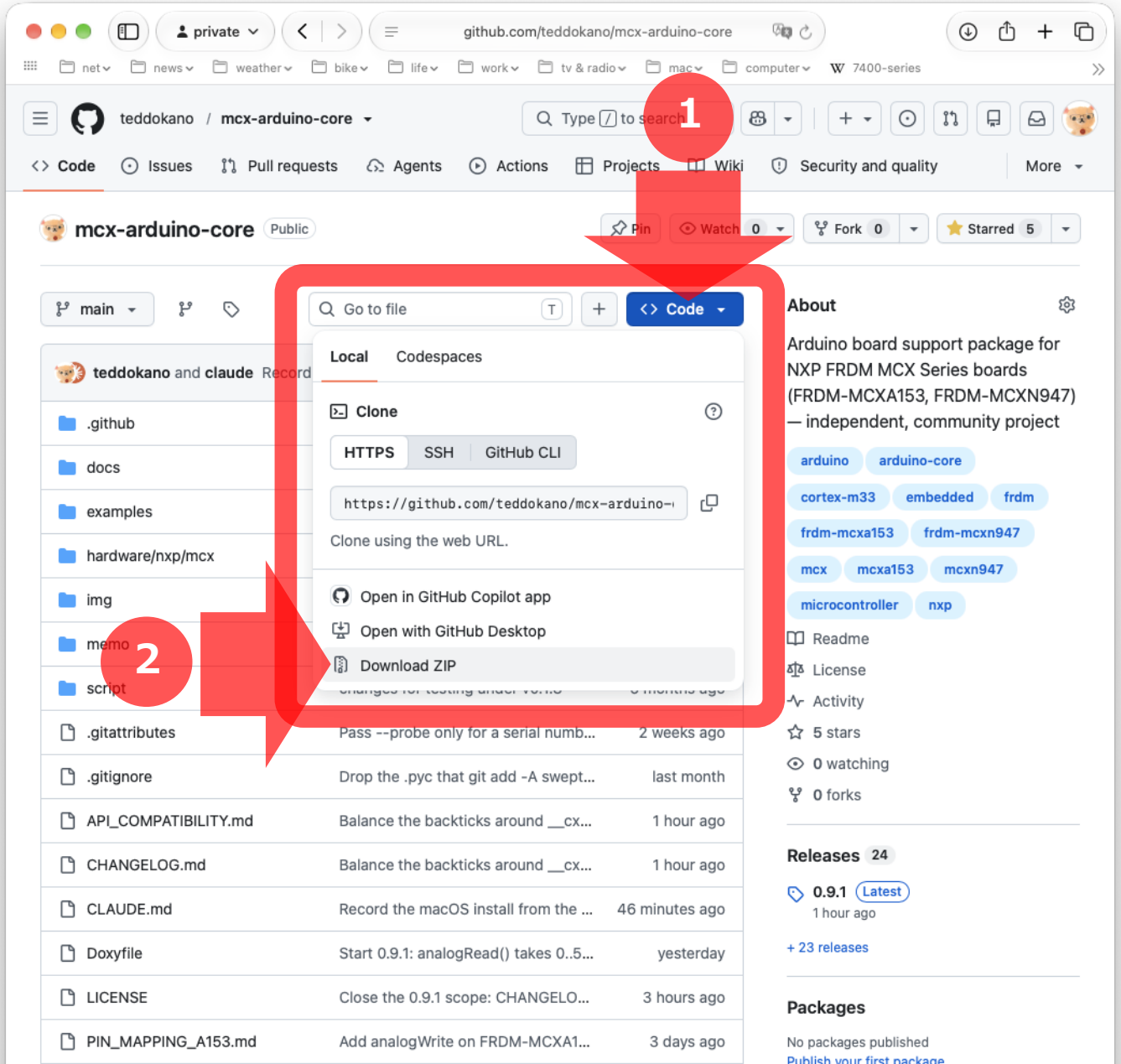
- ただし
  - このコードは**MCUXpresso IDE/VSC用**
  - Arduino IDE用には変更が必要

  - なので..
    mcx-arduino-coreのサンプルを使う



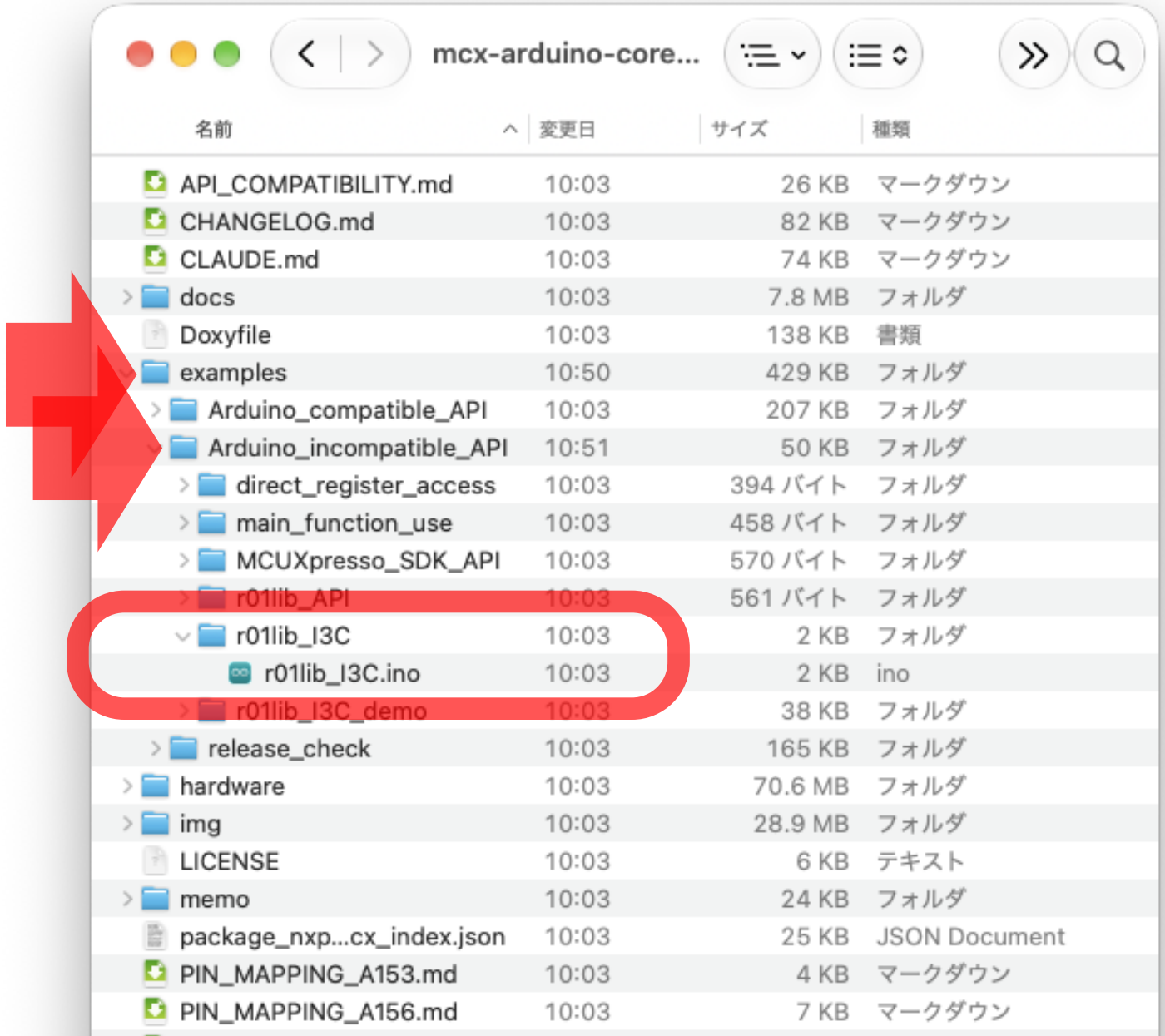
サンプルコードの入手: 「mcx-arduino-core」から(1/2)

[mcx-arduino-core]:
- リポジトリには動作確認用サンプルあり
- I3C関連のコードもここに
- Arduino非互換コード例



サンプルコードの入手: 「mcx-arduino-core」から(2/2)

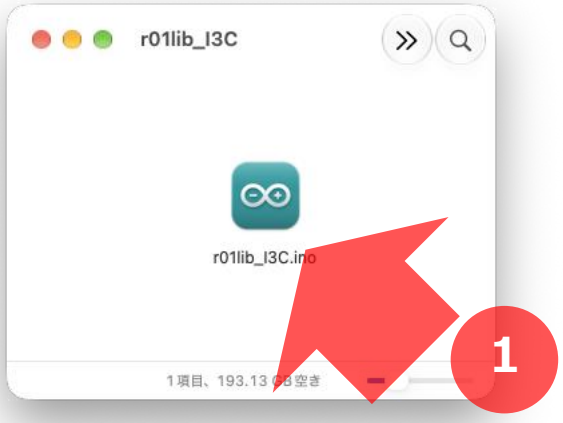
mcx-arduino-core:
- zipをほどいて
- Examples →
- Arduino_incompatible_API →
- R01lib_I3C



コードを動かしてみる

R01lib_I3C.ino

1. R01lib_I3C.ino をダブルクリック
2. ポートとボードを選択
3. アップロードボタンを押す



r01lib_I3C | Arduino IDE 2.3.10

FRDM-MCXA153 (mcx...

r01lib_I3C.ino

```
/*
 * Since this Arduino variant is including "r01lib" layer as its base class,
 * user can use r01lib APIs.
 * One thing that user should care is some definitions like pin name r01lib/soure/io.h is
 * over-ridden by arduino_io.h. The pins should be referenced like P3_13.
 * This is a sample of the using DigitalOut class.
 *
 * I3C_SDA/I3C_SCL are safe to use directly here: unlike D0–D19/A0–A5/MB_*,
 * these names are deliberately excluded from arduino_io.h's pin
 * renumbering, so they always mean the same raw r01lib value whether or
 * not <Arduino.h> has been included -- no need to spell out the physical
 * pin per board.
 */

#include <Arduino.h>

I3C i3c(I3C_SDA, I3C_SCL);  //  SDA, SCL

constexpr uint8_t static_address = 0x48;
constexpr uint8_t dynamic_address = 0x08;
uint8_t w_data[] = { 0 };
uint8_t r_data[2];


int main(void) {
  Serial.begin(115200);
  while (!Serial)
    ;

  Serial.printf("P3T1755 basic operation sample\r\n");

  status_t r1 = i3c.ccc_broadcast(CCC::BROADCAST_RSTDAA, NULL, 0);                    // Reset DAA
  status_t r2 = i3c.ccc_set(CCC::DIRECT_SETDASA, static_address, dynamic_address << 1);  // Set Dynamic Address from S1
  Serial.printf("RSTDAA status=%ld, SETDASA status=%ld\r\n", (long)r1, (long)r2);

  while (true) {
    status_t rw = i3c.write(dynamic_address, w_data, sizeof(w_data), I3C::NO_STOP);
    status_t rr = i3c.read(dynamic_address, r_data, sizeof(r_data));

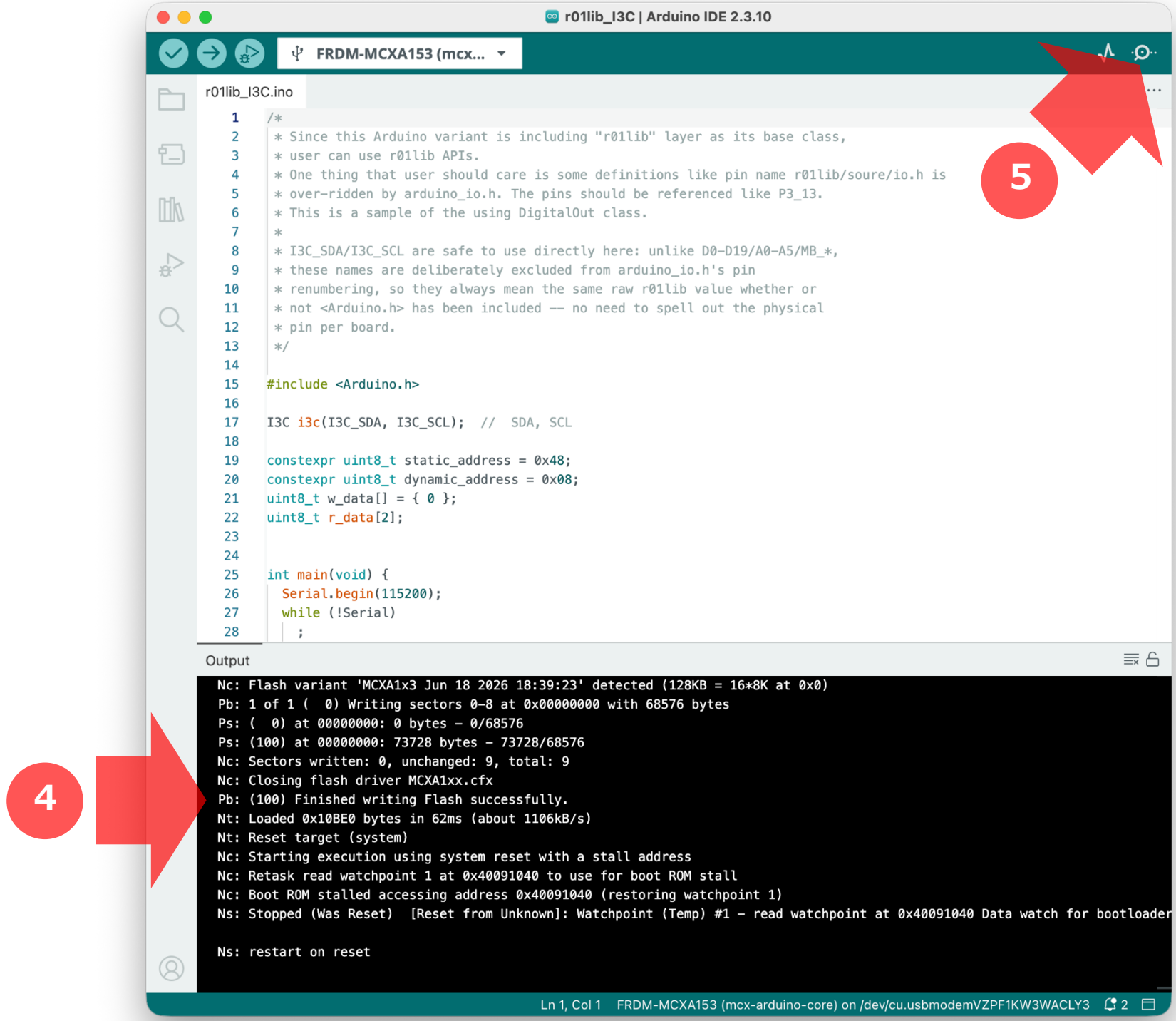
    Serial.printf("write status=%ld, read status=%ld, raw=0x%02X%02X, temp=%f\r\n",
                  (long)rw, (long)rr, r_data[0], r_data[1],
                  (((int)r_data[0]) << 8 | r_data[1]) / 256.0);
    wait(1);
  }
}
```

indexing: 2/57 Ln 1, Col 1 FRDM-MCXA153 (mcx-arduino-core) on /dev/cu.usbmodemVZPF1KW3WACLY3

コードを動かしてみる

R01lib_I3C.ino

4.アップロードが完了したら
5. シリアルモニタボタンを押す



コードを動か

R01lib_I3C.ino

6.ボーレートを115200に設定
7. シリアル出力を確認

```
 */

#include <Arduino.h>

I3C i3c(I3C_SDA, I3C_SCL);  //  SDA, SCL

constexpr uint8_t static_address = 0x48;
constexpr uint8_t dynamic_address = 0x08;
uint8_t w_data[] = { 0 };
uint8_t r_data[2];


int main(void) {
  Serial.begin(115200);
  while (!Serial)
    ;
```

Output　Serial Monitor ×

Message (Enter to send message to 'FRDM-MCXA153 (mcx-arduino-core)' on '/dev/cu.usbmodemVZPF1K...　Both NL & CR　115200 baud

```
P3T1755 basic operation sample
RSTDAA status=0, SETDASA status=0
write status=0, read status=0, raw=0x1AC0, temp=26.750000
write status=0, read status=0, raw=0x1AC0, temp=26.750000
write status=0, read status=0, raw=0x1AC0, temp=26.750000
write status=0, read status=0, raw=0x1AC0, temp=26.750000
```

6

7

温度データ

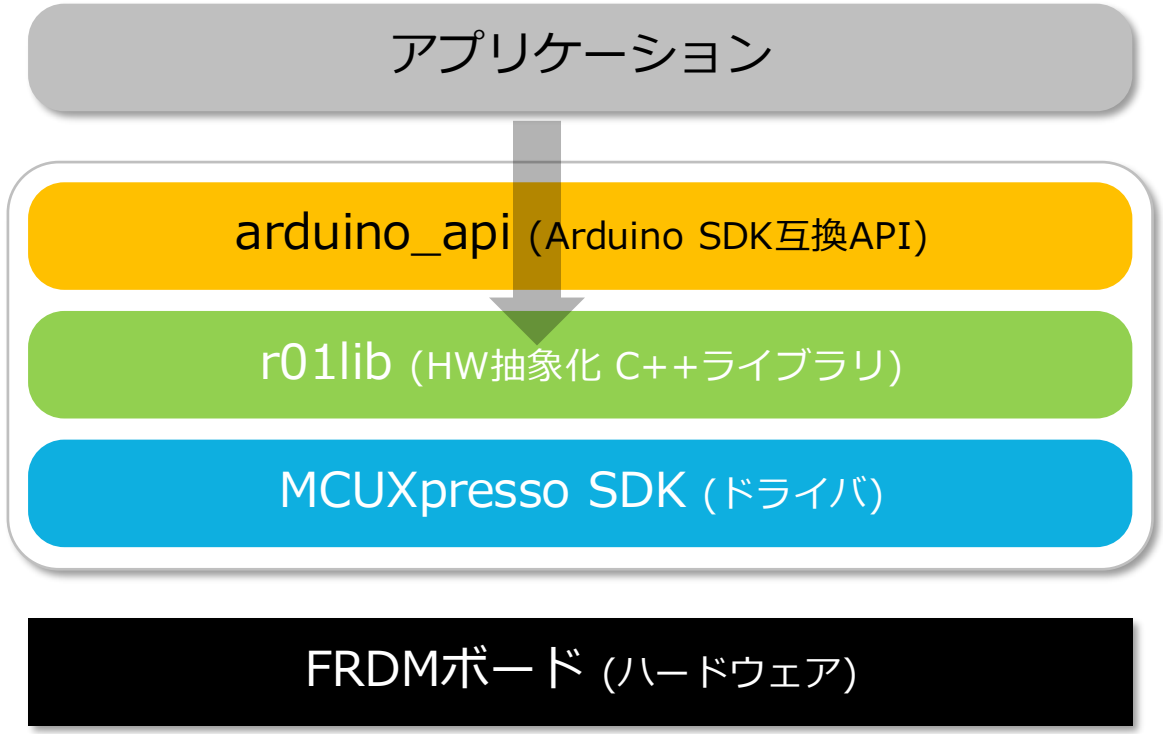
Ln 1, Col 1　FRDM-MCXA153 (mcx-arduino-core) on /dev/cu.usbmodemVZPF1KW3WACLY3　2

コード例: P3T1755とI3Cで通信する最も簡単な例 (r01lib APIを利用)

```
15  #include <Arduino.h>
16
17  I3C i3c(I3C_SDA, I3C_SCL);  //  SDA, SCL
18
19  constexpr uint8_t static_address = 0x48;
20  constexpr uint8_t dynamic_address = 0x08;
21  uint8_t w_data[] = { 0 };
22  uint8_t r_data[2];
23
24
25  int main(void) {
26    Serial.begin(115200);
27    while (!Serial)
28      ;
29
30    Serial.printf("P3T1755 basic operation sample\r\n");
31
32    status_t r1 = i3c.ccc_broadcast(CCC::BROADCAST_RSTDAA, NULL, 0);                      // Reset DAA
33    status_t r2 = i3c.ccc_set(CCC::DIRECT_SETDASA, static_address, dynamic_address << 1);  // Set Dynamic Address from Static Address
34    Serial.printf("RSTDAA status=%ld, SETDASA status=%ld\r\n", (long)r1, (long)r2);
35
36    while (true) {
37      status_t rw = i3c.write(dynamic_address, w_data, sizeof(w_data), I3C::NO_STOP);
38      status_t rr = i3c.read(dynamic_address, r_data, sizeof(r_data));
39
40      Serial.printf("write status=%ld, read status=%ld, raw=0x%02X%02X, temp=%f\r\n",
41                    (long)rw, (long)rr, r_data[0], r_data[1],
42                    (((int)r_data[0]) << 8 | r_data[1]) / 256.0);
43      wait(1);
44    }
45  }
```

Arduino APIとI3C

- **Arduino APIではI3C対応が今のところない(?)**
- APIで対応できない機能は，下層の **r01lib**，MCUXpresso SDK のAPIで対応
- このI3Cの動作はr01libのAPIを使って実装



NXP Public

コード例: P3T1755とI3Cで通信する最も簡単な例 (r01lib APIを利用)

```ino
#include <Arduino.h>

I3C i3c(I3C_SDA, I3C_SCL);  //  SDA, SCL

constexpr uint8_t static_address = 0x48;
constexpr uint8_t dynamic_address = 0x08;
uint8_t w_data[] = { 0 };
uint8_t r_data[2];


int main(void) {
  Serial.begin(115200);
  while (!Serial)
    ;

  Serial.printf("P3T1755 basic operation sample\r\n");

  status_t r1 = i3c.ccc_broadcast(CCC::BROADCAST_RSTDAA, NULL, 0);                    // Reset DAA
  status_t r2 = i3c.ccc_set(CCC::DIRECT_SETDASA, static_address, dynamic_address << 1);  // Set Dynamic Address from Static Address
  Serial.printf("RSTDAA status=%ld, SETDASA status=%ld\r\n", (long)r1, (long)r2);

  while (true) {
    status_t rw = i3c.write(dynamic_address, w_data, sizeof(w_data), I3C::NO_STOP);
    status_t rr = i3c.read(dynamic_address, r_data, sizeof(r_data));

    Serial.printf("write status=%ld, read status=%ld, raw=0x%02X%02X, temp=%f\r\n",
                  (long)rw, (long)rr, r_data[0], r_data[1],
                  (((int)r_data[0]) << 8 | r_data[1]) / 256.0);
    wait(1);
  }
}
```

I3Cコントローラを準備

ARM mbedライクなAPI

I²Cでのアドレス

I3Cでのアドレス

通信用のバッファ/データ

mcx-arduino-coreでは main() が使える

シリアル出力の準備

ダイナミック・アドレスをリセット

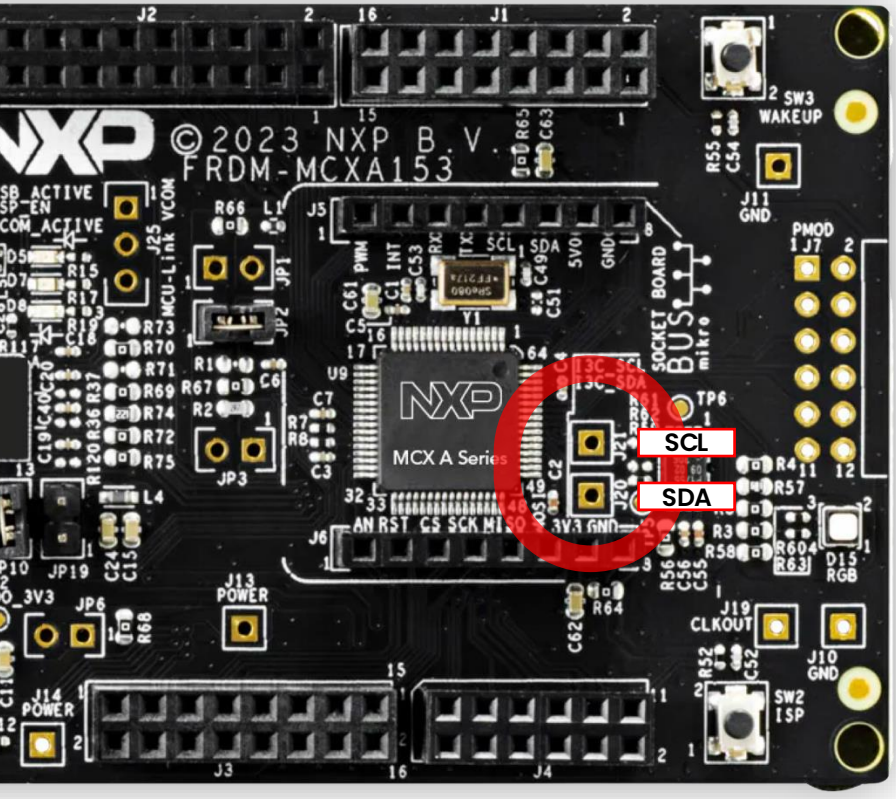
新しいアドレス付与

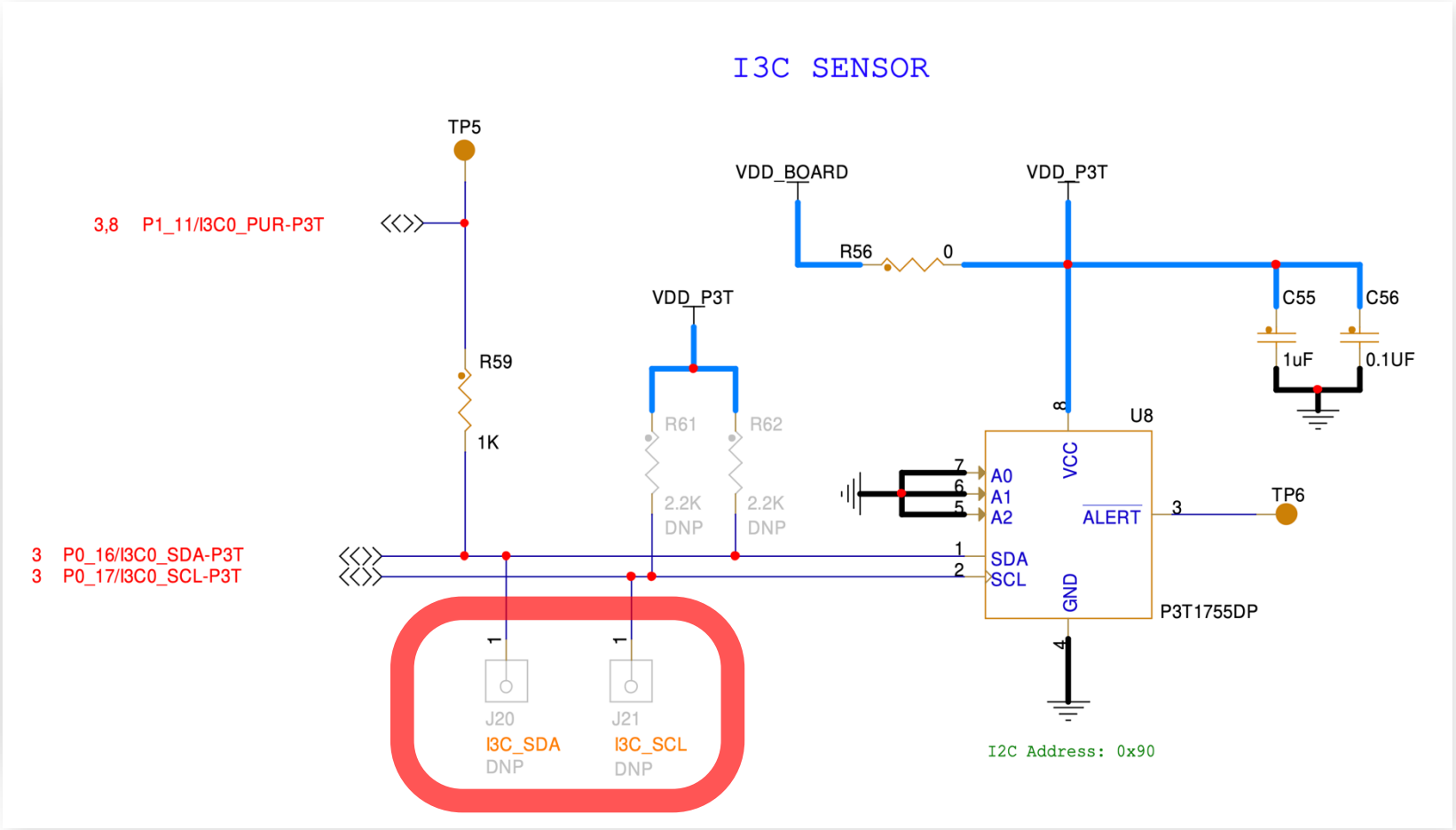
レジスタ指定のためのライト転送

2バイトのリード転送

摂氏温度に変換して表示

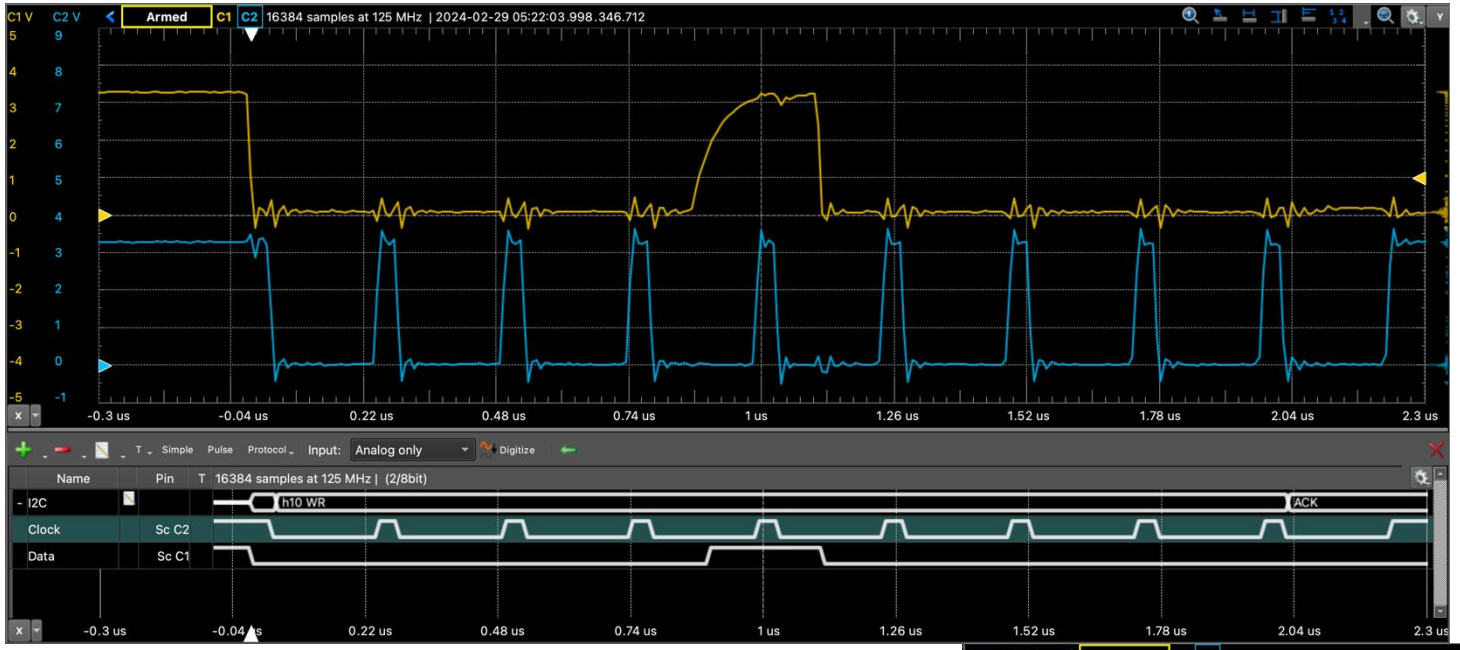
波形を見てみる: ピンの位置





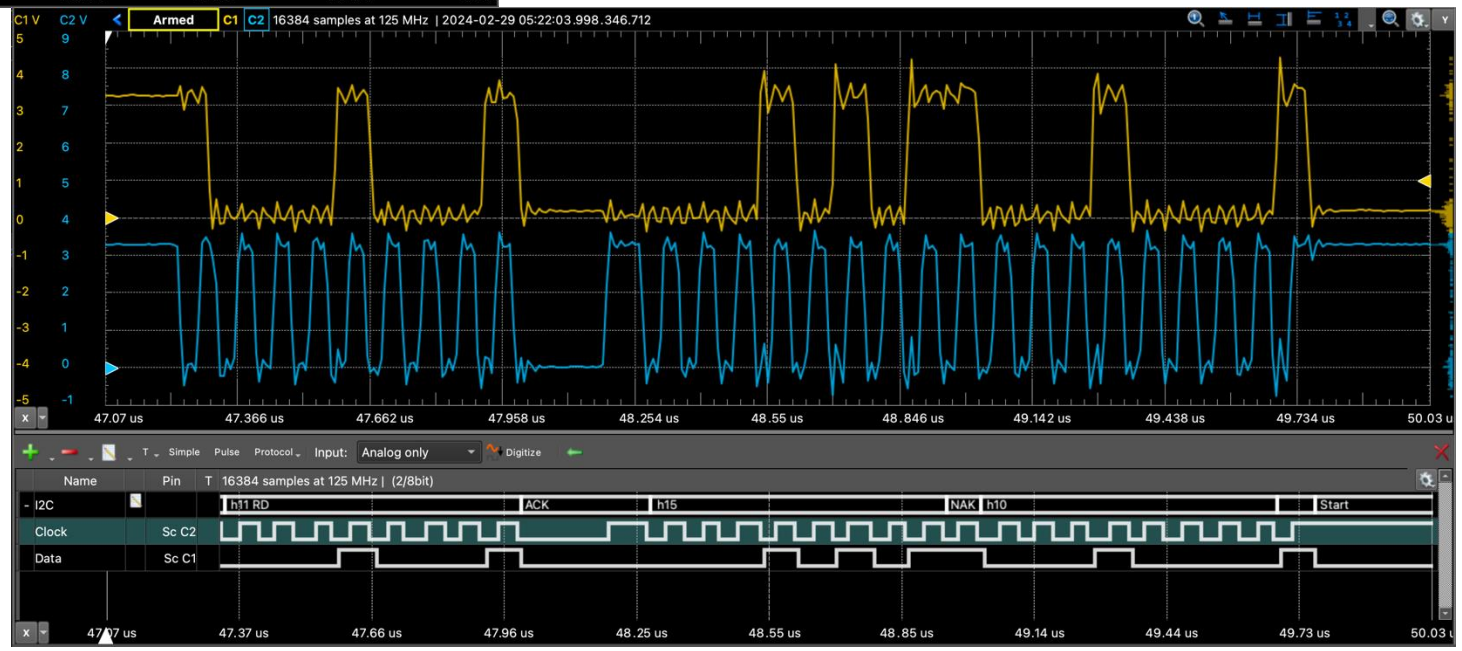
NXP Public

波形を見てみる



I3Cアドレスヘッダ (STARTコンディションに続く部分) ↑

I3Cリード転送 (リピートSTARTに続いて2バイト読み出し) →

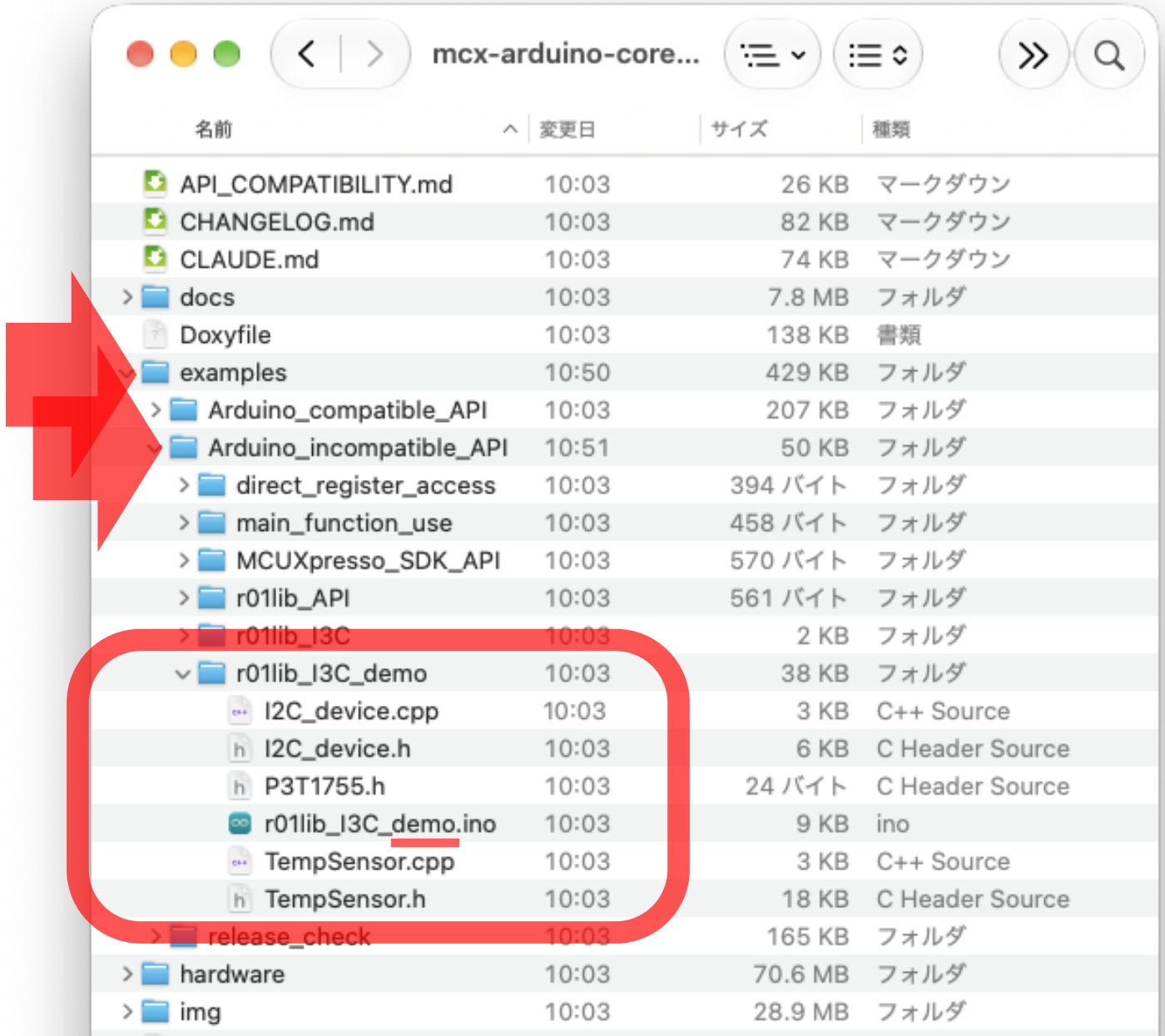


02-4

I3Cデモ

デモコードの入手: 「mcx-arduino-core」から

mcx-arduino-core:
- zipをほどいて
- Examples →
- Arduino_incompatible_API →
- R01lib_I3C_demo

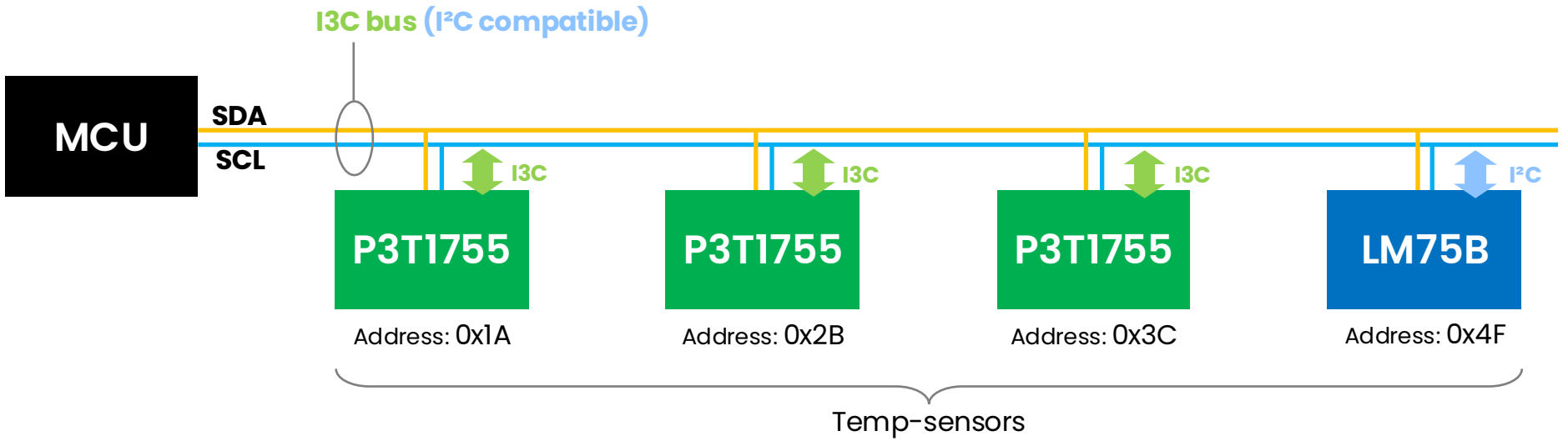


接続と動作

To try this sample, 4 temp sensors are not necessary.
It detects 0~3 I3C sensors and 0~1 I²C sensor automatically.
Temperature reading operation can be performed even if with only one I3C or I²C sensor.

- 最大3個のI3C温度センサ (P3T1755)
- 追加で1個のI²C温度センサを接続可 (LM75B)
- マイコンのI3Cバス・コントローラに接続 (I²C互換)

- 各センサから温度情報を読み出し(毎秒)
- I3CセンサはIBI(インバンド割込)を発生

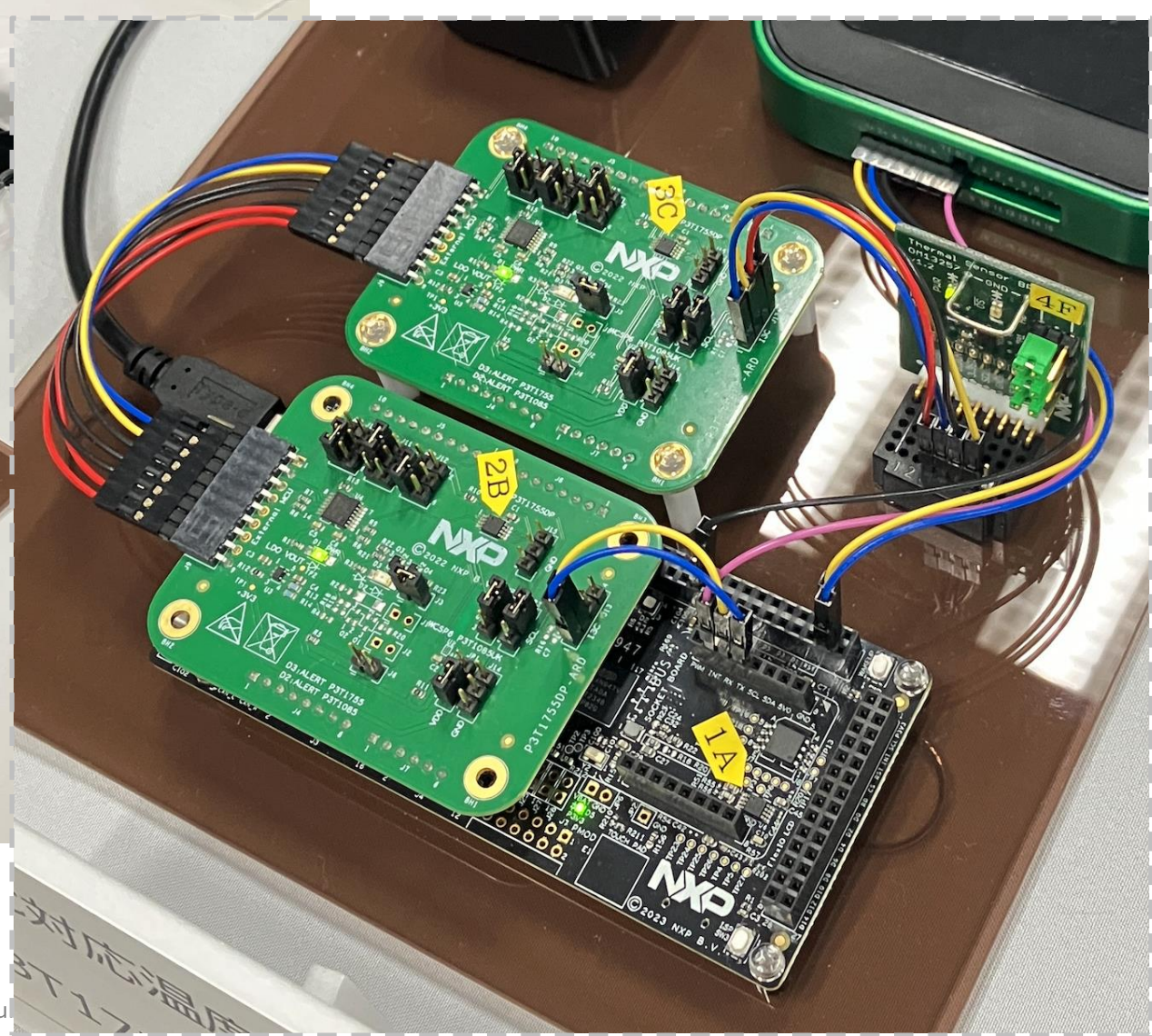


NXP Public

I²Cターゲットも加えた動作デモ

- “R01lib_I3C_demo”のデモ・セットアップ

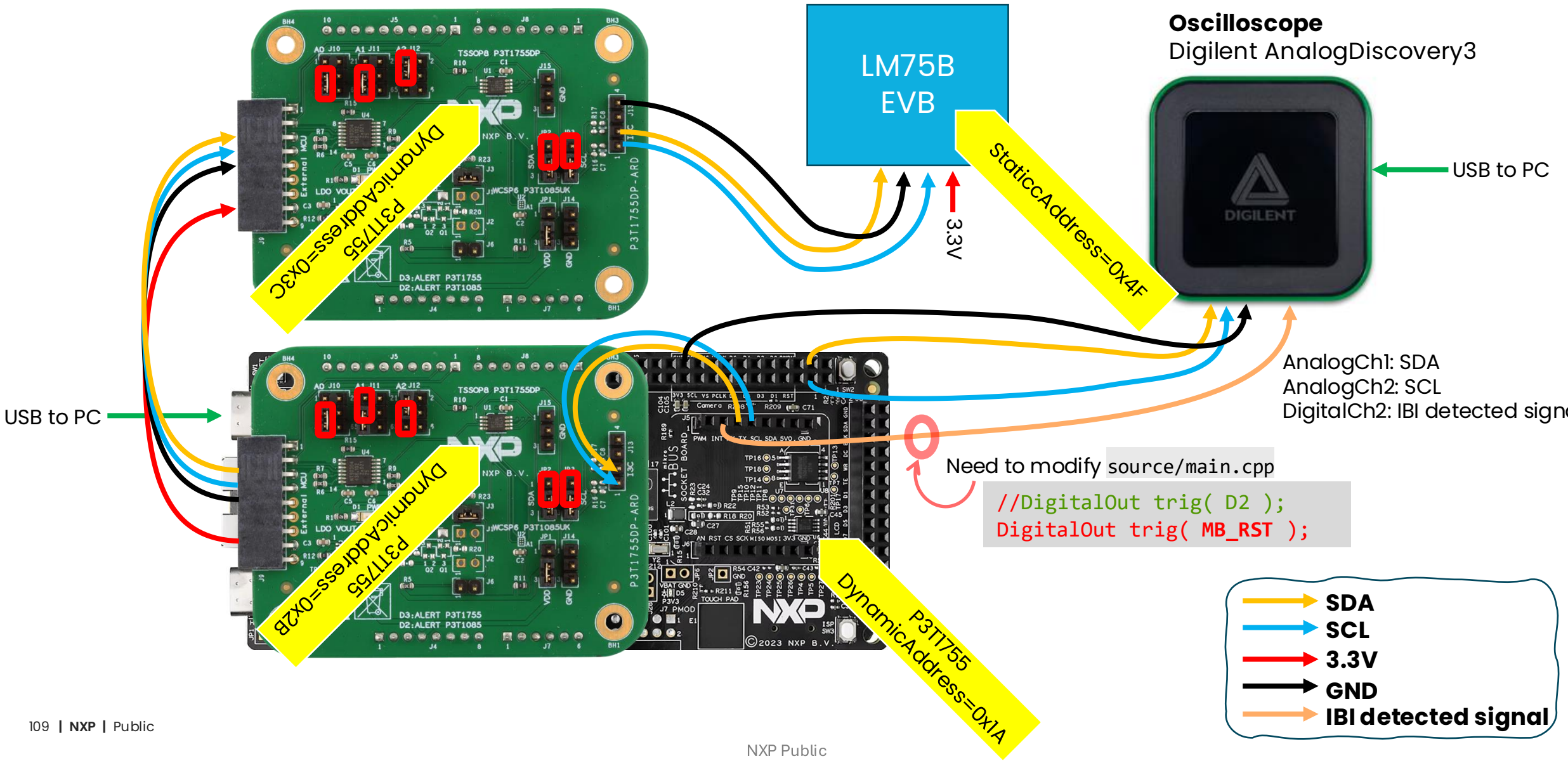
I3C demo
2 x P3T1755DP-ARD
+ LM75B-EVB
+ FRDM-MCXN947



NXP Pu

I²Cターゲットも加えた動作デモ

- “R01lib_I3C_demo”のデモ・セットアップ



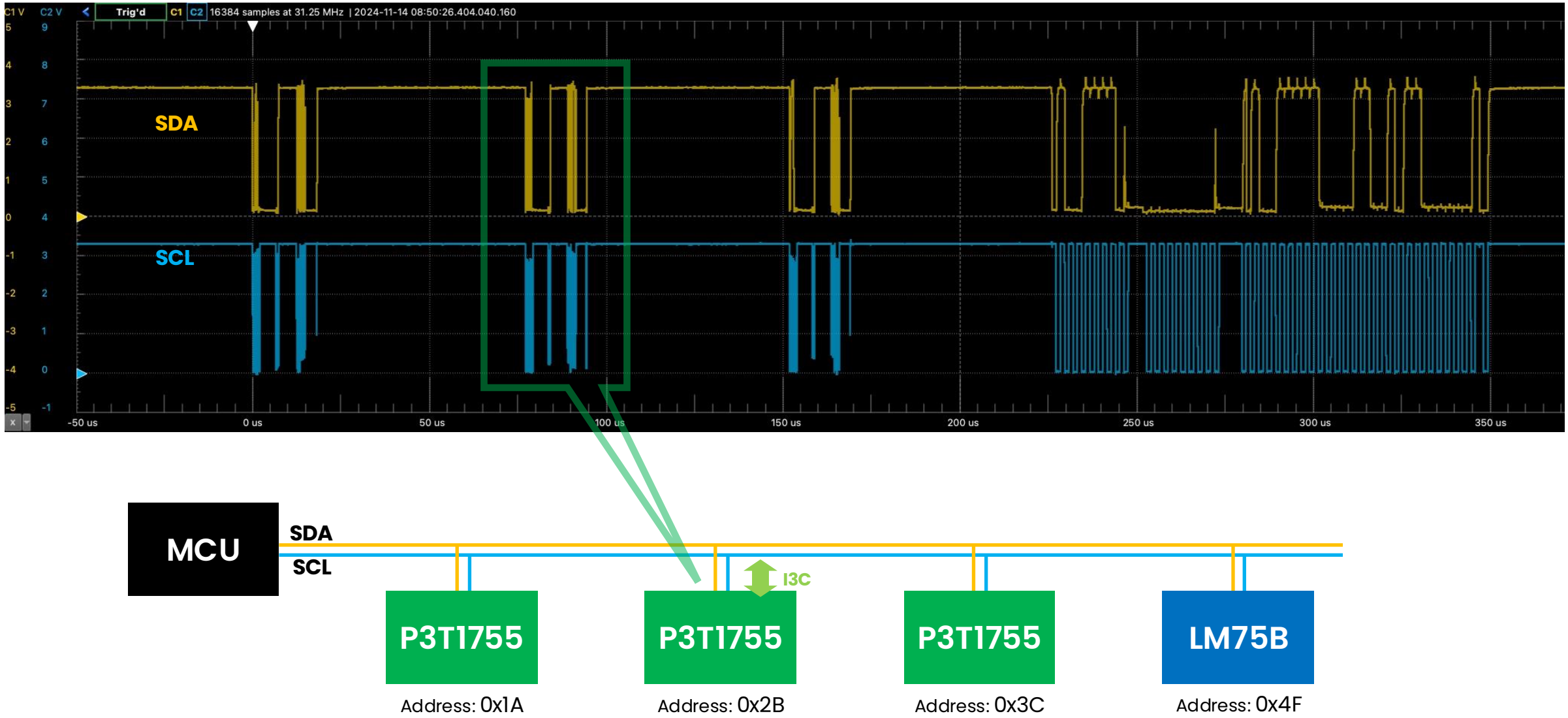
NXP Public

動作の様子 (1/4)



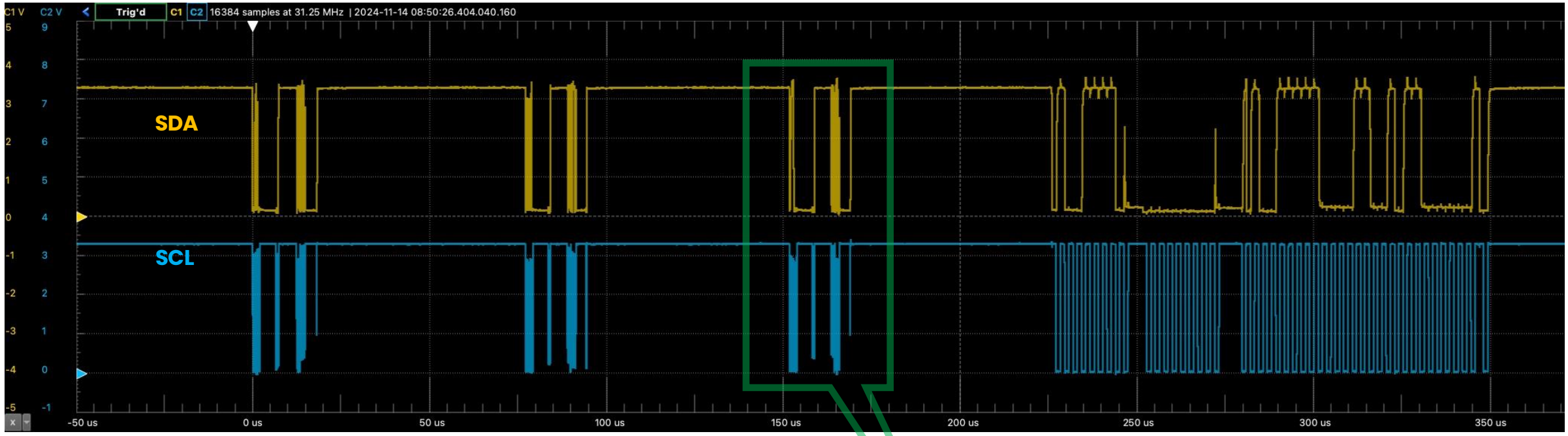
NXP Public

動作の様子 (2/4)



NXP Public

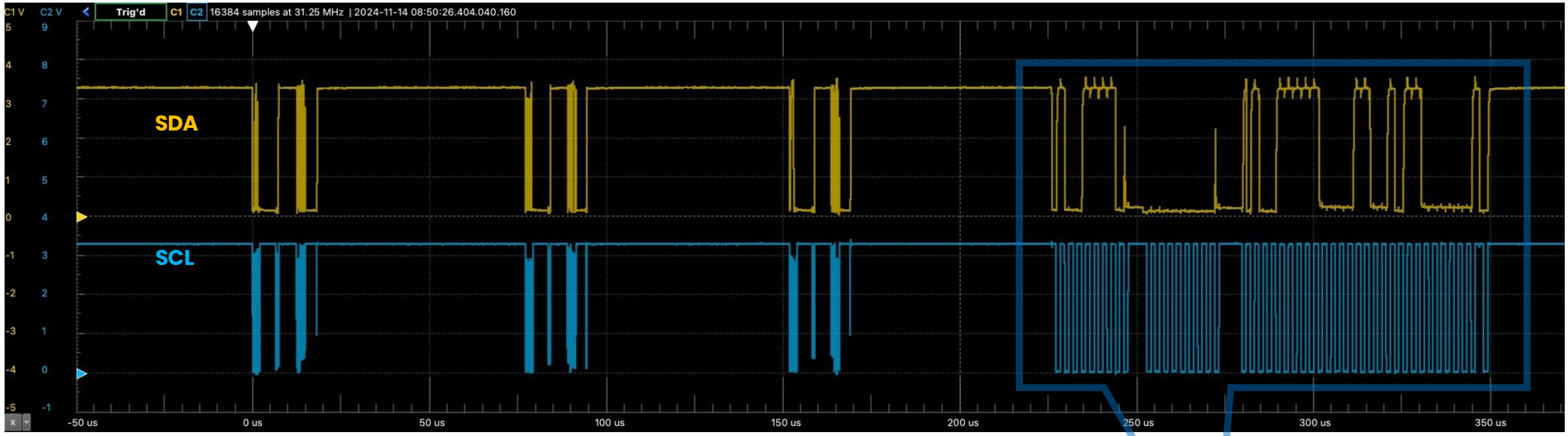
動作の様子 (3/4)

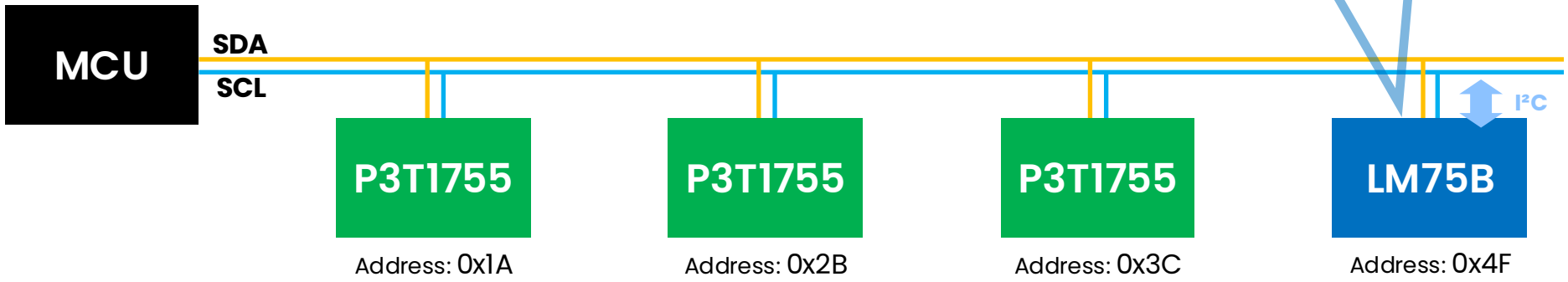




NXP Public

動作の様子 (4/4)

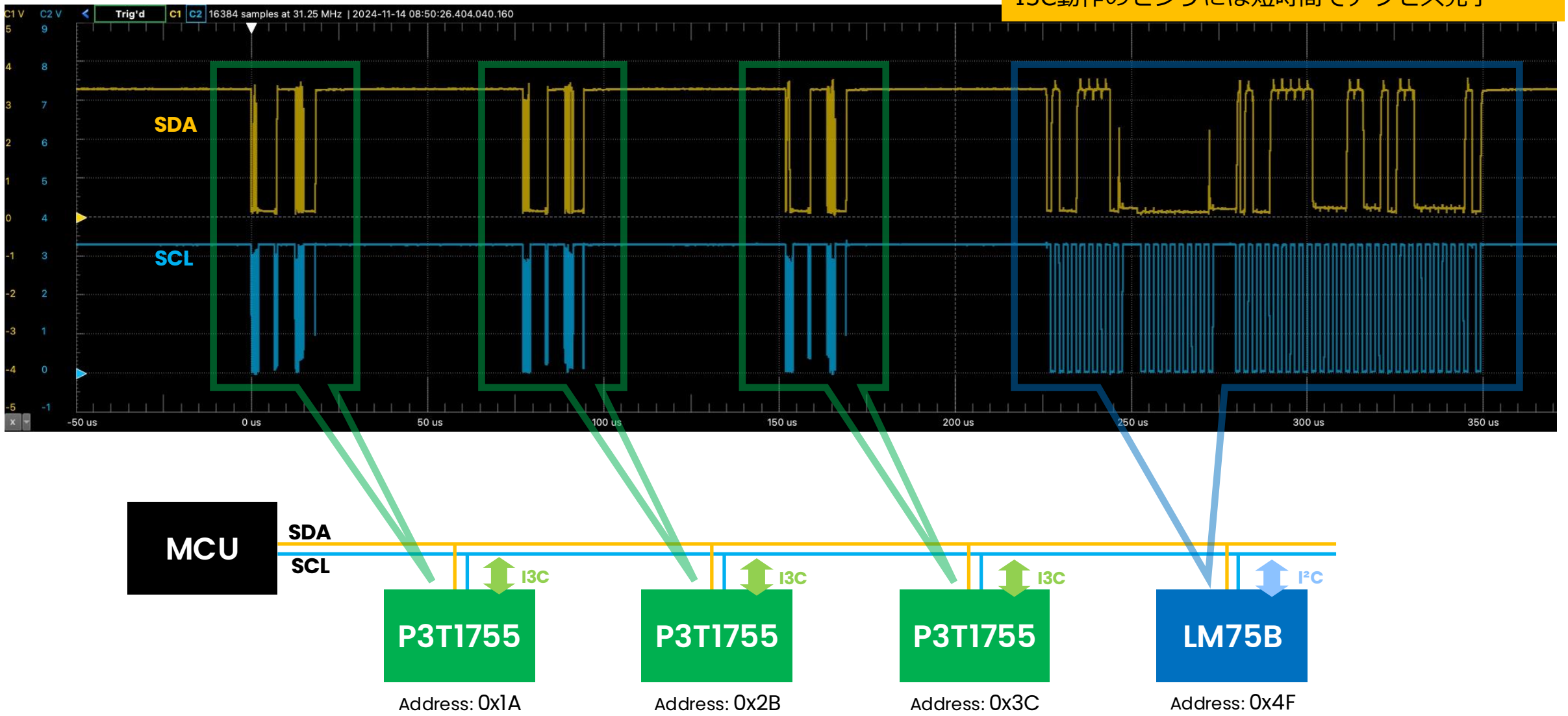




NXP Public

動作の様子

各センサのアクセスから2バイトの読み出し
I3C動作のセンサには短時間でアクセス完了

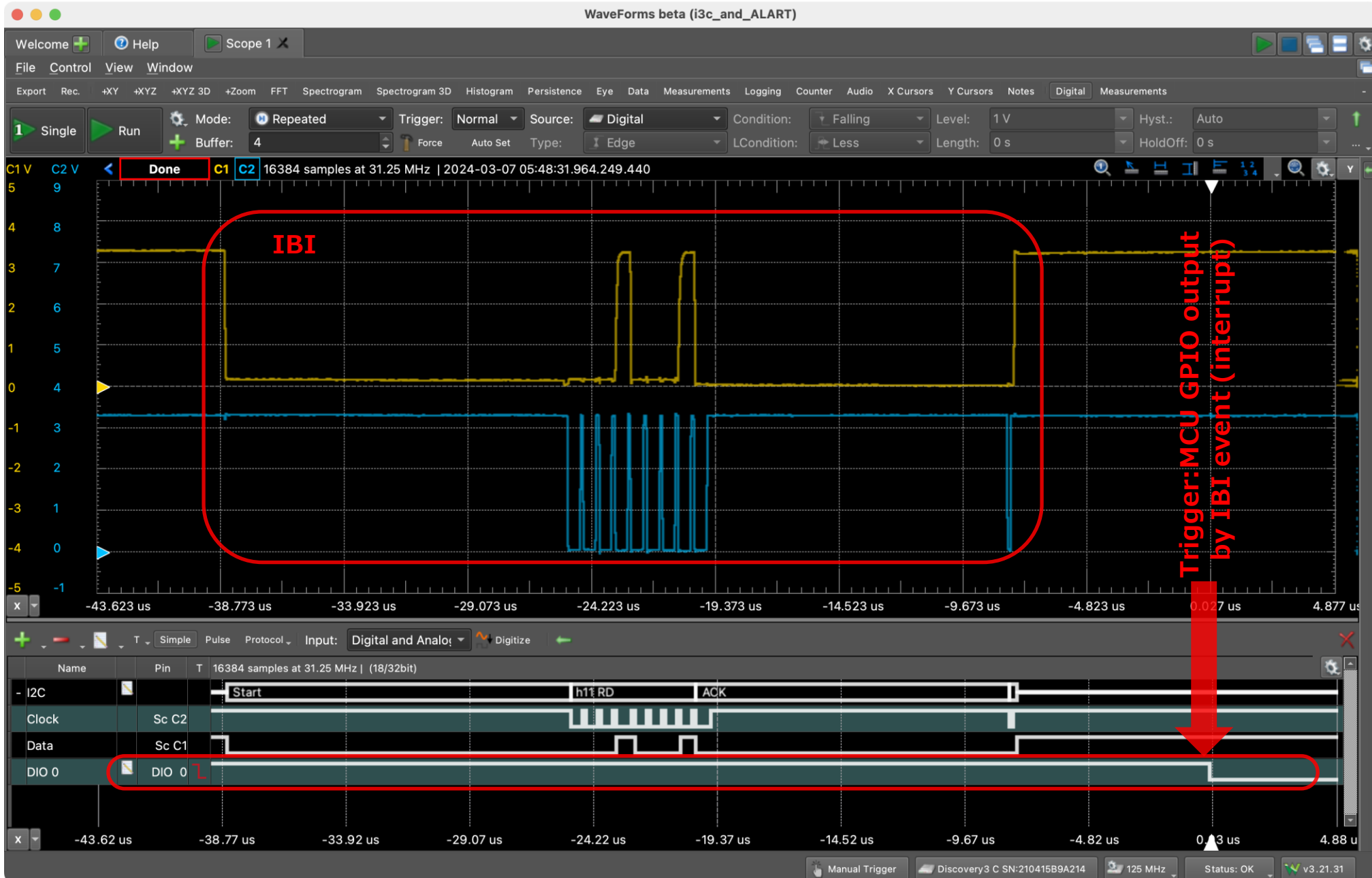


NXP Public

MCXA↔P3T1755 IBI波形 (オシロスコープのトリガのためにMCU GPIOピンからパルスを出力)

IBI

Ch1=SDA
Ch2=SCL
DIO=**D2**



コード例 – I²Cターゲットへのアクセスはどのように行われてる？

```
void setup() {
    ...
    ndev = DAA(dynamic_address_list, sizeof(dynamic_address_list));
    ...
}
...
void loop() {
    ...
    for (int i = 0; i < ndev; i++) {
        float t = sensor[i];
        ...
        Serial.printf("  %8.4f°C @%02X", t, sensor[i].address());
    }

    if (i2c_device) {
        i3c.mode(I3C::I2C_MODE);
        Serial.printf("  %7.3f°C @%02X(I2C)", (float)lm75b, lm75b.address());
        i3c.mode(I3C::I3C_MODE);
    }
    ...
    delay(WAIT_MS);
}
```

全てのI3Cターゲットを I3C通信状態に

I3Cセンサを読む

I²Cターゲットにアクセスするため モードを変更

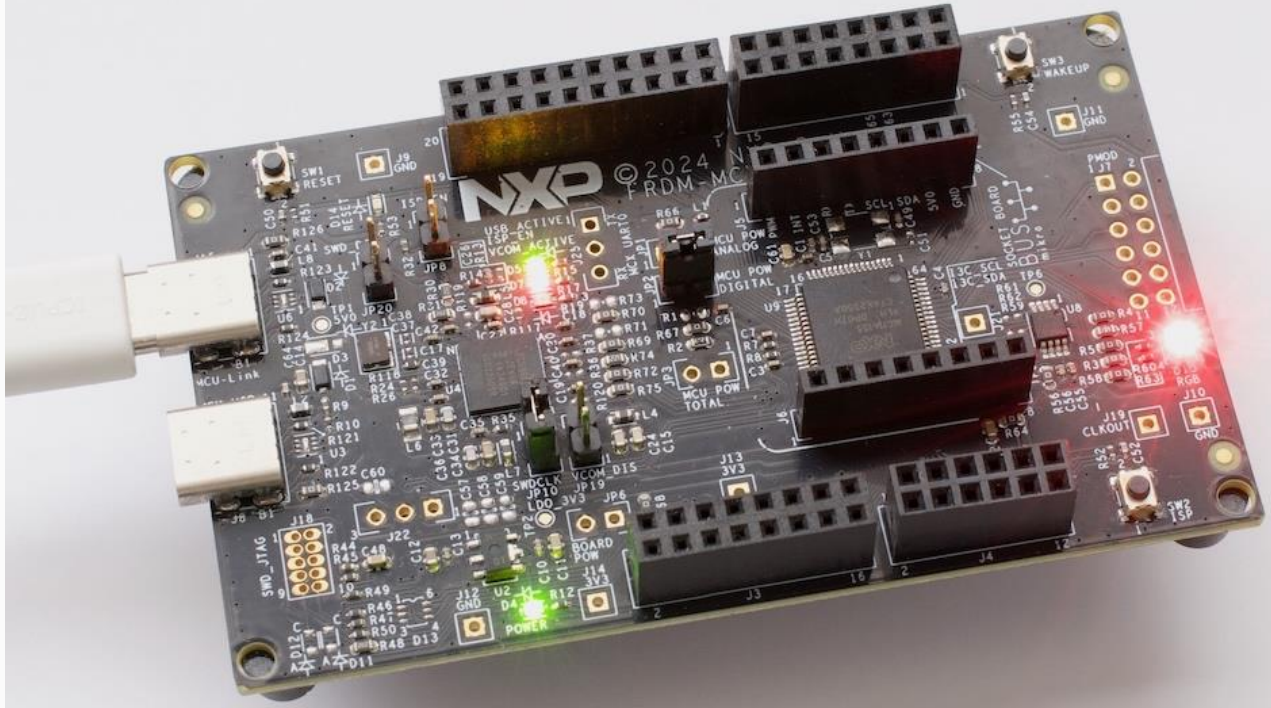
I²Cセンサを読む

I3Cモードに戻す

NXP Public

03

まとめ


NXP
SW3 WAKEUP
SW1 RESET
SW2 ISP
POWER
MCU-Link
SWD_JTAG
BOARD POW
MCU POW TOTAL
MCU POW DIGITAL
MCU POW ANALOG
CLKOUT
SOCKET BOARD

まとめ

NXPのマイコン: MCXシリーズ

- NXPの(歴史的)マイコンファミリが統合→ **MCX**
- **先進機能を統合**したマイコン
- 基本機能のAシリーズも**I3C**に対応
- Arduino型の評価基板: **FRDM-MCX**シリーズ

- 純正フリー開発環境
  - MCUXpresso IDE / MCUXpresso VSC

I3Cバス

- スマホ，PC/サーバ．さらに汎用でも普及が進む次世代シリアルバス
- I²CとSPIのいいとこ取り
- **2線式**，実効レート10Mbps～30Mbps.
- システムの**低コスト化**，**低消費電力化**に貢献
- **I²Cバス互換**

MCXはArduino IDEに対応

- 「ちょっと動かしてみたい」時に，MCXをArduino IDEで
- Arduino SDKの**基本APIに対応**
- 対応ボードは**A153**，**A156**，**N236**，**N947**
- **MCUXpresso SDK**や**中間層抽象化ライブラリも活用可能**

できたこと

- I3Cなど，先進機能搭載のMCXマイコンと評価をはじめやすいFRDM-MCXボード
- この↑↑組み合わせでI3Cを体験してみた
- 次は？
  - 実際に波形を確認したり，他のI3C/I²Cターゲットをつないでみたり..

NXP Public



nxp.com

| **Public** | NXP and the NXP logo are trademarks of NXP B.V. All other product or service names are the property of their respective owners. © 2026 NXP B.V. Version 3.0.